



404 Not Found

Truong Minh Hieu
Nguyen Tri Khoa
Le Nguyen Quoc Binh

2026-10-10

- 1 Contest
- 2 Mathematics
- 3 Numerical
- 4 Number theory
- 5 Combinatorial
- 6 Data structures
- 7 Graph
- 8 Geometry
- 9 Strings
- 10 Various

Contest (1)

Contest Template

```
#include <bits/stdc++.h>
using namespace std;

#define ll long long
#define ull unsigned long long
#define pii pair<int,int>
#define pll pair<long long, long long>
#define fi first
#define se second
#define all(a) (a).begin(), (a).end()
#define sz(a) (int) (a).size()
#define pb push_back
#define lwb lower_bound
#define upb upper_bound
int main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0);
    cout.tie(0);
}

Compare Two Solutions (Stress Testing)
Description: Windows comparator: generate input, run two solutions, then
compare their output files with fc. Stops at the first reported difference;
leaves files for inspection. The code below is the original personal comparator
with whitespace formatting only.
Usage:      Compile this as a separate program and run it in the
solutions' directory.
Set NAME and NTEST; compile the solutions as problem.exe and
problem-2.exe.
Both read problem.INP; the first writes problem.OUT, the second
problem.ANS (via freopen).
Replace the generator under Generate tests here with valid
inputs for your problem.
<bits/stdc++.h>

#define ll long long
using namespace std;
const string NAME = "problem";
const int NTEST = 500;
```

```
1 mt19937 rd(chrono::steady_clock::now().time_since_epoch().count
());
#define rand rd
1 long long Rand(long long l, long long h) { return l + rd() % (h
- l + 1); }
2 int main() {
    srand(time(NULL));
    for (int iTest = 1; iTest <= NTEST; iTest++) {
5 ofstream inp((NAME + ".INP").c_str());
        ll n = Rand(49, 50);
7 inp << n;
        inp.close();
        system((NAME + ".exe").c_str());
        system((NAME + "-2.exe").c_str());
8 if (system(("fc " + NAME + ".OUT " + NAME + ".ANS").c_str()
) != 0) {
13 cout << "Test " << iTest << ": WRONG!\n";
        return 0;
    }
19 cout << "Test " << iTest << ": CORRECT!\n";
    }
23 return 0;
    }
```

Troubleshooting Checklist

Pre-submit:

Write a few simple test cases if sample is not enough.
Are time limits close? If so, generate max cases.
Is the memory usage fine?
Could anything overflow?
Make sure to submit the right file.

Wrong answer:

Print your solution! Print debug output, as well.
Are you clearing all data structures between test cases?
Can your algorithm handle the whole range of input?
Read the full problem statement again.
Do you handle all corner cases correctly?
Have you understood the problem correctly?
Any uninitialized variables?
Any overflows?
Confusing N and M, i and j, etc.?
Are you sure your algorithm works?
What special cases have you not thought of?
Are you sure the STL functions you use work as you think?
Add some assertions, maybe resubmit.
Create some testcases to run your algorithm on.
Go through the algorithm for a simple case.
Go through this list again.
Explain your algorithm to a teammate.
Ask the teammate to look at your code.
Go for a small walk, e.g. to the toilet.
Is your output format correct? (including whitespace)
Rewrite your solution from the start or let a teammate do it.

Runtime error:

Have you tested all corner cases locally?
Any uninitialized variables?
Are you reading or writing outside the range of any vector?
Any assertions that might fail?
Any possible division by 0? (mod 0 for example)
Any possible infinite recursion?
Invalidated pointers or iterators?
Are you using too much memory?
Debug with resubmits (e.g. remapped signals, see Various).

Time limit exceeded:

Do you have any possible infinite loops?
What is the complexity of your algorithm?

Are you copying a lot of unnecessary data? (References)
How big is the input and output? (consider scanf)
Avoid vector, map. (use arrays/unordered_map)
What do your teammates think about your algorithm?

Memory limit exceeded:
What is the max amount of memory your algorithm should need?
Are you clearing all data structures between test cases?

Mathematics (2)

2.1 Recurrences

Use the characteristic polynomial below to derive a closed form for a constant-coefficient homogeneous recurrence; initial values determine the constants d_i . For computing a large index modulo an integer directly, use LinearRecurrence in the Numerical chapter. If $a_n = c_1a_{n-1} + \cdots + c_ka_{n-k}$, and $r_1, \dots, r_k$ are distinct roots of $x^k - c_1x^{k-1} - \cdots - c_k$, there are $d_1, \dots, d_k$ s.t.

$$a_n = d_1r_1^n + \cdots + d_kr_k^n.$$

Non-distinct roots r become polynomial factors, e.g.
 $a_n = (d_1n + d_2)r^n$.

2.2 Geometry

2.2.1 Triangles

Side lengths: a, b, c

Semiperimeter: $p = \frac{a + b + c}{2}$

Area: $A = \sqrt{p(p-a)(p-b)(p-c)}$

Circumradius: $R = \frac{abc}{4A}$

Inradius: $r = \frac{A}{p}$

Length of median (divides triangle into two equal-area triangles):
 $m_a = \frac{1}{2}\sqrt{2b^2 + 2c^2 - a^2}$

Length of bisector (divides angles in two):

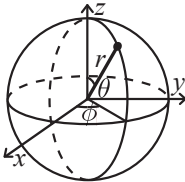
$$s_a = \sqrt{bc \left[1 - \left(\frac{a}{b+c} \right)^2 \right]}$$

Law of sines: $\frac{\sin \alpha}{a} = \frac{\sin \beta}{b} = \frac{\sin \gamma}{c} = \frac{1}{2R}$

Law of cosines: $a^2 = b^2 + c^2 - 2bc \cos \alpha$

Law of tangents: $\frac{a+b}{a-b} = \frac{\tan \frac{\alpha+\beta}{2}}{\tan \frac{\alpha-\beta}{2}}$

2.2.2 Spherical coordinates



2.3 Derivatives/Integrals

$$\begin{aligned}x &= r \sin \theta \cos \phi & r &= \sqrt{x^2 + y^2 + z^2} \\ y &= r \sin \theta \sin \phi & \theta &= \arccos(z/\sqrt{x^2 + y^2 + z^2}) \\ z &= r \cos \theta & \phi &= \operatorname{atan2}(y, x)\end{aligned}$$

$$\begin{aligned}\frac{d}{dx} \arcsin x &= \frac{1}{\sqrt{1-x^2}} & \frac{d}{dx} \arccos x &= -\frac{1}{\sqrt{1-x^2}} \\ \frac{d}{dx} \tan x &= 1 + \tan^2 x & \frac{d}{dx} \arctan x &= \frac{1}{1+x^2} \\ \int \tan ax &= -\frac{\ln|\cos ax|}{a} & \int x \sin ax &= \frac{\sin ax - ax \cos ax}{a^2} \\ \int e^{-x^2} &= \frac{\sqrt{\pi}}{2} \operatorname{erf}(x) & \int x e^{ax} dx &= \frac{e^{ax}}{a^2} (ax - 1)\end{aligned}$$

Integration by parts:

$$\int_a^b f(x)g(x)dx = [F(x)g(x)]_a^b - \int_a^b F(x)g'(x)dx$$

2.4 Sums

$$c^a + c^{a+1} + \cdots + c^b = \frac{c^{b+1} - c^a}{c - 1}, c \neq 1$$

$$1^2 + 2^2 + 3^2 + \cdots + n^2 = \frac{n(2n + 1)(n + 1)}{6}$$

$$1^3 + 2^3 + 3^3 + \cdots + n^3 = \frac{n^2(n + 1)^2}{4}$$

$$1^4 + 2^4 + 3^4 + \cdots + n^4 = \frac{n(n + 1)(2n + 1)(3n^2 + 3n - 1)}{30}$$

2.5 Probability theory

Let X be a discrete random variable with probability $p_X(x)$ of assuming the value x . It will then have an expected value (mean) $\mu = \mathbb{E}(X) = \sum_x x p_X(x)$ and variance $\sigma^2 = V(X) = \mathbb{E}(X^2) - (\mathbb{E}(X))^2 = \sum_x (x - \mathbb{E}(X))^2 p_X(x)$ where σ is the standard deviation. If X is instead continuous it will have a probability density function $f_X(x)$ and the sums above will instead be integrals with $p_X(x)$ replaced by $f_X(x)$.

Expectation is linear:

$$\mathbb{E}(aX + bY) = a\mathbb{E}(X) + b\mathbb{E}(Y)$$

For independent X and Y ,

$$V(aX + bY) = a^2V(X) + b^2V(Y).$$

Polynomial

2.5.1 Discrete distributions

Binomial distribution

The number of successes in n independent yes/no experiments, each which yields success with probability p is $\operatorname{Bin}(n, p)$, $n = 1, 2, \dots$, $0 \leq p \leq 1$.

$$p(k) = \binom{n}{k} p^k (1 - p)^{n - k}$$

$$\mu = np, \sigma^2 = np(1 - p)$$

$\operatorname{Bin}(n, p)$ is approximately $\operatorname{Po}(np)$ for small p .

First success distribution

The number of trials needed to get the first success in independent yes/no experiments, each which yields success with probability p is $\operatorname{Fs}(p)$, $0 \leq p \leq 1$.

$$p(k) = p(1 - p)^{k - 1}, k = 1, 2, \dots$$

$$\mu = \frac{1}{p}, \sigma^2 = \frac{1 - p}{p^2}$$

Poisson distribution

The number of events occurring in a fixed period of time t if these events occur with a known average rate κ and independently of the time since the last event is $\operatorname{Po}(\lambda)$, $\lambda = t\kappa$.

$$p(k) = e^{-\lambda} \frac{\lambda^k}{k!}, k = 0, 1, 2, \dots$$

$$\mu = \lambda, \sigma^2 = \lambda$$

2.5.2 Continuous distributions

Uniform distribution

If the probability density function is constant between a and b and 0 elsewhere it is $\operatorname{U}(a, b)$, $a < b$.

$$f(x) = \begin{cases} \frac{1}{b - a} & a < x < b \\ 0 & \text{otherwise} \end{cases}$$

$$\mu = \frac{a + b}{2}, \sigma^2 = \frac{(b - a)^2}{12}$$

Exponential distribution

The time between events in a Poisson process is $\operatorname{Exp}(\lambda)$, $\lambda > 0$.

$$f(x) = \begin{cases} \lambda e^{-\lambda x} & x \geq 0 \\ 0 & x < 0 \end{cases}$$

$$\mu = \frac{1}{\lambda}, \sigma^2 = \frac{1}{\lambda^2}$$

Normal distribution

Most real random values with mean μ and variance σ^2 are well described by $\mathcal{N}(\mu, \sigma^2)$, $\sigma > 0$.

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x - \mu)^2}{2\sigma^2}}$$

For independent $X_1 \sim \mathcal{N}(\mu_1, \sigma_1^2)$ and $X_2 \sim \mathcal{N}(\mu_2, \sigma_2^2)$,

$$aX_1 + bX_2 + c \sim \mathcal{N}(a\mu_1 + b\mu_2 + c, a^2\sigma_1^2 + b^2\sigma_2^2)$$

Numerical (3)

3.1 Polynomials and recurrences

Use Polynomial for known coefficients, PolyRoots for real zeros, and PolyInterpolate to recover coefficients from sampled values. LinearRecurrence computes a distant sequence term modulo an integer; its coefficients describe previous terms, not powers of x . These snippets are copied separately: Polynomial and LinearRecurrence both name a type `Poly`; rename the latter alias if combining them.

Polynomial

Description: Stores a real polynomial and supports evaluation, differentiation and division by a linear factor. Use for evaluating formulas, finding slopes/critical points, or removing a known root before solving the remaining polynomial. Coefficients `coeff[i]` multiply x^i (constant term first): 2,-3,1 means $2 - 3x + x^2$. `p(x)` returns its value using Horner's rule. `diff` changes `p` to its derivative; copy `p` first if the original is still needed. `divroot(r)` divides by `x-r` and discards the remainder, so normally call it only for a known root. Use a nonempty coefficient vector; after `diff`, constants become empty. Trim zero leading coefficients and handle constants/zero polynomials before `polyRoots`.

Usage: `Poly p{{2,-3,1}}; double value=p(3); // 2`
`Poly derivative=p; derivative.diff(); // {-3,2};`
`derivative(3)=3`
`Poly quotient=p; quotient.divroot(1); // {-2,1};`
`p(x)=(x-1)(x-2)`
Time: $\mathcal{O}(d)$ per operation, where d is the degree; $\mathcal{O}(d)$ storage.

```
struct Poly {
    vector<double> coeff;
    double operator() (double x) const {
        double value = 0;
        for (int i = sz(coeff); i--;) (value *= x) += coeff[i];
        return value;
    }
    void diff() {
        for (int i = 1; i < (sz(coeff)); ++i) coeff[i - 1] = i *
            coeff[i];
        coeff.pop_back();
    }
    void divroot(double root) {
        double carry = coeff.back(), oldCoeff;
        coeff.back() = 0;
        for (int i = sz(coeff) - 1; i--;)
            oldCoeff = coeff[i], coeff[i] = coeff[i + 1] * root +
                carry, carry = oldCoeff;
        coeff.pop_back();
    }
};
```

Polynomial Roots

Description: Numerically finds real solutions of $p(x)=0$; does not find complex roots. Use for polynomial equation solving or intersections of two curves (solve their difference). For extrema of p , solve its derivative and also check interval endpoints; this is not an optimizer by itself. Returned values are approximate doubles, not exact algebraic numbers. Uses derivative roots to split the line into monotone pieces, then bisects sign changes. Use a nonconstant polynomial with nonzero leading coefficient and coefficients in increasing degree order. Choose bounds covering the roots of interest; this implementation may also return candidates outside those bounds. Repeated roots without a sign change are not guaranteed and need separate handling (test derivative roots with a tolerance). Filter results to your bounds.

Usage: auto roots=polyRoots(Poly{{2,-3,1}},-10,10);
// approximately {1,2}; keep only roots inside the desired interval
Poly p{{2,-3,1}}, derivative=p; derivative.diff();
auto critical=polyRoots(derivative,-10,10); // {1.5}, where $p'(x)=0$
Time: $\mathcal{O}(n^2 \log(1/\epsilon))$

```
"Polynomial.h"
vector<double> polyRoots(Poly poly, double minX, double maxX) {
    if (sz(poly.coeff) == 2) return {-poly.coeff[0] / poly.coeff[1]};
    vector<double> roots;
    Poly derivative = poly;
    derivative.diff();
    auto criticalPoints = polyRoots(derivative, minX, maxX);
    criticalPoints.push_back(minX - 1);
    criticalPoints.push_back(maxX + 1);
    sort(all(criticalPoints));
    for (int i = 0; i < (sz(criticalPoints) - 1); ++i) {
        double lo = criticalPoints[i], hi = criticalPoints[i + 1];
        bool positiveLeft = poly(lo) > 0;
        if (positiveLeft ^ (poly(hi) > 0)) {
            for (int it = 0; it < (60); ++it) { // while (h - l > 1e-8)
                double mid = (lo + hi) / 2, value = poly(mid);
                if ((value <= 0) ^ positiveLeft)
                    lo = mid;
                else
                    hi = mid;
            }
            roots.push_back((lo + hi) / 2);
        }
    }
    return roots;
}
```

Polynomial Interpolation

Description: Given n points $(x[i], y[i])$, computes the unique polynomial p of degree at most $n-1$ that passes through them: $p(x) = a[0] * x^0 + \dots + a[n-1] * x^{n-1}$. Use when a polynomial of known degree d is available only through values: sample $d+1$ distinct x values, recover coefficients, then evaluate/differentiate with Poly. This fits every supplied point exactly in exact arithmetic; it is not least-squares regression for noisy observations. If sample locations are free and $n>1$, pick $x[k] = c * \cos(k/(n-1) * \pi), k = 0 \dots n-1$. Use $n>=1$ distinct x coordinates and n matching x/y values. Returns n coefficients in increasing powers; result[0] is the constant term. Floating interpolation can be unstable for large n or poorly spaced x . Use small well-scaled data, or a separate modular method for exact modular answers.

Usage: auto coeff=interpolate({0,1,2},{1,4,9},3);
// approximately {1,2,1}, representing $(x+1)^2$
Poly p{coeff}; double value=p(3); // 16; include Polynomial.h
for Poly
Time: $\mathcal{O}(n^2)$

```
typedef vector<double> vd;
vd interpolate(vd x, vd y, int n) {
    vd coeff(n), basis(n);
    for (int k = 0; k < (n - 1); ++k)
```

```
        for (int i = k + 1; i < (n); ++i) y[i] = (y[i] - y[k]) / (x[i] - x[k]);
    double previous = 0;
    basis[0] = 1;
    for (int k = 0; k < (n); ++k)
        for (int i = 0; i < (n); ++i) {
            coeff[i] += y[k] * basis[i];
            swap(previous, basis[i]);
            basis[i] -= previous * x[k];
        }
    return coeff;
}
```

Linear Recurrence

Description: Computes one distant term of a constant-coefficient linear recurrence modulo mod: $S[i] = \sum_{j=0}^{n-1} S[i-j-1]recurrence[j]$ for $i \geq n$, from its first n terms. Use for Fibonacci-like sequences or counting DP with fixed transitions when iterating to k is too slow. For all terms up to a small k , ordinary $O(nk)$ DP is simpler. This is homogeneous: a constant forcing term or index-dependent coefficients need a transformed recurrence. Polynomial binary exponentiation reduces powers using the characteristic polynomial, obtaining weights of the initial terms. $O(n^2 \log k)$ beats generic $O(n^3 \log k)$ matrix exponentiation. If coefficients are unknown, Berlekamp–Massey can infer them from enough initial terms. Use when recurrence order is small but the requested index is huge. initial contains exactly n initial terms; recurrence[j] multiplies $S[i-j-1]$. index is 0-based and nonnegative. Supply the global mod used by the code; the hidden value 5 is only for repository tests. Products must fit ll. Require $n \geq 1$ and $index < LLONG_MAX$; normalize negative terms/coefficients modulo mod.

Usage: ll fib=linearRec({0,1},{1,1},10); // 55 modulo mod (0 with test mod=5)
// S[i]=2*S[i-1]+3*S[i-2], S[0]=1, S[1]=4:
ll answer=linearRec({1,4},{2,3},2); // 11 modulo mod;
coefficients newest first
ll distant=linearRec({0,1},{1,1},1000000000000LL);
Time: $\mathcal{O}(n^2 \log k)$

```
typedef vector<ll> Poly;
ll linearRec(Poly initial, Poly recurrence, ll index) {
    int n = sz(recurrence);

    auto combine = [&](Poly a, Poly b) {
        Poly result(n * 2 + 1);
        for (int i = 0; i < (n + 1); ++i)
            for (int j = 0; j < (n + 1); ++j) result[i + j] = (result[i + j] + a[i] * b[j]) % mod;
        for (int i = 2 * n; i > n; --i)
            for (int j = 0; j < (n); ++j)
                result[i - 1 - j] = (result[i - 1 - j] + result[i] * recurrence[j]) % mod;
        result.resize(n + 1);
        return result;
    };

    Poly weights(n + 1, power(weights);
    weights[0] = power[1] = 1;

    for (++index; index; index /= 2) {
        if (index % 2) weights = combine(weights, power);
        power = combine(power, power);
    }

    ll result = 0;
    for (int i = 0; i < (n); ++i) result = (result + weights[i + 1] * initial[i]) % mod;
    return result;
}
```

3.2 Matrices

Matrix

Description: Fixed-size square matrices, initially zero; N is a compile-time dimension. Exponent must be nonnegative and A^0 is the identity. Multiplication has no modulus: add reduction or choose a suitable scalar type, with products and accumulated sums fitting that type.

Usage: Matrix<ll,2> a; a.data={{{1,1},{1,0}}};
array<ll,2> state={1,0};
auto next=(a^10)*state; // {89,55}

```
template <class T, int N> struct Matrix {
    typedef Matrix M;
    array<array<T, N>, N> data{};
    M operator*(const M &other) const {
        M result;
        for (int i = 0; i < (N); ++i)
            for (int j = 0; j < (N); ++j)
                for (int k = 0; k < (N); ++k) result.data[i][k] += data[i][j] * other.data[j][k];
        return result;
    }
    array<T, N> operator*(const array<T, N> &vec) const {
        array<T, N> result{};
        for (int i = 0; i < (N); ++i)
            for (int j = 0; j < (N); ++j) result[i] += data[i][j] * vec[j];
        return result;
    }
    M operator^(ll exponent) const {
        assert(exponent >= 0);
        M result, base(*this);
        for (int i = 0; i < (N); ++i) result.data[i][i] = 1;
        while (exponent) {
            if (exponent & 1) result = result * base;
            base = base * base;
            exponent >>= 1;
        }
        return result;
    }
};
```

Determinant

Description: Calculates determinant of a matrix. Destroys the matrix. Input must be square. Row swaps change the determinant sign; elimination modifies the matrix, so copy it first if needed. Zero is tested exactly; ill-conditioned inputs need scaling or higher precision. Do not use floating det for exact integer/modular answers.

Usage: vector<vector<double>>> a={{1,2},{3,4}};
double determinant=det(a); // -2; a is modified
Time: $\mathcal{O}(N^3)$

```
double det(vector<vector<double>>> &matrix) {
    int n = sz(matrix);
    double determinant = 1;
    for (int i = 0; i < (n); ++i) {
        int pivotRow = i;
        for (int j = i + 1; j < (n); ++j)
            if (fabs(matrix[j][i]) > fabs(matrix[pivotRow][i]))
                pivotRow = j;
        if (i != pivotRow) swap(matrix[i], matrix[pivotRow]),
            determinant *= -1;
        determinant *= matrix[i][i];
        if (determinant == 0) return 0;
        for (int j = i + 1; j < (n); ++j) {
            double factor = matrix[j][i] / matrix[i][i];
            if (factor != 0)
                for (int k = i + 1; k < (n); ++k) matrix[j][k] -=
                    factor * matrix[i][k];
        }
    }
}
```

```
    return determinant;
}
```

Integer Determinant

Description: Calculates determinant using modular arithmetics. Modulos can also be removed to get a pure-integer version. Set global mod and normalize entries modulo it. Euclidean row steps work with composite moduli too, unlike elimination requiring inverses. Input must be square and is destroyed. Products before reduction must fit ll; use wider products if needed. Result lies in [0,mod).

Usage: vector<vector<ll>>> a={{1,2},{3,4}};
ll determinant=det(a); // mod=2; a is modified

Time: $\mathcal{O}(N^3)$

```
const ll mod = 12345;
ll det(vector<vector<ll>>> &matrix) {
    int n = sz(matrix);
    ll determinant = 1;
    for (int i = 0; i < (n); ++i) {
        for (int j = i + 1; j < (n); ++j) {
            while (matrix[j][i] != 0) { // gcd step
                ll quotient = matrix[i][i] / matrix[j][i];
                if (quotient)
                    for (int k = i; k < (n); ++k)
                        matrix[i][k] = (matrix[i][k] - matrix[j][k] *
                            quotient) % mod;
                swap(matrix[i], matrix[j]);
                determinant *= -1;
            }
        }
        determinant = determinant * matrix[i][i] % mod;
        if (!determinant) return 0;
    }
    return (determinant + mod) % mod;
}
```

Gaussian Elimination

Description: Solves $A * x = b$. If there are multiple solutions, an arbitrary one is returned. Returns rank, or -1 if no solutions. Data in A and b is lost. For n equations in m unknowns, resize solution to m before calling: its size tells the solver the number of columns. Rank m means unique solution; $0 \leq \text{rank} < m$ means free variables, chosen as zero. Read solution only if $\text{rank} \neq -1$. A and b are modified; copy them if needed. For poorly scaled floating input, adjust eps; modular equations need field arithmetic.

Usage: vector<vd> a={{1,1},{1,-1}}; vd b={3,1}, x(2);
int rank=solveLinear(a,b,x); // rank=2, x={2,1}

Time: $\mathcal{O}(n^2m)$

```
typedef vector<double> vd;
const double eps = 1e-12;
int solveLinear(vector<vd> &matrix, vd &rhs, vd &solution) {
    int n = sz(matrix), m = sz(solution), rank = 0, pivotRow,
        pivotCol;
    if (n) assert(sz(matrix[0]) == m);
    vector<int> colOrder(m);
    iota(all(colOrder), 0);
```

```
    for (int i = 0; i < (n); ++i) {
        double magnitude, pivotValue = 0;
        for (int r = i; r < (n); ++r)
            for (int c = i; c < (m); ++c)
                if ((magnitude = fabs(matrix[r][c])) > pivotValue)
                    pivotRow = r, pivotCol = c, pivotValue = magnitude;
        if (pivotValue <= eps) {
            for (int j = i; j < (n); ++j)
                if (fabs(rhs[j]) > eps) return -1;
            break;
        }
        swap(matrix[i], matrix[pivotRow]);
        swap(rhs[i], rhs[pivotRow]);
```

```
        swap(colOrder[i], colOrder[pivotCol]);
        for (int j = 0; j < (n); ++j) swap(matrix[j][i], matrix[j][
            pivotCol]);
        pivotValue = 1 / matrix[i][i];
        for (int j = i + 1; j < (n); ++j) {
            double factor = matrix[j][i] * pivotValue;
            rhs[j] -= factor * rhs[i];
            for (int k = i + 1; k < (m); ++k) matrix[j][k] -= factor
                * matrix[i][k];
        }
        rank++;
    }

    solution.assign(m, 0);
    for (int i = rank; i--;) {
        rhs[i] /= matrix[i][i];
        solution[colOrder[i]] = rhs[i];
        for (int j = 0; j < (i); ++j) rhs[j] -= matrix[j][i] * rhs[
            i];
    }
    return rank; // (multiple solutions if rank < m)
}
```

Gaussian Elimination: Unique Values

Description: To get all uniquely determined values of x back from SolveLinear, make the following changes: This is an edit recipe for SolveLinear, not a separate callable solver. Eliminate each pivot column from all other rows, then mark variables depending on free columns as undefined. Choose an undefined marker such as NaN. A rank below m does not mean every variable is undetermined.

Usage: // Apply the changes inside solveLinear first.
// Then size x to m and call solveLinear(a,b,x) as usual.

```
"SolveLinear.h"
for (int j = 0; j < (n); ++j)
    if (j != i) // instead of for (int j = i+1; j < (n); ++j)
        // ... then at the end:
        solution.assign(m, undefined);
for (int i = 0; i < (rank); ++i) {
    for (int j = rank; j < (m); ++j)
        if (fabs(matrix[i][j]) > eps) goto fail;
    solution[colOrder[i]] = rhs[i] / matrix[i][i];
fail:;
}
```

Gaussian Elimination over GF(2)

Description: Solves $Ax = b$ over $\mathbb{F}_2$. If there are multiple solutions, one is returned arbitrarily. Returns rank, or -1 if no solutions. Destroys A and b . Use for XOR equations: each row bitset holds the first m coefficients and rhs entries are 0/1. All coefficient bits from m upward must be zero and $m \leq 1000$. Returns rank or -1; free variables are zero. Both matrix and rhs are modified. Each solution bit is that variable's assigned value. For $m > 1000$, enlarge bs ; `_Find_next` requires GNU bitset support.

Usage: vector<bs> a={bs(3),bs(1)}; vector<int> b={1,0}; bs x;
int rank=solveLinear(a,b,x,2); // x[0]=0, x[1]=1

Time: $\mathcal{O}(n^2m)$

```
typedef bitset<1000> bs;
int solveLinear(vector<bs> &matrix, vector<int> &rhs, bs &
    solution, int m) {
    int n = sz(matrix), rank = 0, pivotRow;
    assert(m <= sz(solution));
    vector<int> colOrder(m);
    iota(all(colOrder), 0);
    for (int i = 0; i < (n); ++i) {
        for (pivotRow = i; pivotRow < n; ++pivotRow)
            if (matrix[pivotRow].any()) break;
        if (pivotRow == n) {
            for (int j = i; j < (n); ++j)
                if (rhs[j]) return -1;
            break;
        }
```

```
    }
    int pivotCol = (int)matrix[pivotRow]._Find_next(i - 1);
    swap(matrix[i], matrix[pivotRow]);
    swap(rhs[i], rhs[pivotRow]);
    swap(colOrder[i], colOrder[pivotCol]);
    for (int j = 0; j < (n); ++j)
        if (matrix[j][i] != matrix[j][pivotCol]) {
            matrix[j].flip(i);
            matrix[j].flip(pivotCol);
        }
    for (int j = i + 1; j < (n); ++j)
        if (matrix[j][i]) {
            rhs[j] ^= rhs[i];
            matrix[j] ^= matrix[i];
        }
    rank++;
}
```

```
solution = bs();
for (int i = rank; i--;) {
    if (!rhs[i]) continue;
    solution[colOrder[i]] = 1;
    for (int j = 0; j < (i); ++j) rhs[j] ^= matrix[j][i];
}
return rank; // (multiple solutions if rank < m)
}
```

Matrix Inverse

Description: Invert matrix A . Returns rank; result is stored in A unless singular ($\text{rank} < n$). Can easily be extended to prime moduli; for prime powers, repeatedly set $A^{-1} = A^{-1}(2I - AA^{-1}) \pmod{p^k}$ where A^{-1} starts as the inverse of $A \pmod p$, and k is doubled in each step. Use a square floating matrix. Only read the overwritten matrix as the inverse when the returned rank equals its dimension; otherwise it is singular and partially modified. Row and column pivoting use a fixed tolerance; scale input or adjust it for extreme magnitudes. Copy the original if needed afterward; modular inversion needs a separate implementation.

Usage: vector<vector<double>>> a={{2,0},{0,4}};
int rank=matInv(a); // rank=2, a=diag(0.5,0.25)

Time: $\mathcal{O}(n^3)$

```
int matInv(vector<vector<double>>> &matrix) {
    int n = sz(matrix);
    vector<int> colOrder(n);
    vector<vector<double>>> inverse(n, vector<double>(n));
    for (int i = 0; i < (n); ++i) inverse[i][i] = 1, colOrder[i]
        = i;
```

```
    for (int i = 0; i < (n); ++i) {
        int pivotRow = i, pivotCol = i;
        for (int j = i; j < (n); ++j)
            for (int k = i; k < (n); ++k)
                if (fabs(matrix[j][k]) > fabs(matrix[pivotRow][pivotCol]
                    )) pivotRow = j, pivotCol = k;
        if (fabs(matrix[pivotRow][pivotCol]) < 1e-12) return i;
        matrix[i].swap(matrix[pivotRow]);
        inverse[i].swap(inverse[pivotRow]);
        for (int j = 0; j < (n); ++j)
            swap(matrix[j][i], matrix[j][pivotCol]), swap(inverse[j][
                i], inverse[j][pivotCol]);
        swap(colOrder[i], colOrder[pivotCol]);
        double pivot = matrix[i][i];
        for (int j = i + 1; j < (n); ++j) {
            double factor = matrix[j][i] / pivot;
            matrix[j][i] = 0;
            for (int k = i + 1; k < (n); ++k) matrix[j][k] -= factor
                * matrix[i][k];
            for (int k = 0; k < (n); ++k) inverse[j][k] -= factor *
                inverse[i][k];
        }
    }
```

```
    for (int j = i + 1; j < (n); ++j) matrix[i][j] /= pivot;
    for (int j = 0; j < (n); ++j) inverse[i][j] /= pivot;
    matrix[i][i] = 1;
}

for (int i = n - 1; i > 0; --i)
    for (int j = 0; j < (i); ++j) {
        double pivot = matrix[j][i];
        for (int k = 0; k < (n); ++k) inverse[j][k] -= pivot *
            inverse[i][k];
    }

for (int i = 0; i < (n); ++i)
    for (int j = 0; j < (n); ++j) matrix[colOrder[i]][colOrder[
        j]] = inverse[i][j];
return n;
}
```

Number theory (4)

4.1 Big integers

Big Integer Arithmetic

Description: Signed big integers with +, -, *, /, %, <, ==, != and decimal I/O. Standard C++ only; use for exact counts beyond ll. Input: valid decimal strings with optional sign. Division truncates toward zero; remainder has dividend's sign. Divisor must be nonzero. Decimal schoolbook arithmetic favors short code over speed; best for hundreds of digits. **Usage:** BigInt a(string("12345678901234567890")), b=7; cout << a+b << ' ' << a-b << ' ' << a*b << endl; auto [q,r]=divmod(a,b); assert(q*b+r==a); // also a/b, a%b; cin >> a **Time:** $\mathcal{O}(n+m)$ addition/comparison; $\mathcal{O}((n+1)(m+1))$ multiplication/division, $\mathcal{O}(n+m)$ space; n,m are decimal digit counts. divmod computes quotient/remainder together.

```
#include <iostream>
#include <string>
#include <utility>
#include <vector>
#include <cassert>
using namespace std;
struct Big {
    vector<int> a; // decimal digits, least significant first;
                    empty means zero
    int sign=1;
    Big(long long v=0) : Big(to_string(v)) {}
    explicit Big(const string &s) {
        int p=!s.empty() && (s[0]=='-' || s[0]=='+' );
        sign=!s.empty() && s[0]=='-' ? -1 : 1;
        for (int i=(int)s.size(); i>p;) a.push_back(s[--i]-'0');
        trim(); }
    void trim() {
        while (!a.empty() && !a.back()) a.pop_back();
        if (a.empty()) sign=1; }
    Big abs() const { Big r=*this; r.sign=1; return r; }
    Big operator-( ) const {
        Big r=*this; if (!a.empty()) r.sign=-sign; return r; }
    int cmpAbs(const Big &v) const {
        if (a.size()!=v.a.size()) return a.size()<v.a.size() ? -1 :
            1;
        for (int i=(int)a.size(); i--;)
            if (a[i]!=v.a[i]) return a[i]<v.a[i] ? -1 : 1;
        return 0; }
    int cmp(const Big &v) const {
        return sign!=v.sign ? (sign<v.sign ? -1 : 1) : sign*cmpAbs(
            v);
    }
    bool operator<(const Big &v) const { return cmp(v)<0; }
```

```
bool operator==(const Big &v) const { return cmp(v)==0; }
bool operator!=(const Big &v) const { return cmp(v)!=0; }
Big operator+(const Big &v) const {
    if (cmpAbs(v)<0) return v+*this;
    Big r=*this; int carry=0, same=sign==v.sign;
    for (int i=0; i<(int)a.size() || carry; ++i) {
        if (i==(int)r.a.size()) r.a.push_back(0);
        int d=r.a[i]+carry;
        if (i<(int)v.a.size()) d+=(same ? 1 : -1)*v.a[i];
        carry=same ? d/10 : -(d<0); r.a[i]=(d+10)%10;
    } r.trim(); return r; }
Big operator-(const Big &v) const { return *this+(-v); }
Big operator*(const Big &v) const {
    Big r; r.sign=sign*v.sign; r.a.resize(a.size()+v.a.size());
    for (int i=0; i<(int)a.size(); ++i)
        for (int j=0,carry=0; j<(int)v.a.size() || carry; ++j) {
            int d=r.a[i+j]+carry+(j<(int)v.a.size() ? a[i]*v.a[j] :
                0);
            r.a[i+j]=d%10; carry=d/10;
        } r.trim(); return r; }
friend pair<Big,Big> divmod(const Big &x, const Big &y) {
    assert(!y.a.empty()); Big b=y.abs(),q,r;
    q.a.resize(x.a.size());
    for (int i=(int)x.a.size(); i--;) {
        r.a.insert(r.a.begin(),x.a[i]); r.trim(); // r=r*10+x[i]
        while (r.cmpAbs(b)>=0) r=r-b, ++q.a[i]; // at most 9
            subtractions per digit
        } q.sign=x.sign*y.sign; r.sign=x.sign; q.trim(); r.trim();
        return {q,r};
    }
    Big operator/(const Big &v) const { return divmod(*this,v).
        first; }
    Big operator%(const Big &v) const { return divmod(*this,v).
        second; }
    friend istream &operator>>(istream &in, Big &v) {
        string s; if (in>>s) v=Big(s); return in; }
    friend ostream &operator<<(ostream &out, const Big &v) {
        if (v.sign<0) out << '-';
        if (v.a.empty()) out << '0';
        for (int i=(int)v.a.size(); i--;) out << v.a[i];
        return out; }
};
using BigInt = Big;
```

4.2 Modular arithmetic

Modular Arithmetic

Description: Set global mod first; residues are normalized to [0,mod) and read via .value. Division asserts unless its divisor is coprime with mod. Exponents must be nonnegative; multiplication must fit ll before reduction. **Usage:** Mod a=5, b=3; ll residue=(a+b+a).value; Mod power=a^10; Mod quotient=a/b; // gcd(b,mod)=1 "euclid.h"

```
const ll mod = 17; // change to something else
struct Mod {
    ll value;
    Mod(ll normalized) : Mod(normalized % mod + mod, 0) {}
    Mod(ll normalized, int) : value(normalized < mod ? normalized
        : normalized - mod) {}
    Mod operator+(Mod other) { return {value + other.value, 0}; }
    Mod operator-(Mod other) { return {value - other.value + mod,
        0}; }
    Mod operator*(Mod other) { return {value * other.value % mod,
        0}; }
    Mod operator/(Mod other) { return *this * invert(other); }
    Mod invert(Mod operand) {
        ll inverseCoeff, otherCoeff, gcdValue = euclid(operand.
            value, mod, inverseCoeff, otherCoeff);
        assert(gcdValue == 1);
    }
```

```
        return inverseCoeff;
    }
    Mod operator^(ll exponent) {
        if (!exponent) return 1;
        Mod result = *this ^ (exponent / 2);
        result = result * result;
        return exponent & 1 ? *this * result : result;
    }
};
```

Modular Inverses

Description: Choose prime mod and $2 \leq \text{LIM} \leq \text{mod}$, then run the initialization loop inside setup/main. inverse[i] is valid for $1 \leq i < \text{LIM}$; zero has no inverse. Costs $\mathcal{O}(\text{LIM})$ time and memory.

Usage: // Define prime mod and LIM, then run the shown setup loop. ll invOfThree=inverse[3]; // requires LIM>3 const ll mod = 1000000007, LIM = 200000; ll *inverse = new ll[LIM]; inverse[1] = 1; for (int i = 2; i < (LIM); ++i) inverse[i] = mod - (mod / i) * inverse[mod % i] % mod;

Discrete Logarithm

Description: Returns the smallest $x > 0$ s.t. $a^x = b \pmod m$, or -1 if no such x exists. modLog(a,1,m) can be used to calculate the order of a. Use for discrete logarithms when sqrt(modulus) work/storage fit. Also supports non-coprime bases. The exponent is strictly positive: target=1 asks for a multiplicative order, not exponent zero. Normalize base and target to [0,modulus]; modulus is positive and direct modular products must fit ll. If exponent 0 is allowed, return 0 first when target==1%modulus.

Usage: ll exponent=modLog(2,8,13); // 3 ll absent=modLog(2,3,4); // -1 **Time:** $\mathcal{O}(\sqrt{m})$ ll modLog(ll base, ll target, ll modulus) { ll blockSize = (ll)sqrt(modulus) + 1, power = 1, giantStep = 1, j = 1; unordered_map<ll, ll> babySteps; while (j <= blockSize && (power = giantStep = power * base % modulus) != target % modulus) babySteps[power * target % modulus] = j++; if (power == target % modulus) return j; if (__gcd(modulus, power) == __gcd(modulus, target)) for (int i = 2; i < (blockSize + 2); ++i) if (babySteps.count(power = power * giantStep % modulus)) return blockSize * i - babySteps[power]; return -1; }

Floor Sum and Modular Sum

Description: Computes progression sums without iterating: modsum(count,offset,slope,modulus) is $\sum_{i=0}^{count-1} ((offset + slope \cdot i) \pmod{modulus})$ with nonnegative residues. divsum gives the corresponding sum of floored quotients for nonnegative inputs. Use for lattice-point counts or periodic sums. modulus must be positive; count is nonnegative. modsum normalizes negative offset/slope. Intermediate products must fit the implementation's integer arithmetic.

Usage: ll sum=modsum(4,1,2,5); // (1+3+0+2)=6 ull floors=divsum(4,1,2,5); // (0+0+1+1)=2 **Time:** log(m), with a large constant.

```
ull sumsq(ull count) { return count / 2 * ((count - 1) | 1); }

ull divsum(ull count, ull offset, ull slope, ull modulus) {
    ull answer = slope / modulus * sumsq(count) + offset /
        modulus * count;
    slope %= modulus;
    offset %= modulus;
    if (!slope) return answer;
```

```
ull nextCount = (count * slope + offset) / modulus;
return answer + (count - 1) * nextCount - divsum(nextCount,
    modulus - 1 - offset, modulus, slope);
}

ll modsum(ull count, ll offset, ll slope, ll modulus) {
    offset = ((offset % modulus) + modulus) % modulus;
    slope = ((slope % modulus) + modulus) % modulus;
    return count * offset + slope * sumsq(count) - modulus *
        divsum(count, offset, slope, modulus);
}
```

64-bit Modular Multiplication

Description: Calculate $a \cdot b \bmod c$ (or $a^b \bmod c$) for $0 \leq a, b \leq c \leq 7.2 \cdot 10^{18}$. Use for large-modulus exponentiation and primality/factorization when ordinary ll multiplication would overflow. The three-argument modpow takes an explicit modulus. The large bound assumes an 80-bit long double; for other platforms or larger moduli use unsigned _int128 multiplication/reduction. Normalize operands first and use a positive modulus.

Usage: ull product=modmul(1000000000000000ULL, 1000000000000000ULL, 100000000000000003ULL);

ull power=modpow(2,100,1000000007);

Time: $\mathcal{O}(1)$ for modmul, $\mathcal{O}(\log b)$ for modpow

```
ull modmul(ull lhs, ull rhs, ull modulus) {
    ll remainder = lhs * rhs - modulus * (ull)(l.L / modulus *
        lhs * rhs);
    return remainder + modulus * (remainder < 0) - modulus * (
        remainder >= (ll)modulus);
}

ull modpow(ull base, ull exponent, ull mod) {
    ull result = 1;
    for (; exponent; base = modmul(base, base, mod), exponent /=
        2)
        if (exponent & 1) result = modmul(result, base, mod);
    return result;
}
```

Modular Square Root

Description: Tonelli-Shanks algorithm for modular square roots. Finds x s.t. $x^2 = a \pmod p$ ($-x$ gives the other solution). Use an odd prime p and a quadratic residue a; test modpow(a,(p-1)/2,p)=1 for nonzero a first. Non-residues assert rather than return a failure marker. Handle p=2 separately. Requires the three-argument modpow from ModMulLL.h; all direct products must fit ll. The other root is (p-x)%p. For large p, replace direct modular products with modmul.

Usage: ll x=sqrt(10LL,13LL); // 6 or 7

ll other=(13-x)%13;

Time: $\mathcal{O}(\log^2 p)$ worst case, $\mathcal{O}(\log p)$ for most p

ModMulLL.h

```
ll sqrt(ll a, ll prime) {
    a %= prime;
    if (a < 0) a += prime;
    if (a == 0) return 0;
    assert(modpow(a, (prime - 1) / 2, prime) == 1); // else no
        solution
    if (prime % 4 == 3) return modpow(a, (prime + 1) / 4, prime);
    // a^(n+3)/8 or 2^(n+3)/8 * 2^(n-1)/4 works if p % 8 == 5
    ll oddPart = prime - 1, nonResidue = 2;
    int twos = 0, order;
    while (oddPart % 2 == 0) ++twos, oddPart /= 2;
    while (modpow(nonResidue, (prime - 1) / 2, prime) != prime -
        1) ++nonResidue;
    ll x = modpow(a, (oddPart + 1) / 2, prime);
    ll remainder = modpow(a, oddPart, prime), rootOfUnity =
        modpow(nonResidue, oddPart, prime);
    for (; twos = order) {
        ll power = remainder;
        for (order = 0; order < twos && power != 1; ++order) power
            = power * power % prime;
```

```
    if (order == 0) return x;
    ll correction = modpow(rootOfUnity, 1LL << (twos - order -
        1), prime);
    rootOfUnity = correction * correction % prime;
    x = x * correction % prime;
    remainder = remainder * rootOfUnity % prime;
}
}
```

4.3 Primality

Linear Sieve (Smallest Prime Factors)

Description: lp[x] is the smallest prime factor for $2 \leq x \leq n$; lp[0]=lp[1]=0. Each composite is generated once. Uses more memory than the boolean sieve, but supports fast factorization. Factor only $x \leq n$ with lp; larger x needs trial division or Pollard-rho.

Usage: LinearSieve sieve(100);

int smallest=sieve.lp[84]; // 2

int x=84;

while (x>1) { cout << sieve.lp[x] << ' '; x/=sieve.lp[x]; }

// 2 2 3 7

Time: $\mathcal{O}(N)$ construction and memory; $\mathcal{O}(\log x)$ to factor x using lp.

```
struct LinearSieve {
    vector<int> lp, primes;
    LinearSieve(int n) : lp(n + 1) {
        for (int i = 2; i <= n; i++) {
            if (lp[i] == 0) lp[i] = i, primes.push_back(i);
            for (int p : primes) {
                if (1LL * i * p > n) break;
                lp[i * p] = p;
                if (p == lp[i]) break;
            }
        }
    }
};
```

Miller-Rabin Primality Test

Description: Deterministic Miller-Rabin primality test. Guaranteed to work for numbers up to $7 \cdot 10^{18}$; for larger numbers, use Python and extend A randomly. Use when testing individual large integers without sieving to them. Returns false for 0 and 1. A fixed witness set gives the stated deterministic range, using ModMulLL for modular products. The name isPrime conflicts with the sieve bitset if both snippets are pasted unchanged.

Usage: bool prime=isPrime(1000000007ULL); // true

Time: 7 times the complexity of $a^b \bmod c$.

ModMulLL.h

```
bool isPrime(ull n) {
    if (n < 2 || n % 6 % 4 != 1) return (n | 1) == 3;
    ull bases[] = {2, 325, 9375, 28178, 450775, 9780504,
        1795265022}, twos = __builtin_ctzll(n - 1),
        oddPart = n >> twos;
    for (ull base : bases) { // ^ count trailing zeroes
        ull power = modpow(base % n, oddPart, n), squaresLeft =
            twos;
        while (power != 1 && power != n - 1 && base % n &&
            squaresLeft--)
            power = modmul(power, power, n);
        if (power != n - 1 && squaresLeft != twos) return 0;
    }
    return 1;
}
```

Pollard-Rho Factorization

Description: Pollard-rho randomized factorization algorithm. Returns prime factors of a number, in arbitrary order (e.g. 2299 -> {11, 19, 11}). Call factor on a positive integer; factor(1) is empty and factor(0) is unsupported. Repeated factors are retained and output is unsorted. Sort to group prime powers for divisor counts or totients. pollard is an internal splitter for composite n, not a primality test. ModMulLL platform/range limits apply.

```
Usage: auto factors=factor(2299); sort(all(factors));
// {11,11,19}
Time:  $\mathcal{O}\left(n^{1/4}\right)$ , less for numbers with small factors.
ModMulLL.h, "MillerRabin.h"

ull pollard(ull n) {
    ull slow = 0, fast = 0, iterations = 30, product = 2, seed =
        1, nextProduct;
    auto advance = [&](ull slow) { return modmul(slow, slow, n) +
        seed; };
    while (iterations++ % 40 || __gcd(product, n) == 1) {
        if (slow == fast) slow = ++seed, fast = advance(slow);
        if ((nextProduct = modmul(product, max(slow, fast) - min(
            slow, fast), n)))
            product = nextProduct;
        slow = advance(slow), fast = advance(advance(fast));
    }
    return __gcd(product, n);
}

vector<ull> factor(ull n) {
    if (n == 1) return {};
    if (isPrime(n)) return {n};
    ull divisor = pollard(n);
    auto leftFactors = factor(divisor), rightFactors = factor(n /
        divisor);
    leftFactors.insert(leftFactors.end(), all(rightFactors));
    return leftFactors;
}
```

4.4 Divisibility

Extended Euclidean Algorithm

Description: Returns $g = \gcd(a, b)$ and coefficients with $ax + by = g$. Use positive a,b. For $ax+by=c$ require g to divide c, then scale coefficients by c/g. For an inverse modulo m require g=1 and normalize x to [0,m). Products must fit ll. For negative inputs, solve using absolute values and flip the corresponding coefficient signs; handle a=b=0 separately.

Usage: ll x,y; ll g=euclid(30,18,x,y); // 30*x+18*y=6

```
ll euclid(ll a, ll b, ll &x, ll &y) {
    if (!b) return x = 1, y = 0, a;
    ll gcdValue = euclid(b, a % b, y, x);
    return y -= a / b * x, gcdValue;
}
```

Chinese Remainder Theorem (CRT)

Description: crt(a,m,b,n) returns x with $x \equiv a \pmod m$, $x \equiv b \pmod n$. Use positive moduli and normalized residues; require $(a-b) \bmod \gcd(m,n) = 0$ or it asserts. Returns $0 \leq x < \text{lcm}(m,n)$, including non-coprime moduli. Requires $mn < 2^{62}$.

Usage: ll x=crt(2,3,3,5); // 8 modulo 15

// Check (a-b)%gcd(m,n)==0 before calling.

Time: log(n)

euclid.h

```
ll crt(ll a, ll m, ll b, ll n) {
    if (n > m) swap(a, b), swap(m, n);
    ll x, y, gcdValue = euclid(m, n, x, y);
    assert((a - b) % gcdValue == 0); // else no solution
    x = (b - a) % n * x % n / gcdValue * m + a;
    return x < 0 ? x + m * n / gcdValue : x;
}
```

4.4.1 Bézout’s identity

For integers a, b not both zero, $d = \gcd(a, b) > 0$ is the smallest positive integer for which there are integer solutions to

$$ax + by = d$$

If (x,y) is one solution, then all solutions are given by

(x + kb/gcd(a,b), y - ka/gcd(a,b)), k in Z

Euler’s Totient Function

Description: Choose LIM and call calculatePhi once for phi[1..LIM-1]. phi[n] counts residues coprime with n; φ(n) = n ∏p|n(1 − 1/p). Reduce exponents modulo phi(n) directly only when base and n are coprime.

Usage: calculatePhi(); int count=phi[12]; // 4
const int LIM = 5000000;
int phi[LIM];
void calculatePhi() {
for (int i = 0; i < (LIM); ++i) phi[i] = i & 1 ? i : i / 2;
for (int i = 3; i < LIM; i += 2)
if (phi[i] == i)
for (int j = i; j < LIM; j += i) phi[j] -= phi[j] / i;
}

4.5 Fractions

Continued Fractions

Description: Given N and a real number x ≥ 0, finds the closest rational approximation p/q with p,q ≤ N. It will obey |p/q − x| ≤ 1/qN. For consecutive convergents, pk+1qk − qk+1pk = (−1)k. (pk/qk alternates between > x and < x.) If x is rational, y eventually becomes ∞; if x is the root of a degree 2 polynomial the a’s eventually become cyclic. Use to approximate nonnegative real input by a bounded rational. limit>=1 bounds both numerator and denominator, not only the denominator. Returns (p,q). Precision comes from type d; use long double when increasing the limit substantially. For negative x, approximate -x and negate the returned numerator. x must be finite.

Usage: auto [p,q]=approximate(3.141592653589793,100);
// {22,7}
Time: O(log N)

typedef double d; // for N ~ 1e7; long double for N ~ 1e9
pair<ll, ll> approximate(d x, ll limit) {
ll prevNum = 0, prevDen = 1, numerator = 1, denominator = 0,
inf = LLONG_MAX;
d remainder = x;
for (;;) {
ll maxStep = min(numerator ? (limit - prevNum) / numerator
: inf,
denominator ? (limit - prevDen) /
denominator : inf),
quotient = (ll)floor(remainder), step = min(quotient,
maxStep),
nextNum = step * numerator + prevNum, nextDen = step *
denominator + prevDen;
if (quotient > step) {
// If step > quotient/2, we have a semi-convergent that
gives us a
// better approximation; if step = quotient/2, we *may*
have one.
// Return {numerator, denominator} here for a more
canonical approximation.
return (abs(x - (d)nextNum / (d)nextDen) < abs(x - (d)
numerator / (d)denominator))
? make_pair(nextNum, nextDen)
: make_pair(numerator, denominator);
}
if (abs(remainder = 1 / (remainder - (d)quotient)) > 3 *
limit) return {nextNum, nextDen};
prevNum = numerator;
numerator = nextNum;
prevDen = denominator;
denominator = nextDen;

}
}

Fraction Binary Search

Description: Given f and N, finds the smallest fraction p/q ∈ [0, 1] such that f(p/q) is true, and p,q ≤ N. You may want to throw an exception from f if it finds an exact solution, in which case N can be removed. The predicate must be monotone false-to-true as p/q increases, and true at 1. limit>=1 bounds both p and q. Compare fractions by cross multiplication to avoid rounding; products must fit ll. Finds an exact bounded fraction rather than a floating approximation. For positive fractions beyond 1, change initial hi to 1,0; predicate must handle q=0.

Usage: Frac answer=fracBS([],(Frac x) { return 3*x.p>=x.q; },10);
// answer is {1,3}
Time: O(log(N))

struct Frac {
ll p, q;
};
template <class F> Frac fracBS(F predicate, ll limit) {
bool moveHi = 1, prevAdvanced = 1, advanced = 1;
Frac lo{0, 1}, hi{1, 1}; // Set hi to 1/0 to search (0, N]
if (predicate(lo)) return lo;
assert(predicate(hi));
while (prevAdvanced || advanced) {
ll advance = 0, step = 1; // move hi if moveHi, else lo
for (int shift = 0; step; (step *= 2) >= shift) {
advance += step;
Frac mid{lo.p * advance + hi.p, lo.q * advance + hi.q};
if (abs(mid.p) > limit || mid.q > limit || moveHi == !
predicate(mid)) {
advance -= step;
shift = 2;
}
}
hi.p += lo.p * advance;
hi.q += lo.q * advance;
moveHi = !moveHi;
swap(lo, hi);
prevAdvanced = advanced;
advanced = !!advance;
}
return moveHi ? hi : lo;
}

4.6 Pythagorean Triples

The Pythagorean triples are uniquely generated by

a = k · (m^2 − n^2), b = k · (2mn), c = k · (m^2 + n^2),

with m > n > 0, k > 0, m⊥n, and either m or n even.

4.7 Primes

p = 962592769 is such that 2^21 | p − 1, which may be useful. For hashing use 970592641 (31-bit number), 31443539979727 (45-bit), 3006703054056749 (52-bit). There are 78498 primes less than 1 000 000.

Primitive roots exist modulo any prime power p^a, except for p = 2, a > 2, and there are φ(φ(p^a)) many. For p = 2, a > 2, the group Z_2^x_a is instead isomorphic to Z_2 × Z_2^{a-2}.

4.8 Estimates

∑_{d|n} d = O(n log log n).

The number of divisors of n is at most around 100 for n < 5e4, 500 for n < 1e7, 2000 for n < 1e10, 200 000 for n < 1e19.

Combinatorial (5)

5.1 Permutations

5.1.1 Factorial

<i>n</i>	1	2	3	4	5	6	7	8	9	10
<i>n</i> !	1	2	6	24	120	720	5040	40320	362880	3628800
<i>n</i>	11	12	13	14	15	16	17			
<i>n</i> !	4.0e7	4.8e8	6.2e9	8.7e10	1.3e12	2.1e13	3.6e14			
<i>n</i>	20	25	30	40	50	100	150	171		
<i>n</i> !	2e18	2e25	3e32	8e47	3e64	9e157	6e262	>DBL_MAX		

Permutation Ranking

Description: Maps a permutation of 0..n-1 to a unique ID in [0,n!], without preserving lexicographic order. The return type is int; use n<=12. For 13..20 use ll rank; larger labels also need a wider mask.

Usage: vector<int> p={2,0,1}; int state=permToInt(p);
Time: O(n)

int permToInt(vector<int> &permutation) {
int usedMask = 0, i = 0, rank = 0;
for (int value : permutation)
rank = rank * ++i + __builtin_popcount(usedMask & ~(1 <<
value)),
usedMask |= 1 << value; // (note: minus, not ~!)
return rank;
}

5.1.2 Cycles

Let g_S(n) be the number of n-permutations whose cycle lengths all belong to the set S. Then

∑_{n=0}^∞ g_S(n) x^n / n! = exp (∑_{n∈S} x^n / n)

5.1.3 Derangements

Permutations of a set such that none of the elements appear in their original position.

D(n) = (n − 1)(D(n − 1) + D(n − 2)) = nD(n − 1) + (−1)^n = ⌊ n! / e ⌋

5.1.4 Burnside’s lemma

Given a group G of symmetries and a set X, the number of elements of X up to symmetry equals

1 / |G| ∑_{g ∈ G} |X^g|,

where X^g are the elements fixed by g (g.x = x).

If f(n) counts “configurations” (of some sort) of length n, we can ignore rotational symmetry using G = Z_n to get

g(n) = 1/n ∑_{k=0}^{n-1} f(gcd(n, k)) = 1/n ∑_{k|n} f(k)φ(n/k).

5.2 Partitions and subsets

5.2.1 Partition function

Number of ways of writing n as a sum of positive integers, disregarding the order of the summands.

$$p(0) = 1, \, p(n) = \sum_{k \in \mathbb{Z} \setminus \{0\}} (-1)^{k+1} p(n - k(3k - 1)/2)$$
$$p(n) \sim 0.145/n \cdot \exp(2.56\sqrt{n})$$

$\frac{n}{p(n)}$	0	1	2	3	4	5	6	7	8	9	20	50	100
	1	1	2	3	5	7	11	15	22	30	627	$\sim 2\text{e}5$	$\sim 2\text{e}8$

5.2.2 Lucas’ Theorem

Let n, m be non-negative integers and p a prime. Write $n = n_k p^k + \dots + n_1 p + n_0$ and $m = m_k p^k + \dots + m_1 p + m_0$. Then $\binom{n}{m} \equiv \prod_{i=0}^k \binom{n_i}{m_i} \pmod{p}$.

5.2.3 Binomials

Multinomial Coefficients

Description: Counts multiset arrangements: $\frac{(\sum k_i)!}{\prod k_i!}$. counts[i] must be nonnegative. Exact integer calculation without a modulus; the result and intermediate products must fit ll. For large exact counts use big integers; modular counts need factorials/inverses, not integer /.
Usage: vector<int> counts={2,1}; ll ways=multinomial(counts);
// 3

```
ll multinomial(vector<int> &counts) {
    ll ways = 1, total = counts.empty() ? 1 : counts[0];
    for (int i = 1; i < (sz(counts)); ++i)
        for (int j = 0; j < (counts[i]); ++j) ways = ways * ++total
            / (j + 1);
    return ways;
}
```

5.3 General purpose numbers

5.3.1 Bernoulli numbers

EGF of Bernoulli numbers is $B(t) = \frac{t}{e^t - 1}$ (FFT-able).

$$B[0, \dots] = [1, -\tfrac{1}{2}, \tfrac{1}{6}, 0, -\tfrac{1}{30}, 0, \tfrac{1}{42}, \dots]$$

Sums of powers:

$$\sum_{i=1}^n i^m = \frac{1}{m+1} \sum_{k=0}^m \binom{m+1}{k} B_k \cdot (n+1)^{m+1-k}, \quad m \geq 1$$

Euler-Maclaurin formula for infinite sums:

$$\sum_{i=m}^{\infty} f(i) = \int_m^{\infty} f(x) dx - \sum_{k=1}^{\infty} \frac{B_k}{k!} f^{(k-1)}(m)$$
$$\approx \int_m^{\infty} f(x) dx + \frac{f(m)}{2} - \frac{f'(m)}{12} + \frac{f'''(m)}{720} + O(f^{(5)}(m))$$

5.3.2 Stirling numbers of the first kind

Number of permutations on n items with k cycles.

$$c(n, k) = c(n - 1, k - 1) + (n - 1)c(n - 1, k), \, c(0, 0) = 1$$
$$\sum_{k=0}^n c(n, k) x^k = x(x + 1) \dots (x + n - 1)$$

$$c(8, k) = 8, 0, 5040, 13068, 13132, 6769, 1960, 322, 28, 1$$
$$c(n, 2) = 0, 0, 1, 3, 11, 50, 274, 1764, 13068, 109584, \dots$$

5.3.3 Eulerian numbers

Number of permutations $\pi \in S_n$ in which exactly k elements are greater than the previous element. k j :s s.t. $\pi(j) > \pi(j + 1)$, $k + 1$ j :s s.t. $\pi(j) \geq j$, k j :s s.t. $\pi(j) > j$.

$$E(n, k) = (n - k)E(n - 1, k - 1) + (k + 1)E(n - 1, k)$$
$$E(n, 0) = E(n, n - 1) = 1$$
$$E(n, k) = \sum_{j=0}^k (-1)^j \binom{n+1}{j} (k+1-j)^n$$

5.3.4 Stirling numbers of the second kind

Partitions of n distinct elements into exactly k groups.

$$S(n, k) = S(n - 1, k - 1) + kS(n - 1, k)$$
$$S(n, 1) = S(n, n) = 1$$
$$S(n, k) = \frac{1}{k!} \sum_{j=0}^k (-1)^{k-j} \binom{k}{j} j^n$$

5.3.5 Bell numbers

Total number of partitions of n distinct elements. $B(n) = 1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, \dots$ For p prime,

$$B(p^m + n) \equiv mB(n) + B(n + 1) \pmod{p}$$

5.3.6 Labeled unrooted trees

on n vertices: n^{n-2}
on k existing trees of size n_i : $n_1 n_2 \dots n_k n^{k-2}$
with degrees d_i : $(n - 2)! / ((d_1 - 1)! \dots (d_n - 1)!)$

5.3.7 Catalan numbers

$$C_n = \frac{1}{n+1} \binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1} = \frac{(2n)!}{(n+1)!n!}$$
$$C_0 = 1, \, C_{n+1} = \frac{2(2n+1)}{n+2} C_n, \, C_{n+1} = \sum C_i C_{n-i}$$

- $C_n = 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, \dots$
- sub-diagonal monotone paths in an $n \times n$ grid.
 - strings with n pairs of parenthesis, correctly nested.
 - binary trees with with $n + 1$ leaves (0 or 2 children).
 - ordered trees with $n + 1$ vertices.
 - ways a convex polygon with $n + 2$ sides can be cut into triangles by connecting vertices with straight lines.
 - permutations of $[n]$ with no 3-term increasing subseq.

Data structures (6)

Ordered Set (PBDS)

Description: GNU PBDS set with 0-based ranks. order_of_key(x) counts elements <x; find_by_order(k) returns an iterator, or end if absent. For duplicates use (value,uniqueId), not less.equal. Replace null_type for a map. join requires disjoint ordered key ranges, not interleaved sets.
Usage: OrderedSet<int> s; s.insert(8); s.insert(10);
int smaller=s.order_of_key(10); // 1
int kth=*s.find_by_order(0); // 8
Time: $\mathcal{O}(\log N)$

```
#include <bits/extc++.h>
using namespace __gnu_pbds;
```

```
template <class T>
using OrderedSet = tree<T, null_type, less<T>, rb_tree_tag,
    tree_order_statistics_node_update>;
void example() {
    OrderedSet<int> ordered, other;
    ordered.insert(8);
    auto inserted = ordered.insert(10).first;
    assert(inserted == ordered.lower_bound(9));
    assert(ordered.order_of_key(10) == 1);
    assert(ordered.order_of_key(11) == 2);
    assert(*ordered.find_by_order(0) == 8);
    ordered.join(other); // assuming T < T2 or T > T2, merge t2
                          into t
}
```

Custom Hash Map

Description: Integer-key map with randomized SplitMix64: fixed collision attacks are harder, but worst-case time is not guaranteed. Collisions affect speed, not correctness. CustomHash also works with unordered_map; only the displayed container needs GNU PBDS. Values are int. hashMap[key] inserts zero if absent; find does not insert. Iteration is unsorted.
Usage: hashMap[42]++;
auto it=hashMap.find(42);
if (it!=hashMap.end()) cout << it->second;
hashMap.clear(); // between test cases
// Standard-library alternative:
unordered_map<ll,int,CustomHash> freq;
freq.reserve(2*n); // optional, reduces rehashing
Time: Expected $\mathcal{O}(1)$ per insert/find/erase; worst-case $\mathcal{O}(N)$ per operation.

```
#include <ext/pb_ds/assoc_container.hpp>
struct CustomHash {
    static ull mix(ull x) {
        // Fixed constants mix all 64 bits; unsigned overflow is
        // intentional.
        x += 0x9e3779b97f4a7c15ULL;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
        x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
        return x ^ (x >> 31);
    }
    size_t operator() (ll key) const {
        static const ull seed = chrono::steady_clock::now().
            time_since_epoch().count();
        return mix((ull)key + seed);
    }
};
__gnu_pbds::gp_hash_table<ll, int, CustomHash> hashMap;
```

Iterative Segment Tree

Description: KACTL’s 2N iterative maximum tree: 1-based point assignment and inclusive [l,r]. Initially uniform initialValue; no power-of-two padding. Empty queries return unit. For another aggregate, edit T/merge/unit; preserve the two accumulator orders. Initialize with unit then update leaves, or rebuild parents if the aggregate changes.

Usage: IterativeSegTree seg(5,0);
seg.update(3,7); // a[3]=7, not +=7
int best=seg.query(2,4); // 7; includes positions 2,3,4
int single=seg.query(3,3); // 7
// To build from a[1..n]: update(i,a[i]) for i=1..n.
Time: $\mathcal{O}(N)$ construction, $\mathcal{O}(\log N)$ per update/query; $\mathcal{O}(N)$ memory (2N values).

```
struct IterativeSegTree {
    typedef int T;
    static constexpr T unit = INT_MIN;
    T merge(T a, T b) const { return max(a, b); }
    vector<T> seg;
    int n;
    IterativeSegTree(int n = 0, T initialValue = unit) : seg(2 *
        n, initialValue), n(n) {}
    void update(int pos, T val) {
        for (seg[pos += n - 1] = val; pos /= 2;) seg[pos] = merge(
            seg[pos * 2], seg[pos * 2 + 1]);
    }
    T query(int l, int r) const {
        if (n == 0 || l > r) return unit;
        T leftResult = unit, rightResult = unit;
        for (l += n - 1, r += n - 1; l <= r; l /= 2, r /= 2) {
            if (l & 1) leftResult = merge(leftResult, seg[l++]);
            if (!(r & 1)) rightResult = merge(seg[r--], rightResult);
        }
        return merge(leftResult, rightResult);
    }
};
```

Segment Tree + Walk

Description: Point assignment and range maximum. Positions are 1-based; query(l,r) is inclusive. Initially every position equals initialValue. To change the aggregate, edit T, merge and its identity unit; use a wider T if needed. firstAtLeast(l,r,val) walks left first to find the smallest position in [l,r] with value>=val, or -1 if absent. A node with maximum<val is skipped. This search relies on maximum; changing merge also changes the walk. update needs n>=1 and 1<=pos<=n; use LazySegTree for range updates.

Usage: SegTree seg(5,0); seg.update(3,7);
int best=seg.query(2,4); // 7; positions 2,3,4
int pos=seg.firstAtLeast(2,5,6); // 3; first value >=6 in [2,5]
// Find the first available capacity >=need, then consume it:
int need=4; pos=seg.firstAtLeast(1,5,need);
if (pos!=-1) seg.update(pos,seg.query(pos,pos)-need);
Time: $\mathcal{O}(\log N)$ per operation; $\mathcal{O}(N)$ memory.

```
struct SegTree {
    typedef int T;
    static constexpr T unit = INT_MIN;
    int n;
    vector<T> seg;
    SegTree(int n = 0, T initialValue = unit) : n(n), seg(4 * n +
        4, unit) {
        if (n > 0) build(1, 1, n, initialValue);
    }
    T merge(T a, T b) const { return max(a, b); }
    void build(int id, int lo, int hi, T val) {
        if (lo == hi) {
            seg[id] = val;
            return;
        }
        int mid = (lo + hi) / 2;
        build(id * 2, lo, mid, val);
        build(id * 2 + 1, mid + 1, hi, val);
        seg[id] = merge(seg[id * 2], seg[id * 2 + 1]);
    }
    void update(int id, int lo, int hi, int pos, T val) {
        if (lo == hi) {
```

```
            seg[id] = val;
            return;
        }
        int mid = (lo + hi) / 2;
        if (pos <= mid)
            update(id * 2, lo, mid, pos, val);
        else
            update(id * 2 + 1, mid + 1, hi, pos, val);
        seg[id] = merge(seg[id * 2], seg[id * 2 + 1]);
    }
    void update(int pos, T val) { update(1, 1, n, pos, val); }
    T query(int id, int lo, int hi, int l, int r) const {
        if (r < lo || hi < l) return unit;
        if (l <= lo && hi <= r) return seg[id];
        int mid = (lo + hi) / 2;
        return merge(query(id * 2, lo, mid, l, r), query(id * 2 +
            1, mid + 1, hi, l, r));
    }
    T query(int l, int r) const {
        if (n == 0 || l > r) return unit;
        return query(1, 1, n, l, r);
    }
    int firstAtLeast(int id, int lo, int hi, int l, int r, T val)
        const {
        if (r < lo || hi < l || seg[id] < val) return -1; // No
            valid position here.
            if (lo == hi) return lo;
            int mid = (lo + hi) / 2;
            int pos = firstAtLeast(id * 2, lo, mid, l, r, val);
            if (pos != -1) return pos; // Left first gives the smallest
                position.
            return firstAtLeast(id * 2 + 1, mid + 1, hi, l, r, val);
        }
    int firstAtLeast(int l, int r, T val) const {
        if (n == 0 || l > r) return -1;
        return firstAtLeast(1, 1, n, l, r, val);
    }
};
```

Lazy Segment Tree

Description: Range assignment, addition and maximum on inclusive, 1-based [l,r]. The vector constructor ignores values[0]; default initial value is -inf, so initialize finite values before addition. Values and additions must fit int. Keep hasSet for assignments; changing to sums also requires segment lengths. For multiplication/other tags, also change their composition order.

Usage: LazySegTree seg(5,0); seg.add(1,3,4); seg.set(2,2,9);
int best=seg.query(1,5); // 9
Time: $\mathcal{O}(\log N)$ per operation; $\mathcal{O}(N)$ memory.

```
const int inf = 10000000000;
struct LazySegTree {
    struct Node {
        int val = -inf;
        int lazySet = 0;
        int lazyAdd = 0;
        bool hasSet = false;
    };
    int n;
    vector<Node> seg;
    LazySegTree(int n, int initialValue = -inf) : n(n), seg(4 * n
        + 4, Node{initialValue}) {}
    LazySegTree(const vector<int> &values) : LazySegTree(max(0,
        sz(values) - 1)) {
        if (n > 0) build(1, 1, n, values);
    }
    void build(int id, int lo, int hi, const vector<int> &values)
        {
            if (lo == hi) {
                seg[id].val = values[lo];
                return;
            }
```

```
        }
        int mid = (lo + hi) / 2;
        build(id * 2, lo, mid, values);
        build(id * 2 + 1, mid + 1, hi, values);
        pull(id);
    }
    void pull(int id) { seg[id].val = max(seg[id * 2].val, seg[id
        * 2 + 1].val); }
    void applySet(int id, int val) {
        seg[id].val = val;
        seg[id].lazySet = val;
        seg[id].lazyAdd = 0;
        seg[id].hasSet = true;
    }
    void applyAdd(int id, int val) {
        seg[id].val += val;
        if (seg[id].hasSet)
            seg[id].lazySet += val;
        else
            seg[id].lazyAdd += val;
    }
    void push(int id) {
        // Assignment overwrites older updates; addition comes
            after assignment.
        if (seg[id].hasSet) {
            applySet(id * 2, seg[id].lazySet);
            applySet(id * 2 + 1, seg[id].lazySet);
            seg[id].hasSet = false;
        }
        if (seg[id].lazyAdd != 0) {
            applyAdd(id * 2, seg[id].lazyAdd);
            applyAdd(id * 2 + 1, seg[id].lazyAdd);
            seg[id].lazyAdd = 0;
        }
    }
    void update(int id, int lo, int hi, int l, int r, int val,
        bool isAssignment) {
        if (r < lo || hi < l) return;
        if (l <= lo && hi <= r) {
            if (isAssignment)
                applySet(id, val);
            else
                applyAdd(id, val);
            return;
        }
        push(id);
        int mid = (lo + hi) / 2;
        update(id * 2, lo, mid, l, r, val, isAssignment);
        update(id * 2 + 1, mid + 1, hi, l, r, val, isAssignment);
        pull(id);
    }
    void set(int l, int r, int val) {
        if (n > 0 && l <= r) update(1, 1, n, l, r, val, true);
    }
    void add(int l, int r, int val) {
        if (n > 0 && l <= r) update(1, 1, n, l, r, val, false);
    }
    int query(int id, int lo, int hi, int l, int r) {
        if (r < lo || hi < l) return INT_MIN;
        if (l <= lo && hi <= r) return seg[id].val;
        push(id);
        int mid = (lo + hi) / 2;
        return max(query(id * 2, lo, mid, l, r), query(id * 2 + 1,
            mid + 1, hi, l, r));
    }
    int query(int l, int r) {
        if (n == 0 || l > r) return INT_MIN;
        return query(1, 1, n, l, r);
    }
};
```

```
};
```

Persistent Segment Tree

Description: Persistent point assignment and range maximum on 1..N, inclusive [l,r]. Version 0 is uniformly initialValue (default 0). update(version,pos,val) copies the changed root-to-leaf path, returns a new version ID, and preserves every old version; branching is allowed. Node 0 represents an unchanged uniform subtree. Use node IDs, not references across push.back. Change max and unit for another aggregate; sums also need segment lengths for uniform subtrees.

Usage: PersistentSegTree seg(n); // initially all zero; require n>=1
int v1=seg.update(0,3,7); // assign a[3]=7
int v2=seg.update(v1,2,9);
int branch=seg.update(0,1,-5); // branch from the original version
ll best=seg.query(v2,2,3); // 9; seg.query(v1,2,3) is still 7
ll old=seg.query(0,1,n); // 0; positions and bounds are 1-based
Time: $\mathcal{O}(\log N)$ per update/query; $\mathcal{O}(1+U\log N)$ memory for U updates.

```
struct PersistentSegTree {
    static constexpr ll unit = LLONG_MIN;
    struct Node {
        int left, right;
        ll val;
    };
    int n;
    vector<Node> nodes;
    vector<int> roots = {0};
    PersistentSegTree(int n, ll initialValue = 0) : n(n), nodes(
        1, {0, 0, initialValue}) {}
    int updateNode(int id, int lo, int hi, int pos, ll val) {
        int cur = sz(nodes);
        nodes.push_back(nodes[id]);
        if (lo == hi) {
            nodes[cur].val = val;
            return cur;
        }
        int mid = (lo + hi) / 2;
        if (pos <= mid)
            nodes[cur].left = updateNode(nodes[id].left, lo, mid, pos
                , val);
        else
            nodes[cur].right = updateNode(nodes[id].right, mid + 1,
                hi, pos, val);
        nodes[cur].val = max(nodes[nodes[cur].left].val, nodes[
            nodes[cur].right].val);
        return cur;
    }
    int update(int version, int pos, ll val) {
        roots.push_back(updateNode(roots[version], 1, n, pos, val))
            ;
        return sz(roots) - 1;
    }
    ll queryNode(int id, int lo, int hi, int l, int r) const {
        if (r < lo || hi < l) return unit;
        if (id == 0 || (l <= lo && hi <= r)) return nodes[id].val;
        int mid = (lo + hi) / 2;
        return max(queryNode(nodes[id].left, lo, mid, l, r),
            queryNode(nodes[id].right, mid + 1, hi, l, r));
    }
    ll query(int version, int l, int r) const {
        if (l > r) return unit;
        return queryNode(roots[version], 1, n, l, r);
    }
};
```

Disjoint Set Union (DSU)

Description: Union by size with path compression, for vertices 1..n. get returns the component representative; s[get(u)] is its size. This version cannot roll back merges.

Usage: DSU dsu(5);
dsu.join(1,2);
bool same=dsu.joined(1,2); // true
int size=dsu.s[dsu.get(1)]; // 2
Time: $\mathcal{O}(N)$ initialization; amortized $\mathcal{O}(\alpha(N))$ per operation.

```
struct DSU {
    vector<int> par, s;
    DSU(int n) { init(n); }
    void init(int n) {
        par.resize(n + 1);
        iota(all(par), 0);
        s.assign(n + 1, 1);
    }
    int get(int u) { return par[u] == u ? u : par[u] = get(par[u]
        ); }
    void join(int u, int v) {
        u = get(u), v = get(v);
        if (u == v) return;
        if (s[u] < s[v]) swap(u, v);
        par[v] = u;
        s[u] += s[v];
    }
    bool joined(int u, int v) { return get(u) == get(v); }
};
```

Rollback DSU

Description: Rollback DSU on 0..n-1; allocate n+1 for labels 1..n. Save time() before temporary joins and rollback(checkpoint) afterward. Checkpoints are history lengths, not join counts. Do not add path compression unless its changes are also recorded.

Usage: RollbackDSU uf(n); int checkpoint=uf.time();
uf.join(u,v); bool connected=uf.find(u)==uf.find(v);
uf.rollback(checkpoint);
Time: $\mathcal{O}(\log(N))$

```
struct RollbackDSU {
    vector<int> parentOrSize;
    vector<pii> history;
    RollbackDSU(int n) : parentOrSize(n, -1) {}
    int size(int x) { return -parentOrSize[find(x)]; }
    int find(int x) { return parentOrSize[x] < 0 ? x : find(
        parentOrSize[x]); }
    int time() { return sz(history); }
    void rollback(int checkpoint) {
        for (int i = time(); i-- > checkpoint;) parentOrSize[
            history[i].first] = history[i].second;
        history.resize(checkpoint);
    }
    bool join(int a, int b) {
        a = find(a), b = find(b);
        if (a == b) return false;
        if (parentOrSize[a] > parentOrSize[b]) swap(a, b);
        history.push_back({a, parentOrSize[a]});
        history.push_back({b, parentOrSize[b]});
        parentOrSize[a] += parentOrSize[b];
        parentOrSize[b] = a;
        return true;
    }
};
```

Convex Hull Trick (CHT)

Description: Add lines of the form slope*x+intercept, and query maximum values at points x. Useful for dynamic programming (“convex hull trick”). Lines and queries may arrive in any order; x, slopes and intercepts are integers. Call add before the first query. For minimum queries, insert (-slope,-intercept) and negate the result. Products, differences and intersection arithmetic must fit ll. lastX marks the final integer x where a line is best. Floating variants need floating types throughout, infinite bounds and ordinary division.

Usage: CHT hull; hull.add(2,3); hull.add(-1,7);
ll best=hull.query(4); // max(11,3)=11
Time: $\mathcal{O}(\log N)$

```
struct CHTLine {
    mutable ll slope, intercept, lastX;
    bool operator<(const CHTLine &other) const { return slope <
        other.slope; }
    bool operator<(ll x) const { return lastX < x; }
};
struct CHT : multiset<CHTLine, less<>> {
    // (for doubles, use inf = 1/.0, div(a,b) = a/b)
    static const ll inf = LLONG_MAX;
    ll div(ll a, ll b) { // floored division
        return a / b - ((a ^ b) < 0 && a % b);
    }
    bool isect(iterator line, iterator nextLine) {
        if (nextLine == end()) return line->lastX = inf, 0;
        if (line->slope == nextLine->slope)
            line->lastX = line->intercept > nextLine->intercept ? inf
                : -inf;

        else
            line->lastX = div(nextLine->intercept - line->intercept,
                line->slope - nextLine->slope);
        return line->lastX >= nextLine->lastX;
    }
    void add(ll slope, ll intercept) {
        auto nextLine = insert({slope, intercept, 0}), line =
            nextLine++, prevLine = line;
        while (isect(line, nextLine)) nextLine = erase(nextLine);
        if (prevLine != begin() && isect(--prevLine, line)) isect(
            prevLine, line = erase(line));
        while ((line = prevLine) != begin() && (--prevLine)->lastX
            >= line->lastX)
            isect(prevLine, erase(line));
    }
    ll query(ll x) {
        assert(!empty());
        auto line = *lower_bound(x);
        return line.slope * x + line.intercept;
    }
};
```

Implicit Treap

Description: A short self-balancing tree. It acts as a sequential container with log-time splits/joins, and is easy to augment with additional data. This is an implicit treap: in-order position is the key. split(root,k) separates the first k values; merge(a,b) concatenates sequences. Positions and ranges are 0-based, with ranges [l,r]. Save roots returned by ins/merge, and recalc after changing children. Extend recalc for sums or lazy tags; push pending tags before split/merge descent. move’s targetPos is in the original sequence and must lie outside (left,right); moving inside is a no-op to handle first.

Usage: TreapNode *root=nullptr;
root=ins(root,new TreapNode(7),0);
auto [a,b]=split(root,1); root=merge(a,b);
Time: $\mathcal{O}(\log N)$

```
struct TreapNode {
    TreapNode *left = 0, *right = 0;
    int val, priority, size = 1;
    TreapNode(int val) : val(val), priority(rand()) {}
};
```

```
void recalc();
};
int cnt(TreapNode *node) { return node ? node->size : 0; }
void TreapNode::recalc() { size = cnt(left) + cnt(right) + 1; }
template <class F> void each(TreapNode *node, F visit) {
    if (node) {
        each(node->left, visit);
        visit(node->val);
        each(node->right, visit);
    }
}
pair<TreapNode *, TreapNode *> split(TreapNode *node, int leftCount) {
    if (!node) return {};
    if (cnt(node->left) >= leftCount) { // "node->val >= key" for lower_bound(k)
        auto [leftPart, rightPart] = split(node->left, leftCount);
        node->left = rightPart;
        node->recalc();
        return {leftPart, node};
    } else {
        auto [leftPart, rightPart] = split(node->right, leftCount - cnt(node->left) - 1); // and just "key"
        node->right = leftPart;
        node->recalc();
        return {node, rightPart};
    }
}
TreapNode *merge(TreapNode *left, TreapNode *right) {
    if (!left) return right;
    if (!right) return left;
    if (left->priority > right->priority) {
        left->right = merge(left->right, right);
        return left->recalc(), left;
    } else {
        right->left = merge(left, right->left);
        return right->recalc(), right;
    }
}
TreapNode *ins(TreapNode *root, TreapNode *node, int pos) {
    auto [left, right] = split(root, pos);
    return merge(merge(left, node), right);
}
// Example application: move the range [left, right) to index targetPos
void move(TreapNode *&root, int left, int right, int targetPos) {
    TreapNode *prefix, *middle, *suffix;
    tie(prefix, middle) = split(root, left);
    tie(middle, suffix) = split(middle, right - left);
    if (targetPos <= left)
        root = merge(ins(prefix, middle, targetPos), suffix);
    else
        root = merge(prefix, ins(suffix, middle, targetPos - right));
}
```

Fenwick Tree (BIT) + Walk

Description: 1-based sums: update(pos,delta) adds rather than assigns; query(pos) sums [1,pos] and query(l,r) is inclusive. lowerBound finds the first prefix reaching a target; it requires nonnegative elements and returns 0 for target<=0 or n+1 if absent. This is Fenwick walking (binary lifting): skip a block only if its whole sum is still below the target. With frequencies, lowerBound(k) locates the kth item, counting duplicates. Updates may be negative, but every resulting element must remain nonnegative for walking to work. Never update position 0. Range add/point query uses a difference BIT; range sums need two BITs.

```
Usage: Fenwick bit(5); bit.update(2,3); bit.update(4,7);
ll sum=bit.query(2,4); // 10
int pos=bit.lowerBound(4); // 4
// With 3 items at position 2 and 7 at position 4:
int kth=bit.lowerBound(5); // fifth item is at position 4
// To reach a positive target starting from position 1:
int l=3; ll target=2;
ll before=bit.query(l-1);
pos=bit.lowerBound(before+target);
// n+1 means insufficient sum in [l,n].
Time:  $\mathcal{O}(\log N)$  per operation;  $\mathcal{O}(N)$  memory.

```

```
struct Fenwick {
    int n;
    vector<ll> bit;
    Fenwick(int n = 0) : n(n), bit(n + 1, 0) {}
    void update(int pos, ll delta) {
        for (int i = pos; i <= n; i += i & -i) bit[i] += delta;
    }
    ll query(int pos) const {
        ll sum = 0;
        for (int i = pos; i > 0; i -= i & -i) sum += bit[i];
        return sum;
    }
    ll query(int l, int r) const {
        if (l > r) return 0;
        return query(r) - query(l - 1);
    }
}
int lowerBound(ll targetSum) const {
    if (targetSum <= 0) return 0;
    int pos = 0;
    int step = 1;
    while (step <= n / 2) step *= 2;
    // Walk by decreasing powers of two; targetSum is the remaining sum needed.
    for (; step > 0; step /= 2) {
        int next = pos + step;
        if (next <= n && bit[next] < targetSum) {
            // Skip this entire block; no prefix ending inside it can reach the target.
            pos = next;
            targetSum -= bit[next];
        }
    }
    return pos + 1;
}
};
```

2D Fenwick Tree (BIT)

Description: Offline 2D Fenwick tree. x is in [1, n]; y can be any integer. Register every update coordinate with fakeUpdate before init. query(x,y) sums points with first coordinate <= x and second coordinate <= y. Construct a new object to reset. All future update locations must be registered, even if their eventual values are unknown. Query coordinates need not be registered. Sum a rectangle [x1,x2] by [y1,y2] with four prefix queries using x1-1 and y1-1. Updates add a delta rather than assign a value.

Usage: Fenwick2D bit(n);
bit.fakeUpdate(2,10); bit.fakeUpdate(4,20); bit.init();
bit.update(2,10,5); ll sum=bit.query(3,15); // 5
Time: $\mathcal{O}(\log^2 N)$ per operation; $\mathcal{O}(U \log N)$ memory for U registered points.

```
"FenwickTree.h"
struct Fenwick2D {
    int n;
    vector<vector<int>>> yCoords;
    vector<Fenwick> bit;
    Fenwick2D(int n) : n(n), yCoords(n + 1), bit(n + 1) {}
    void fakeUpdate(int x, int y) {
        for (int i = x; i <= n; i += i & -i) yCoords[i].push_back(y);
    }
};
```

```
}
void init() {
    for (int i = 1; i <= n; i++) {
        sort(all(yCoords[i]));
        yCoords[i].erase(unique(all(yCoords[i])), yCoords[i].end());
        bit[i] = Fenwick(sz(yCoords[i]));
    }
}
void update(int x, int y, ll delta) {
    for (int i = x; i <= n; i += i & -i) {
        int pos = lower_bound(all(yCoords[i]), y) - yCoords[i].begin() + 1;
        bit[i].update(pos, delta);
    }
}
ll query(int x, int y) const {
    ll sum = 0;
    for (int i = x; i > 0; i -= i & -i) {
        int pos = upper_bound(all(yCoords[i]), y) - yCoords[i].begin();
        sum += bit[i].query(pos);
    }
    return sum;
}
};
```

Persistent Fenwick Tree (2D)

Description: Timestamped 2D Fenwick tree: point addition and historical rectangle sums. Coordinates are 1-based; rectangles are inclusive. Initially zero. Updates MUST arrive in nondecreasing time order; same-time additions are combined. Queries may use any time and include all updates at or before it. Only the latest state can be updated; branching is unsupported. Each BIT cell stores time,sum; binary search selects its historical value. The dense N*M history vectors can be expensive. For 1D, remove the y loops and use history[x]. Compress huge sparse coordinates first; prefix queries use upper_bound ranks.

Usage: PersistentFenwick2D bit(n,m);
bit.update(2,3,5,1); // add 5 at (2,3), time 1
bit.update(2,3,-2,2); // time 2; negative deltas are allowed
ll old=bit.query(1,1,1,n,m); // rectangle sum at time 1: 5
ll now=bit.query(2,n,m); // prefix rectangle [1,n] x [1,m] at time 2: 3
// Queries before the first update return zero; sums must fit ll.
Time: Amortized $\mathcal{O}(\log N \log M)$ update, $\mathcal{O}(\log N \log M \log(U + 1))$ query; $\mathcal{O}(NM + U \log N \log M)$ memory for U updates.

```
struct PersistentFenwick2D {
    int n, m;
    vector<vector<vector<pair<int, ll>>>> history;
    PersistentFenwick2D(int n, int m)
        : n(n), m(m), history(n + 1, vector<vector<pair<int, ll>>>(m + 1)) {}
    void update(int x, int y, ll delta, int time) {
        for (int i = x; i <= n; i += i & -i)
            for (int j = y; j <= m; j += j & -j) {
                auto &cell = history[i][j];
                ll value = (cell.empty() ? 0 : cell.back().second) + delta;
                if (!cell.empty() && cell.back().first == time)
                    cell.back().second = value;
                else
                    cell.push_back({time, value});
            }
    }
    ll query(int time, int x, int y) const {
        ll sum = 0;
        for (int i = x; i > 0; i -= i & -i)
            for (int j = y; j > 0; j -= j & -j) {
```

```
const auto &cell = history[i][j];
auto it = upper_bound(all(cell), time,
    [](int t, const pair<int, ll> &
        entry) { return t < entry.
            first; });
if (it != cell.begin()) sum += prev(it)->second;
}
return sum;
}
ll query(int time, int x1, int y1, int x2, int y2) const {
    if (x1 > x2 || y1 > y2) return 0;
    return query(time, x2, y2) - query(time, x1 - 1, y2) -
        query(time, x2, y1 - 1) +
        query(time, x1 - 1, y1 - 1);
}
};
```

Sparse Table (RMQ)

Description: Static minimum and maximum on 1-based inclusive [l,r], with l<=r. Input a[0] is ignored. Rebuild after updates; overlapping blocks work for min/max, not sums.

Usage: SparseTable rmq(vector<int>>{0,7,2,5,1});

int minimum=rmq.getMin(2,3); // 2

int maximum=rmq.getMax(2,3); // 5

Time: $\mathcal{O}(N \log N)$ construction and memory; $\mathcal{O}(1)$ per query.

```
struct SparseTable {
    vector<vector<int>> stMin, stMax;
    SparseTable(const vector<int> &a) : stMin(1, a), stMax(1, a)
    {
        int n = sz(a) - 1;
        for (int k = 1; (1LL << k) <= n; k++) {
            stMin.emplace_back(n + 1);
            stMax.emplace_back(n + 1);
            for (int i = 1; i + (1 << k) - 1 <= n; i++) {
                int j = i + (1 << (k - 1));
                stMin[k][i] = min(stMin[k - 1][i], stMin[k - 1][j]);
                stMax[k][i] = max(stMax[k - 1][i], stMax[k - 1][j]);
            }
        }
    }
    int getMin(int l, int r) const {
        int k = __lg(r - l + 1);
        return min(stMin[k][l], stMin[k][r - (1 << k) + 1]);
    }
    int getMax(int l, int r) const {
        int k = __lg(r - l + 1);
        return max(stMax[k][l], stMax[k][r - (1 << k) + 1]);
    }
};
```

Sliding-Window Minimum and Maximum

Description: Minimum and maximum of every length-k window in a static array. Deques store indices; remove expired indices from the front and dominated values from the back. Equal values keep the newest index. For 1<=k<=n, result[i] describes 0-based [i,i+k-1]. Invalid k returns two empty vectors. Use a monotonic stack instead for nearest smaller/greater queries.

Usage: vector<int> a={4,2,2,5,1};

auto [minimum,maximum]=slidingWindowMinMax(a,3);

// minimum={2,2,1}, maximum={4,5,5}

// For a 1-based array, remove the unused index 0 before calling.

Time: $\mathcal{O}(N)$ time; $\mathcal{O}(k)$ deque memory plus $\mathcal{O}(N)$ output.

```
template <class T> pair<vector<T>, vector<T>>
    slidingWindowMinMax(const vector<T> &a, int k) {
    vector<T> minimum, maximum;
    if (k <= 0 || k > sz(a)) return {minimum, maximum};
    deque<int> minQ, maxQ;
    for (int i = 0; i < sz(a); i++) {
```

```
while (!minQ.empty() && minQ.front() <= i - k) minQ.
    pop_front();
while (!maxQ.empty() && maxQ.front() <= i - k) maxQ.
    pop_front();
while (!minQ.empty() && a[minQ.back()] >= a[i]) minQ.
    pop_back();
while (!maxQ.empty() && a[maxQ.back()] <= a[i]) maxQ.
    pop_back();
minQ.push_back(i);
maxQ.push_back(i);
if (i >= k - 1) {
    minimum.push_back(a[minQ.front()]);
    maximum.push_back(a[maxQ.front()]);
}
}
return {minimum, maximum};
}
```

Mo’s Algorithm on Arrays

Description: Offline inclusive queries [l,r] on a[1..N]; require 1<=l<=r<=N. Fill add/del/calc and begin with empty state; side=0/1 means left/right endpoint. No value updates between queries; answers follow input order. The included Hilbert helper precomputes sorting keys without block-size tuning. bits covers every endpoint; int indices fit all keys. A single-element query has l=r. Compress values for frequency arrays; queries mixed with updates need Mo with a time dimension.

Usage: auto answers=mo({{1,3},{2,5},{3,3}});

// Fill add/del/calc; [1,3] means positions 1,2,3; [3,3] means only position 3.

Time: $\mathcal{O}(Q \log Q + Q \log(N + 1) + N\sqrt{Q})$ with $\mathcal{O}(1)$ add/del/calc; $\mathcal{O}(Q + \log(N + 1))$ memory excluding custom state.

```
inline ull hilbertOrder(int x, int y, int bits, int rotation =
    0) {
    if (bits == 0) return 0;
    int half = 1 << (bits - 1);
    int quadrant = x < half ? (y < half ? 0 : 3) : (y < half ? 1
        : 2);
    quadrant = (quadrant + rotation) & 3;
    const int turn[4] = {3, 0, 0, 1};
    int nextRotation = (rotation + turn[quadrant]) & 3;
    ull squareSize = 1ULL << (2 * bits - 2);
    ull offset = hilbertOrder(x & (half - 1), y & (half - 1),
        bits - 1, nextRotation);
    if (quadrant == 0 || quadrant == 3) offset = squareSize -
        offset - 1;
    return quadrant * squareSize + offset;
}
void add(int pos, int side) { ... } // add a[pos]
void del(int pos, int side) { ... } // remove a[pos]
int calc(){...} // current answer
vector<int> mo(vector<pii> queries) {
    int left = 1, right = 0; // Initially empty; maintained
        interval is [left,right].
    int maxEndpoint = 0, bits = 0;
    for (auto [l, r] : queries) maxEndpoint = max(maxEndpoint, r)
        ;
    while ((1LL << bits) <= maxEndpoint) bits++;
    vector<int> order(sz(queries)), answers(sz(queries));
    vector<ull> key(sz(queries));
    for (int i = 0; i < sz(queries); i++)
        key[i] = hilbertOrder(queries[i].first, queries[i].second,
            bits);
    iota(all(order), 0);
    sort(all(order), [&](int queryA, int queryB) { return key[
        queryA] < key[queryB]; });
    for (int queryId : order) {
        auto [queryLeft, queryRight] = queries[queryId];
        while (left > queryLeft) add(--left, 0);
        while (right < queryRight) add(++right, 1);
```

```
while (left < queryLeft) del(left++, 0);
while (right > queryRight) del(right--, 1);
answers[queryId] = calc();
}
return answers;
}
```

Mo’s Algorithm on Trees

Description: KACTL’s direct endpoint walking with alternating DFS order and snake sorting. Inclusive vertex paths on 1..N; adj has size N+1, both edge directions, and default root=1. Fill add/del/calc; side=0/1 selects the endpoint. Begin with empty state; reset before reuse. No value updates between queries. rank becomes a climb stack after sorting; no LCA table is needed. Both endpoints are included, even for (u,u). Answers follow input order. DFS stack can be linear. For edge values, process the traversed edge in step and omit add(root,0), keeping active[root]. Block size is near N/sqrt(Q); adjust if needed.

Usage: // Fill add/del/calc for the desired statistic (e.g. distinct vertex values).

vector<pii> queries={{u,v},{u,u}};

auto answers=moTree(queries,adj); // 1-based vertices, both

path endpoints included

// For another root in 1..N: moTree(queries,adj,root).

// Compress values first if callbacks index a frequency array

by value.

Time: $\mathcal{O}(N + Q \log Q + N\sqrt{Q})$ with $\mathcal{O}(1)$ add/del/calc; $\mathcal{O}(N + Q)$ mem-
ory.

void add(int u, int side) { ... } // Add vertex u at this end
of the path.

void del(int u, int side) { ... } // Remove vertex u at this
end of the path.

int calc(){...} // Current path answer.

vector<int> moTree(const vector<pii> &queries, const vector<
vector<int>> &adj, int root = 1) {

if (queries.empty()) return {};

int n = sz(adj) - 1, timer = 1, pos[2] = {root, root};

int blockSize = max(1, (int)(n / sqrt(sz(queries))));

vector<int> order(sz(queries)), answers(sz(queries));

vector<int> rank(n + 1), tin(n + 1), tout(n + 1), parent(n +
1);

vector<bool> active(n + 1);

add(root, 0);

active[root] = true;

auto dfs = [&](int u, int p, bool oddDepth, auto &self) ->
void {

parent[u] = p;

tin[u] = timer;

if (oddDepth) rank[u] = timer++;

for (int v : adj[u])

if (v != p) self(v, u, !oddDepth, self);

if (!oddDepth) rank[u] = timer++;

tout[u] = timer - 1; // Inclusive subtree bounds in the
alternating DFS order.

};

dfs(root, 0, false, dfs);

iota(all(order), 0);

sort(all(order), [&](int a, int b) {

int blockA = rank[queries[a].first] / blockSize;

int blockB = rank[queries[b].first] / blockSize;

if (blockA != blockB) return blockA < blockB;

int rightA = rank[queries[a].second], rightB = rank[queries
[b].second];

return blockA & 1 ? rightA > rightB : rightA < rightB;

});

for (int id : order) {

for (int side = 0; side < 2; side++) {
int ¤t = pos[side];
int target = side == 0 ? queries[id].first : queries[id].
second;

```
int climbed = 0;
auto step = [&](int next) {
    if (active[next]) {
        del(current, side);
        active[current] = false;
    } else {
        add(next, side);
        active[next] = true;
    }
    current = next;
};
// Climb target until it is an ancestor of current; save
// its descent path.
while (!(tin[target] <= tin[current] && tout[current] <=
    tout[target])) {
    rank[++climbed] = target; // DFS ranks are no longer
    // needed after sorting.
    target = parent[target];
}
while (current != target) step(parent[current]);
while (climbed) step(rank[climbed--]);
}
answers[id] = calc();
}
return answers;
}
```

Graph (7)

7.1 Fundamentals

Dial’s Algorithm (Small Integer Weights)

Description: Single-source shortest paths for integer weights in [0,K], with small K. Use K+1 cyclic buckets; current is the real distance, only bucket indices use modulo. Skip stale entries. K=0 and zero-weight edges work; K=1 covers 0-1 BFS. Allocate adj for your indexing; unreachable distances are LLONG_MAX. Negative weights are invalid. Large K needs too much scanning/memory; use heap Dijkstra.

Usage: vector<vector<pii>> adj(n+1); // {neighbor,weight}, weights 0..9

adj[u].push_back({v,3}); // add reverse edge too if undirected

auto dist=dial(adj,1,9); // source=1, K=9

if (dist[target]!=LLONG_MAX) cout << dist[target];

// Use dial(adj,source,1) for weights only 0 and 1.

Time: $\mathcal{O}(N + M + NK)$ time; $\mathcal{O}(N + M + K)$ auxiliary memory including queued entries.

```
vector<ll> dial(const vector<vector<pii>> &adj, int source, int
    K) {
    int bucketCount = K + 1;
    vector<queue<int>> buckets(bucketCount);
    vector<ll> dist(sz(adj), LLONG_MAX);
    ll current = 0, pending = 1;
    dist[source] = 0;
    buckets[0].push(source);
    while (pending) {
        while (buckets[current % bucketCount].empty()) current++;
        auto &bucket = buckets[current % bucketCount];
        int u = bucket.front();
        bucket.pop();
        pending--;
        if (dist[u] != current) continue; // Ignore an outdated
        // queued distance.
        for (auto [v, weight] : adj[u]) {
            ll nextDist = current + weight;
            if (nextDist >= dist[v]) continue;
            dist[v] = nextDist;
            buckets[nextDist % bucketCount].push(v);
            pending++;
        }
    }
}
```

```
}
}
return dist;
}
```

Bellman-Ford Shortest Paths

Description: Calculates shortest paths from s in a graph that might have negative edge weights. Unreachable nodes get $\text{dist} = \text{inf}$; nodes reachable through negative-weight cycles get $\text{dist} = -\text{inf}$. Assumes $V^2 \max |w_i| < \sim 2^{63}$. Use fresh default-initialized nodes for each run; allocate $n+1$ for labels $1..n$. Edges are directed; add both directions for an undirected graph. prev gives a predecessor only for finite shortest paths. The algorithm sorts edges in place; do not reconstruct paths through negative cycles.

Usage: vector<BellmanFordState> nodes(3);

vector<BellmanFordEdge> edges={{0,1,-2},{1,2,5}};

bellmanFord(nodes,edges,0); // nodes[2].dist==3

Time: $\mathcal{O}(VE)$

```
const ll inf = LLONG_MAX;
struct BellmanFordEdge {
    int from, to, weight, sortKey() { return from < to ? from : -
        from; }
};
struct BellmanFordState {
    ll dist = inf;
    int prev = -1;
};
void bellmanFord(vector<BellmanFordState> &nodes, vector<
    BellmanFordEdge> &edges, int source) {
    nodes[source].dist = 0;
    sort(all(edges), [](BellmanFordEdge first, BellmanFordEdge
        second) {
        return first.sortKey() < second.sortKey();
    });
    int rounds = sz(nodes) / 2 + 2; // /3+100 with shuffled
    // vertices
    for (int i = 0; i < (rounds); ++i)
        for (BellmanFordEdge edge : edges) {
            BellmanFordState fromState = nodes[edge.from], &toState =
                nodes[edge.to];
            if (abs(fromState.dist) == inf) continue;
            ll newDist = fromState.dist + edge.weight;
            if (newDist < toState.dist) {
                toState.prev = edge.from;
                toState.dist = (i < rounds - 1 ? newDist : -inf);
            }
        }
    for (int i = 0; i < (rounds); ++i)
        for (BellmanFordEdge edge : edges)
            if (nodes[edge.from].dist == -inf) nodes[edge.to].dist =
                -inf;
}
```

Topological Sort

Description: DAG skeleton with vertices $1..n$: define maxN and read n,m first. The stack pops vertices in topological order; fill $\text{ans}[u]$ with its position when popping (the skeleton does not fill it). A detected cycle prints an error and exits. Clear g , visited and topo between cases. To handle cycles without exiting, return a failure flag; deep graphs need iterative DFS or Kahn’s algorithm.

Usage: // Define maxN and read n,m before the shown main body.

// Pop topo to obtain the order and assign $\text{ans}[u]=++\text{position}$.

// Reset visited and clear g for another test case.

```
int visited[maxN], ans[maxN];
vector<int> g[maxN];
stack<int> topo; // reversed order of topo;
void dfs(int u) {
    visited[u] = 1;
```

```
for (auto v : g[u]) {
    if (visited[v] == 1) {
        cout << "Error: graph contains a cycle";
        exit(0);
    }
    if (!visited[v]) dfs(v);
}
topo.push(u);
visited[u] = 2;
}
int main() {
    while (m--) {
        int u, v; cin >> u >> v;
        g[u].push_back(v);
    }
    for (int i = 1; i <= n; ++i)
        if (!visited[i]) dfs(i);
}
```

7.2 Matching

Hopcroft-Karp Matching

Description: Use for the largest number of disjoint allowed pairs between two groups, e.g. workers/jobs or students/projects, with at most one pair per vertex and no costs. A perfect assignment of all left vertices exists iff the result equals the left size. adj lists right neighbors for each left vertex; the two sides use separate 1-based indices. adj has $N+1$ entries; $\text{adj}[0]$ is unused. Initialize matchRight with $M+1$ entries of -1 , with index 0 unused. Returns matching size; $\text{matchRight}[v]$ is its matched left vertex, or -1 .

Usage: vector<vector<int>> adj={{},{1,2},{2}};

vector<int> matchRight(3,-1);

int size=hopcroftKarp(adj,matchRight); // 2

Time: $\mathcal{O}(E\sqrt{V})$

```
int hopcroftKarp(vector<vector<int>> &adj, vector<int> &
    matchRight) {
    int n = sz(adj), matchingSize = 0;
    vector<int> matchLeft(n, -1), queue(n), dist(n);
    auto dfs = [&](auto self, int u) -> bool {
        int nextDist = exchange(dist[u], 0) + 1;
        for (int v : adj[u])
            if (matchRight[v] == -1 || (dist[matchRight[v]] ==
                nextDist && self(self, matchRight[v])))
                return matchLeft[u] = v, matchRight[v] = u, 1;
        return 0;
    };
    for (int queueSize = 0, foundPath = 0;; queueSize = foundPath
        = 0, dist.assign(n, 0)) {
        for (int i = 1; i < n; ++i)
            if (matchLeft[i] == -1) queue[queueSize++] = i, dist[i] =
                1;
        for (int i = 0; i < (queueSize); ++i)
            for (int v : adj[queue[i]])
                if (matchRight[v] == -1)
                    foundPath = 1;
                else if (!dist[matchRight[v]])
                    dist[matchRight[v]] = dist[queue[i]] + 1, queue[
                        queueSize++] = matchRight[v];
            if (!foundPath) return matchingSize;
        for (int i = 1; i < n; ++i)
            if (matchLeft[i] == -1) matchingSize += dfs(dfs, i);
    }
}
```

Bipartite Vertex Cover

Description: Use to select the fewest vertices covering every edge in a bipartite graph, e.g. minimum rows/columns covering marked cells. Cover size equals maximum matching size. adj has n+1 entries, listing right indices 1..m; index 0 is unused. Returned IDs <=n are left vertices; IDs n+v denote right vertex v. Vertices outside the cover form a maximum independent set.

Usage: auto vertices=cover(adj,n,m);
// id<=n: left id; otherwise right id-n (both 1-based).

Time: $\mathcal{O}\left(E\sqrt{V}\right)$

```
"HopcroftKarp.h"
vector<int> cover(vector<vector<int>> &adj, int n, int m) {
    vector<int> matchRight(m + 1, -1);
    int matchingSize = hopcroftKarp(adj, matchRight);
    vector<bool> reachableLeft(n + 1, true), reachableRight(m + 1);
    for (int matchedLeft : matchRight)
        if (matchedLeft != -1) reachableLeft[matchedLeft] = false;
    vector<int> stack, cover;
    for (int i = 1; i <= n; ++i)
        if (reachableLeft[i]) stack.push_back(i);
    while (!stack.empty()) {
        int i = stack.back();
        stack.pop_back();
        reachableLeft[i] = 1;
        for (int v : adj[i])
            if (!reachableRight[v] && matchRight[v] != -1) {
                reachableRight[v] = true;
                stack.push_back(matchRight[v]);
            }
    }
    for (int i = 1; i <= n; ++i)
        if (!reachableLeft[i]) cover.push_back(i);
    for (int i = 1; i <= m; ++i)
        if (reachableRight[i]) cover.push_back(n + i);
    assert(sz(cover) == matchingSize);
    return cover;
}
```

Hungarian Algorithm (Assignment)

Description: Minimum-cost assignment of every left vertex to a distinct right vertex. Use when each worker/job pair has a cost and every worker must be assigned once. For maximum pair count without costs, use HopcroftKarp; for capacities or partial assignments with costs, use MinCostMaxFlow. costs has size (N+1)*(M+1), with row/column 0 unused and N<=M. Returns totalCost,match, where match has N+1 entries, index 0 unused, and match[i] is the assigned column. Negate costs for maximum cost. Missing edges need a large finite cost and a check afterward. Use ll if costs/potentials/totals may exceed int. N>M cannot assign every row; transpose only if assigning every vertex on the smaller side.

Usage: vector<vector<ll>> costs={{0,0,0},{0,3,1},{0,2,4}};
auto {minCost,match}=weightedMatching(costs); // cost=3
// Row i is assigned to column match[i], both 1-based.

Time: $\mathcal{O}\left(N^2M\right)$

```
template <class T> pair<T, vector<int>> weightedMatching(vector
<vector<T>> &costs) {
    int i = sz(costs) - 1, m = sz(costs[0]) - 1, col, scannedCols
    , row;
    vector<T> dist(m + 1), potential(m + 1);
    vector<int> matchLeft(i + 1), matchRight(m + 1, -1), colOrder
    (m + 1), parentRow(m + 1);
    T minDist = 0, newDist, cost = 0;
    while (i > 0) {
        for (int col = 1; col <= m; ++col)
            dist[col] = costs[i][col], colOrder[col] = col, parentRow
            [col] = i;
        for (scannedCols = 1;;) {
            for (int j = scannedCols; j <= m; ++j) {
```

```
                col = colOrder[j], newDist = dist[col] - potential[col
                ];
                if (j == scannedCols || minDist > newDist)
                    minDist = newDist, swap(colOrder[scannedCols],
                    colOrder[j]);
            }
            if ((row = matchRight[col = colOrder[scannedCols++]]) ==
                -1) break;
            for (int j = 1; j <= m; ++j)
                if (dist[j] > (newDist = costs[row][j] - costs[row][col
                ] + dist[col]))
                    dist[j] = newDist, parentRow[j] = row;
        }
        cost += dist[col];
        while (--scannedCols) potential[colOrder[scannedCols]] =
            dist[colOrder[scannedCols]] - minDist;
        for (; row != i; swap(col, matchLeft[row])) row =
            matchRight[col] = parentRow[col];
        --i;
    }
    return {cost, matchLeft};
}
```

7.3 Flow

Dinic Max Flow

Description: Flow algorithm with complexity $\mathcal{O}(VE\log U)$ where $U = \max|\text{cap}|$. $\mathcal{O}(\min(E^{1/2}, V^{2/3})E)$ if $U = 1$; $\mathcal{O}(\sqrt{VE})$ for bipartite matching. Dinic(n) uses vertices 0..n-1; capacities are nonnegative, with distinct source and sink. Discard self-loops; sums of capacities must fit ll. calc returns additional max flow and changes residual capacities. After calc, leftOfMinCut tests membership in the source side; Edge::flow gives positive flow. See MinCut.h for code to list both sides and the cut edges using original capacities. Fresh object for another network/source-sink pair; lower bounds need a circulation reduction. Full 64-bit capacity scaling: change 31 phases/shift 30 to 63 phases/shift 62.

Usage: Dinic d(3); d.addEdge(0,1,5); d.addEdge(1,2,3);
ll flow=d.calc(0,2); // 3; use reverse capacity c for an undirected edge

```
struct Dinic {
    struct Edge {
        int to, rev;
        ll c, oc;
        ll flow() { return max(oc - c, 0LL); } // if you need flows
    };
    vector<int> lvl, ptr, q;
    vector<vector<Edge>> adj;
    Dinic(int n) : lvl(n), ptr(n), q(n), adj(n) {}
    void addEdge(int a, int b, ll c, ll rcap = 0) {
        adj[a].push_back({b, sz(adj[b]), c, c});
        adj[b].push_back({a, sz(adj[a]) - 1, rcap, rcap});
    }
    ll dfs(int v, int t, ll f) {
        if (v == t || !f) return f;
        for (int &i = ptr[v]; i < sz(adj[v]); i++) {
            Edge &e = adj[v][i];
            if (lvl[e.to] == lvl[v] + 1)
                if (ll p = dfs(e.to, t, min(f, e.c))) {
                    e.c -= p, adj[e.to][e.rev].c += p;
                    return p;
                }
        }
        return 0;
    }
    ll calc(int s, int t) {
        ll flow = 0;
        q[0] = s;
        for (int L = 0; L < (31); ++L) do { // 'int L=30' maybe
            faster for random data
```

```
            lvl = ptr = vector<int>(sz(q));
            int qi = 0, qe = lvl[s] = 1;
            while (qi < qe && !lvl[t]) {
                int v = q[qi++];
                for (Edge e : adj[v])
                    if (!lvl[e.to] && e.c >> (30 - L))
                        q[qe++] = e.to, lvl[e.to] = lvl[v] + 1;
            }
            while (ll p = dfs(s, t, LLONG_MAX)) flow += p;
        } while (lvl[t]);
        return flow;
    }
    bool leftOfMinCut(int a) { return lvl[a] != 0; }
};
```

Minimum Cut from Max Flow

Description: To extract a minimum s - t cut, first run max-flow to completion. Let S be the vertices reachable from s using positive residual capacity; T is the rest. Dinic stores this final reachability: d.leftOfMinCut(v) is true exactly for v in S. The cut consists of original directed edges from S to T, and its capacity equals max flow. Sum original capacities (oc), not residual capacities (c), which are zero on these edges. Zero-capacity reverse edges are bookkeeping, not original cut edges. With addEdge(u,v,c,c), an undirected crossing edge is counted once, in the direction from S to T. For other flow implementations, BFS/DFS from s in the final residual graph gives S: follow e.c > 0 in Dinic, or e.cap-e.flow > 0 in MinCostMaxFlow. Include reverse edges.

Usage: Dinic d(3); int s=0, t=2;
d.addEdge(0,1,5); d.addEdge(1,2,3);
ll flow=d.calc(s,t); // 3; finish max-flow before extracting the cut
vector<int> sourceSide, sinkSide;
for (int v=0; v<3; ++v)
(d.leftOfMinCut(v) ? sourceSide : sinkSide).push_back(v);
// sourceSide={0,1}, sinkSide={2}
vector<pair<int,int>> cutEdges;
ll cutCapacity=0;
for (int u=0; u<3; ++u) if (d.leftOfMinCut(u))
for (auto &e : d.adj[u])
if (!d.leftOfMinCut(e.to) && e.oc>0) {
 cutEdges.push_back({u,e.to});
 cutCapacity+=e.oc;
}
// cutEdges={{1,2}}; parallel edges appear separately
assert(cutCapacity==flow); // for a fresh network
Time: $\mathcal{O}(V + E)$ after max-flow.

Min-Cost Max-Flow

Description: Min-cost max-flow. If costs can be negative, call setpi before maxflow, but note that negative cost cycles are not supported. To obtain the actual flow, look at positive values only. MinCostMaxFlow(n) uses vertices 0..n-1; capacities nonnegative, source and sink distinct. Build all edges before solving; maxflow changes edge flows and returns flow,cost. For exactly F units, add a new source with an edge of capacity F to s; require returned flow F. Distances/potentials must stay within INF; products and doubled totalCost must fit ll.

Usage: MinCostMaxFlow m(3); m.addEdge(0,1,2,-1);
m.addEdge(1,2,2,3);
m.setpi(0); auto [flow,cost]=m.maxflow(0,2); // {2,4}; solve once
Time: $\mathcal{O}(FE\log(V))$ where F is max flow. $\mathcal{O}(VE)$ for setpi.

#include <ext/pb_ds/priority_queue.hpp>

```
const ll INF = numeric_limits<ll>::max() / 4;
struct MinCostMaxFlow {
    struct Edge {
        int from, to, rev;
        ll cap, cost, flow;
```

```
};
int n;
vector<vector<Edge>> adj;
vector<int> seen;
vector<ll> dist, potential;
vector<Edge *> parentEdge;
MinCostMaxFlow(int n) : n(n), adj(n), seen(n), dist(n),
    potential(n), parentEdge(n) {}
void addEdge(int from, int to, ll cap, ll cost) {
    if (from == to) return;
    adj[from].push_back(Edge{from, to, sz(adj[to]), cap, cost,
        0});
    adj[to].push_back(Edge{to, from, sz(adj[from]) - 1, 0, -
        cost, 0});
}
void path(int s) {
    fill(all(seen), 0);
    fill(all(dist), INF);
    dist[s] = 0;
    ll adjustedDist;
    __gnu_pbds::priority_queue<pair<ll, int>> pq;
    vector<decltype(pq)::point_iterator> handles(n);
    pq.push({0, s});

    while (!pq.empty()) {
        s = pq.top().second;
        pq.pop();
        seen[s] = 1;
        adjustedDist = dist[s] + potential[s];
        for (Edge &e : adj[s])
            if (!seen[e.to]) {
                ll newDist = adjustedDist - potential[e.to] + e.cost;
                if (e.cap - e.flow > 0 && newDist < dist[e.to]) {
                    dist[e.to] = newDist;
                    parentEdge[e.to] = &e;
                    if (handles[e.to] == pq.end())
                        handles[e.to] = pq.push({-dist[e.to], e.to});
                    else
                        pq.modify(handles[e.to], {-dist[e.to], e.to});
                }
            }
        for (int i = 0; i < n; ++i) potential[i] = min(potential[i]
            + dist[i], INF);
    }
}
pair<ll, ll> maxflow(int s, int t) {
    ll totalFlow = 0, totalCost = 0;
    while (path(s), seen[t]) {
        ll pushed = INF;
        for (Edge *edge = parentEdge[t]; edge; edge = parentEdge[
            edge->from])
            pushed = min(pushed, edge->cap - edge->flow);

        totalFlow += pushed;
        for (Edge *edge = parentEdge[t]; edge; edge = parentEdge[
            edge->from]) {
            edge->flow += pushed;
            adj[edge->to][edge->rev].flow -= pushed;
        }
    }
    for (int i = 0; i < n; ++i)
        for (Edge &e : adj[i]) totalCost += e.cost * e.flow;
    return {totalFlow, totalCost / 2};
}
// If some costs can be negative, call this before maxflow:
void setpi(int s) { // (otherwise, leave this out)
    fill(all(potential), INF);
    potential[s] = 0;
```

```
int roundsLeft = n, changed = 1;
ll newPotential;
while (changed-- && roundsLeft--)
    for (int i = 0; i < n; ++i)
        if (potential[i] != INF)
            for (Edge &e : adj[i])
                if (e.cap)
                    if ((newPotential = potential[i] + e.cost) <
                        potential[e.to])
                        potential[e.to] = newPotential, changed = 1;
    assert(roundsLeft >= 0); // negative cost cycle
};
```

Global Min Cut

Description: Find a global minimum cut in an undirected graph, as represented by an adjacency matrix. Matrix has size N*N, indexed 0..N-1. It is symmetric, nonnegative, with zero diagonal and N>=2. Returned vertices are 0-based. Returns cut weight, vertices on one side; copies the matrix. Disconnected graphs give 0. Weight sums must fit int; otherwise use ll matrix/weights/result and LLONG-MIN/MAX sentinels. Usage: vector<vector<int>> mat={{0,3},{3,0}}; auto [weight,side]=globalMinCut(mat); // weight=3 Time: $\mathcal{O}(V^3)$

```
pair<int, vector<int>> globalMinCut(vector<vector<int>> mat) {
    pair<int, vector<int>> best = {INT_MAX, {}};
    int n = sz(mat);
    vector<vector<int>> co(n);
    for (int i = 0; i < n; ++i) co[i] = {i};
    for (int ph = 1; ph < (n); ++ph) {
        vector<int> w = mat[0];
        size_t s = 0, t = 0;
        for (int it = 0; it < (n - ph); ++it) { //  $\mathcal{O}(V^2) \rightarrow \mathcal{O}(E \log V)$  with prio. queue
            w[it] = INT_MIN;
            s = t, t = max_element(w.begin(), w.end()) - w.begin();
            for (int i = 0; i < n; ++i) w[i] += mat[t][i];
        }
        best = min(best, {w[t] - mat[t][t], co[t]});
        co[s].insert(co[s].end(), all(co[t]));
        for (int i = 0; i < n; ++i) mat[s][i] += mat[t][i];
        for (int i = 0; i < n; ++i) mat[i][s] = mat[s][i];
        mat[0][t] = INT_MIN;
    }
    return best;
}
```

7.4 DFS algorithms

Strongly Connected Components (SCC)

Description: Directed SCCs; default vertices 1..n, or pass firstVertex=0. comp[u] is a 0-based component ID and components[id] lists its vertices. Inter-component edges go to smaller IDs; process IDs in descending order for topological order. Build a new object after changing edges. Usage: SCC scc(adj); bool same=scc.comp[u]==scc.comp[v]; for (int id=sz(scc.components)-1;id>=0;id--) { // Process scc.components[id] in topological order. } Time: $\mathcal{O}(N + E)$ time and memory.

```
struct SCC {
    int timer = 0;
    vector<int> num, low, comp, activeStack;
    vector<bool> inStack;
    vector<vector<int>> components;
    SCC(const vector<vector<int>> &adj, int firstVertex = 1) {
        int n = sz(adj);
        num.assign(n, 0);
        low.assign(n, 0);
```

```
comp.assign(n, -1);
inStack.assign(n, false);
for (int u = firstVertex; u < n; u++)
    if (num[u] == 0) dfs(adj, u);
}
void dfs(const vector<vector<int>> &adj, int u) {
    timer++;
    num[u] = timer;
    low[u] = timer;
    activeStack.push_back(u);
    inStack[u] = true;
    for (int v : adj[u]) {
        if (num[v] == 0) {
            dfs(adj, v);
            low[u] = min(low[u], low[v]);
        } else if (inStack[v]) {
            low[u] = min(low[u], num[v]);
        }
    }
    if (low[u] != num[u]) return;
    int id = sz(components);
    components.push_back({});
    while (true) {
        int v = activeStack.back();
        activeStack.pop_back();
        inStack[v] = false;
        comp[v] = id;
        components[id].push_back(v);
        if (v == u) break;
    }
}
};
```

Biconnected Components

Description: Finds all biconnected components in an undirected graph, and runs a callback for the edges in each. In a biconnected component there are at least two internally disjoint paths between any two nodes (a cycle exists through them). Note that a node can be in several components. An edge which is not in a component is a bridge, i.e., not part of any cycle. Assign each undirected edge a unique ID and store that same ID at both endpoints, including parallel edges. The callback receives edge IDs, not vertex IDs. This implementation omits bridges; process the marked bridge branch separately if needed. Clear edgeStack and reset timer before a new graph. Self-loops are omitted; emit them separately if required. Usage: adj.assign(n,{}); int id=0; // For each (u,v), add (v,id) and (u,id) at both ends, then id++. timer=0; edgeStack.clear(); bicomps([](const vector<int> &ids) { ... }); Time: $\mathcal{O}(E + V)$

```
vector<int> num, edgeStack;
vector<vector<pii>> adj;
int timer;
template <class F> int dfs(int u, int parentEdge, F &
    visitComponent) {
    int entryTime = num[u] = ++timer, low = entryTime;
    for (auto [v, edgeId] : adj[u])
        if (edgeId != parentEdge) {
            if (num[v]) {
                low = min(low, num[v]);
                if (num[v] < entryTime) edgeStack.push_back(edgeId);
            } else {
                int stackStart = sz(edgeStack);
                int childLow = dfs(v, edgeId, visitComponent);
                low = min(low, childLow);
                if (childLow == entryTime) {
                    edgeStack.push_back(edgeId);
                    visitComponent(vector<int>(edgeStack.begin() +
                        stackStart, edgeStack.end()));
```

```
        edgeStack.resize(stackStart);
    } else if (childLow < entryTime)
        edgeStack.push_back(edgeId);
    else { /* e is a bridge */
    }
}
}
return low;
}
template <class F> void bicomps(F visitComponent) {
    num.assign(sz(adj), 0);
    for (int i = 0; i < (sz(adj)); ++i)
        if (!num[i]) dfs(i, -1, visitComponent);
}
```

2-SAT

Description: 0-based variables; literals x and ~x mean true and false. either(a,b) adds a OR b; implication a=>b is either(~a,b). setValue fixes a literal true. Read values only if solve() succeeds. atMostOne can add auxiliary variables. For 1-based labels, convert x to x-1 before complementing: negative literal is ~(x-1).

Usage: TwoSat ts(3); ts.either(0,~1); ts.setValue(2); bool possible=ts.solve(); if (possible) { int valueOfZero=ts.values[0]; } **Time:** $\mathcal{O}(N + E)$, where N is the number of boolean variables, and E is the number of clauses.

```
"scc.h"
struct TwoSat {
    int variableCount;
    vector<vector<int>> adj;
    vector<int> values; // 0 = false, 1 = true
    TwoSat(int variableCount = 0) : variableCount(variableCount),
        adj(2 * variableCount) {}
    int addVar() { // (optional)
        adj.emplace_back();
        adj.emplace_back();
        return variableCount++;
    }
    void either(int literalA, int literalB) {
        literalA = max(2 * literalA, -1 - 2 * literalA);
        literalB = max(2 * literalB, -1 - 2 * literalB);
        adj[literalA].push_back(literalB ^ 1);
        adj[literalB].push_back(literalA ^ 1);
    }
    void setValue(int x) { either(x, x); }
    void atMostOne(const vector<int> &literals) { // (optional)
        if (sz(literals) <= 1) return;
        int currentLiteral = ~literals[0];
        for (int i = 2; i < (sz(literals)); ++i) {
            int auxiliaryVar = addVar();
            either(currentLiteral, ~literals[i]);
            either(currentLiteral, auxiliaryVar);
            either(~literals[i], auxiliaryVar);
            currentLiteral = ~auxiliaryVar;
        }
        either(currentLiteral, ~literals[1]);
    }
    bool solve() {
        SCC scc(adj, 0);
        values.assign(variableCount, 0);
        for (int i = 0; i < variableCount; i++) {
            if (scc.comp[2 * i] == scc.comp[2 * i + 1]) return false;
            // Node 2*i means false; node 2*i+1 means true.
            values[i] = scc.comp[2 * i] > scc.comp[2 * i + 1];
        }
        return true;
    }
};
```

Euler Walk

Description: Hierholzer traversal uses every edge exactly once. Vertices are 0-based and edge IDs are 0..edgeCount-1. For undirected edges, add both directions with the same ID; for directed edges, add one entry. For an open trail, choose an odd-degree start in an undirected graph, or a vertex with outdegree=indegree+1 in a directed graph. For a cycle choose a vertex incident to an edge. Returns edgeCount+1 vertices or an empty vector on failure. Open trails have different endpoints. For 1-based vertices allocate n+1 and pass source explicitly; edge IDs remain 0-based. To require a cycle, also check walk.front()==walk.back().

Usage: vector<vector<pii>> adj(3); adj[0].push_back({1,0}); adj[1].push_back({2,1}); auto walk=eulerWalk(adj,2,0); // directed: {0,1,2}

```
Time:  $\mathcal{O}(V + E)$ 
vector<int> eulerWalk(vector<vector<pii>> &adj, int edgeCount,
    int source = 0) {
    int n = sz(adj);
    vector<int> balance(n), nextEdge(n), usedEdge(edgeCount),
        walk, stack = {source};
    balance[source]++; // to allow Euler paths, not just cycles
    while (!stack.empty()) {
        int u = stack.back(), v, edgeId, &it = nextEdge[u], degree
            = sz(adj[u]);
        if (it == degree) {
            walk.push_back(u);
            stack.pop_back();
            continue;
        }
        tie(v, edgeId) = adj[u][it++];
        if (!usedEdge[edgeId]) {
            balance[u]--, balance[v]++;
            usedEdge[edgeId] = 1;
            stack.push_back(v);
        }
    }
    for (int degreeDelta : balance)
        if (degreeDelta < 0 || sz(walk) != edgeCount + 1) return {}
    ;
    return {walk.rbegin(), walk.rend()};
}
```

7.5 Coloring

Edge Coloring

Description: Given a simple, undirected graph with max degree D , computes a $(D + 1)$ -coloring of the edges such that no neighboring edges share a color. (D -coloring is NP-hard, but can be done for bipartite graphs by repeated matchings of max-degree nodes.) Vertices are 0..n-1; input must have no self-loops or parallel edges. colors[i] is the 0-based color of input edge i. The algorithm recolors alternating chains to free a color at both endpoints. It guarantees at most $D+1$ colors, not the minimum number.

Usage: vector<pii> edges={{0,1},{1,2},{2,0}}; auto colors=edgeColoring(3,edges);

```
Time:  $\mathcal{O}(NM)$ 
vector<int> edgeColoring(int n, vector<pii> edges) {
    // fanColor holds degree counts first, then the colors used
    // by the fan.
    vector<int> fanColor(n + 1), colors(sz(edges)), fan(n),
        freeColor(n), colorPos;
    for (pii edge : edges) ++fanColor[edge.first], ++fanColor[
        edge.second];
    int u, v, colorCount = *max_element(all(fanColor)) + 1;
    vector<vector<int>> adj(n, vector<int>(colorCount, -1));
    for (pii edge : edges) {
        tie(u, v) = edge;
        fan[0] = v;
        colorPos.assign(colorCount, 0);
        int current = u, lastVertex = u, alternateColor, baseColor
            = freeColor[u], fanSize = 0, i = 0;
```

```
        while (alternateColor = freeColor[v],
            !colorPos[alternateColor] && (v = adj[u][
                alternateColor]) != -1)
            colorPos[alternateColor] = ++fanSize, fanColor[fanSize] =
                alternateColor, fan[fanSize] = v;
        fanColor[colorPos[alternateColor]] = baseColor;
        for (int color = alternateColor; current != -1;
            color ^= baseColor ^ alternateColor, current = adj[
                current][color])
            swap(adj[current][color], adj[lastVertex = current][color
                ^ baseColor ^ alternateColor]);
        while (adj[fan[i]][alternateColor] != -1) {
            int left = fan[i], right = fan[++i], rotatedColor =
                fanColor[i];
            adj[u][rotatedColor] = left;
            adj[left][rotatedColor] = u;
            adj[right][rotatedColor] = -1;
            freeColor[right] = rotatedColor;
        }
        adj[u][alternateColor] = fan[i];
        adj[fan[i]][alternateColor] = u;
        for (int y : {fan[0], u, lastVertex})
            for (int &z = freeColor[y] = 0; adj[y][z] != -1; z++);
    }
    for (int i = 0; i < (sz(edges)); ++i)
        for (tie(u, v) = edges[i]; adj[u][colors[i]] != v;) ++
            colors[i];
    return colors;
}
```

7.6 Heuristics

Maximal Cliques

Description: Runs a callback for all maximal cliques in a graph (given as a symmetric bitset matrix; self-edges not allowed). Callback is given a bitset representing the maximal clique. Maximal means no vertex can be added; it does not mean largest. For a maximum clique use MaximumClique, or examine every callback and keep the largest. Vertices are 0-based and there are at most 128. Pass candidates containing exactly the valid vertex bits; the default mask includes bits beyond n. For more vertices, enlarge B and keep candidate bits within adj.

Usage: vector adj(n); B candidates; for (int u=0;u<n;u++) candidates.set(u); // Set adj[u][v]=adj[v][u]=1 for each edge. cliques(adj,[](B clique) { ... },candidates);

Time: $\mathcal{O}\left(3^{n/3}\right)$, much faster for sparse graphs

```
typedef bitset<128> B;
template <class F>
void cliques(vector<B> &adj, F visitClique, B candidates = ~B()
    , B excluded = {}, B clique = {}) {
    if (!candidates.any()) {
        if (!excluded.any()) visitClique(clique);
        return;
    }
    auto pivot = (candidates | excluded).Find_first();
    auto branches = candidates & ~adj[pivot];
    for (int i = 0; i < (sz(adj)); ++i)
        if (branches[i]) {
            clique[i] = 1;
            cliques(adj, visitClique, candidates & adj[i], excluded &
                adj[i], clique);
            clique[i] = candidates[i] = 0;
            excluded[i] = 1;
        }
    }
```

Maximum Clique

Description: Quickly finds a maximum clique of a graph (given as symmetric bitset matrix; self-edges not allowed). Can be used to find a maximum independent set by finding a maximum clique of the complement graph. Exact branch-and-bound uses color bounds to prune search while retaining the largest clique. Runtime can still be exponential. Supply a nonempty symmetric adjacency matrix with at most 200 vertices and a zero diagonal. maxClique returns 0-based vertex IDs; construct a fresh object for a new search. Handle n=0 before constructing; enlarge vb's bitset if n>200. **Usage:** vb adj(n); // set adj[u][v]=adj[v][u]=1 for each edge MaximumClique solver(adj); auto vertices=solver.maxClique(); **Time:** Runs in about 1s for n=155 and worst case random graphs (p=.90). Runs faster for sparse graphs.

```
typedef vector<bitset<200>> vb;
struct MaximumClique {
    double limit = 0.025, searchCount = 0;
    struct Vertex {
        int id, bound = 0; // degree during init, then a coloring
                             upper bound
    };
    typedef vector<Vertex> vv;
    vb adj;
    vv vertices;
    vector<vector<int>>> colorClasses;
    vector<int> bestClique, clique, searchStats, prevStats;
    void init(vv &candidates) {
        for (auto &v : candidates) v.bound = 0;
        for (auto &v : candidates)
            for (auto other : candidates) v.bound += adj[v.id][other.id];
        sort(all(candidates), [](auto a, auto b) { return a.bound > b.bound; });
        int maxDegree = candidates[0].bound;
        for (int i = 0; i < (sz(candidates)); ++i) candidates[i].bound = min(i, maxDegree) + 1;
    }
    void expand(vv &candidates, int level = 1) {
        searchStats[level] += searchStats[level - 1] - prevStats[level];
        prevStats[level] = searchStats[level - 1];
        while (sz(candidates)) {
            if (sz(clique) + candidates.back().bound <= sz(bestClique)) return;
            clique.push_back(candidates.back().id);
            vv neighbors;
            for (auto v : candidates)
                if (adj[candidates.back().id][v.id]) neighbors.push_back({v.id});
            if (sz(neighbors)) {
                if (searchStats[level]++ / ++searchCount < limit) init(neighbors);
                int writePos = 0, maxColor = 1, minColor = max(sz(bestClique) - sz(clique) + 1, 1);
                colorClasses[1].clear(), colorClasses[2].clear();
                for (auto v : neighbors) {
                    int color = 1;
                    auto isAdjacent = [&](int i) { return adj[v.id][i]; };
                    while (any_of(all(colorClasses[color]), isAdjacent)) color++;
                    if (color > maxColor) maxColor = color, colorClasses[maxColor + 1].clear();
                    if (color < minColor) neighbors[writePos++].id = v.id;
                    colorClasses[color].push_back(v.id);
                }
                if (writePos > 0) neighbors[writePos - 1].bound = 0;
                for (int color = minColor; color < (maxColor + 1); ++color)
```

```
                for (int i : colorClasses[color])
                    neighbors[writePos].id = i, neighbors[writePos++].bound = color;
                expand(neighbors, level + 1);
            } else if (sz(clique) > sz(bestClique))
                bestClique = clique;
            clique.pop_back(), candidates.pop_back();
        }
    }
    vector<int> maxClique() {
        init(vertices), expand(vertices);
        return bestClique;
    }
    MaximumClique(vb adjacency)
        : adj(adjacency), colorClasses(sz(adj) + 1), searchStats(sz(colorClasses)),
          prevStats(searchStats) {
        for (int i = 0; i < (sz(adj)); ++i) vertices.push_back({i});
    }
};
```

Maximum Independent Set

Description: A maximum independent set in G is a **maximum clique** (largest possible size) in the complement of G . Use MaximumClique. A **maximal clique** only means no vertex can be added; it may be smaller than the maximum, so an arbitrary maximal clique is insufficient. For bipartite graphs, take all vertices outside a minimum vertex cover; see MinimumVertexCover. Complement only pairs of distinct vertices; keep the diagonal zero. The returned clique vertices are the independent-set vertices in the original graph. Maximal independent sets, like maximal cliques, may be much smaller than maximum ones. **Usage:** vb complement(n); for (int u=0;u<n;u++) for (int v=0;v<n;v++) complement[u][v]=u!=v && !adj[u][v]; MaximumClique solver(complement); auto independent=solver.maxClique();

7.7 Trees

Binary Lifting + LCA

Description: Binary lifting and LCA in one snippet: up[u][k] is the ancestor 2^k steps above u. Build on a connected tree; root defaults to 1 and is its own parent. goUp needs $0 \leq \text{steps} \leq h[u]$. tin is single-entry DFS order. distance counts edges; weightedDistance sums weights using ll root distances. Rebuild after edge changes; DFS is recursive. Use for kth ancestors; jump aggregates need an extra table/merge per level. Euler-tour + RMQ gives $O(1)$ LCA queries. For a forest, build/query each component separately. **Usage:** LCA tree(adj); // vector<vector<int>>, unit edge weights int common=tree.lca(u,v); int ancestor=tree.goUp(u,2); // requires tree.h[u]>=2 int edges=tree.distance(u,v); vector<vector<pll>>> weightedAdj(n+1); // Each pair is {neighbor,weight}; add both directions for an undirected tree. LCA weighted(weightedAdj); ll length=weighted.weightedDistance(u,v); **Time:** $O(N \log N)$ construction and memory; $O(\log N)$ per query.

```
struct LCA {
    int LOG, timer = 0;
    vector<vector<pll>>> adj;
    vector<vector<int>>> up;
    vector<int> h, tin;
    vector<ll> rootDist;
    LCA(int n)
        : LOG(__lg(max(1, n)) + 1), adj(n), up(n, vector<int>(LOG)), h(n), tin(n), rootDist(n) {}
};
```

```
LCA(const vector<vector<int>>> &graph, int root = 1) : LCA(sz(graph)) {
    for (int u = 0; u < sz(graph); u++)
        for (int v : graph[u]) adj[u].push_back({v, 1});
    dfs(root, root);
}
LCA(const vector<vector<pll>>> &graph, int root = 1) : LCA(sz(graph)) {
    adj = graph;
    dfs(root, root);
}
void dfs(int u, int parent) {
    tin[u] = timer++;
    up[u][0] = parent;
    for (int k = 1; k < LOG; k++) up[u][k] = up[up[u][k - 1]][k - 1];
    for (auto [v, weight] : adj[u]) {
        if (v == parent) continue;
        h[v] = h[u] + 1;
        rootDist[v] = rootDist[u] + weight;
        dfs(v, u);
    }
}
int goUp(int u, int steps) const {
    for (int k = 0; k < LOG; k++)
        if (steps >> k & 1) u = up[u][k];
    return u;
}
int lca(int u, int v) const {
    if (h[u] < h[v]) swap(u, v);
    u = goUp(u, h[u] - h[v]);
    if (u == v) return u;
    for (int k = LOG - 1; k >= 0; k--)
        if (up[u][k] != up[v][k]) u = up[u][k], v = up[v][k];
    return up[u][0];
}
int distance(int u, int v) const { return h[u] + h[v] - 2 * h[lca(u, v)]; }
ll weightedDistance(int u, int v) const {
    return rootDist[u] + rootDist[v] - 2 * rootDist[lca(u, v)];
}
};
```

LCA: Euler Tour + RMQ

Description: Euler-tour + RMQ version of LCA. Record u on entry and again after each child; the LCA is the shallowest vertex between the first occurrences of u and v. This is different from the entry/exit tour used by tree Mo. Build on a connected tree; default root=1. Choose this for many LCA queries. Binary lifting also supports kth ancestors and jump aggregates, with less tour-table overhead. Rebuild after changing edges; DFS is recursive. **Usage:** LCAEuler tree(adj); // vector<vector<int>>, undirected or parent-to-child int common=tree.lca(u,v); int edges=tree.distance(u,v); // With separately computed weighted root distances: // length = rootDist[u] + rootDist[v] // - 2 * rootDist[common]; **Time:** $O(N \log N)$ construction and memory; $O(1)$ per LCA/distance query.

```
struct LCAEuler {
    vector<int> h, first, tour;
    vector<vector<int>>> table;
    LCAEuler(const vector<vector<int>>> &adj, int root = 1) : h(sz(adj)), first(sz(adj), -1) {
        dfs(adj, root, root);
        table.push_back(tour);
        int m = sz(tour);
        for (int k = 1; (1LL << k) <= m; k++) {
            table.emplace_back(m - (1 << k) + 1);
        }
    }
};
```

```
        for (int i = 0; i < sz(table[k]); i++)
            table[k][i] = shallower(table[k - 1][i], table[k - 1][i]
                + (1 << (k - 1)));
    }
}
void dfs(const vector<vector<int>> &adj, int u, int parent) {
    first[u] = sz(tour);
    tour.push_back(u);
    for (int v : adj[u]) {
        if (v == parent) continue;
        h[v] = h[u] + 1;
        dfs(adj, v, u);
        tour.push_back(u);
    }
}
int shallower(int u, int v) const { return h[u] <= h[v] ? u : v; }
int lca(int u, int v) const {
    int l = first[u], r = first[v];
    if (l > r) swap(l, r);
    int k = __lg(r - l + 1);
    return shallower(table[k][l], table[k][r - (1 << k) + 1]);
}
int distance(int u, int v) const { return h[u] + h[v] - 2 * h[lca(u, v)]; }
};
```

Virtual Tree Construction

Description: Given a rooted tree and a subset S of nodes, compute the minimal subtree that contains all the nodes by adding all (at most $|S| - 1$) pairwise LCA's and compressing edges. Returns a list of (par, orig_index) representing a tree rooted at virtual index 0. The root points to itself. Use when a query touches only a small subset of a large tree. Build LCA once and compress each subset separately. At virtual index i, result[i].second is the original vertex and result[i].first is its virtual parent index. A compressed edge represents an original path; compute its length from depths or weighted root distances.

Usage: LCA tree(adj);
auto compressed=compressTree(tree,{u,v,w});
for (int i=1;i<sz(compressed);i++) {
int parentIndex=compressed[i].first;
int originalVertex=compressed[i].second;
}
Time: $\mathcal{O}(|S|(\log |S| + \log N))$

```
"LCA.h"
typedef vector<pair<int, int>> vpi;
vpi compressTree(LCA &lca, const vector<int> &subset) {
    if (subset.empty()) return {};
    vector<int> virtualIndex;
    virtualIndex.resize(sz(lca.tin));
    vector<int> vertices = subset, &entryTime = lca.tin;
    auto cmp = [&](int a, int b) { return entryTime[a] < entryTime[b]; };
    sort(all(vertices), cmp);
    int m = sz(vertices) - 1;
    for (int i = 0; i < (m); ++i) {
        int a = vertices[i], b = vertices[i + 1];
        vertices.push_back(lca.lca(a, b));
    }
    sort(all(vertices), cmp);
    vertices.erase(unique(all(vertices)), vertices.end());
    for (int i = 0; i < (sz(vertices)); ++i) virtualIndex[vertices[i]] = i;
    vpi virtualTree = {pii(0, vertices[0])};
    for (int i = 0; i < (sz(vertices) - 1); ++i) {
        int a = vertices[i], b = vertices[i + 1];
        virtualTree.emplace_back(virtualIndex[lca.lca(a, b)], b);
    }
    return virtualTree;
}
```

```
}
Heavy-Light Decomposition (HLD)
Description: Decomposes a tree into vertex disjoint heavy paths and light edges such that the path from any leaf to the root contains at most  $\log(n)$  light edges. Code does additive modifications and max queries, but can support commutative segtree modifications/queries on paths and subtrees. Takes as input the full adjacency list. VALS.EDGES being true means that values are stored in the edges, as opposed to the nodes. All values initialized to the segtree default. Root must be 0. Use a nonempty connected undirected tree. The constructor reorders its own adjacency copy; vertices are 0..n-1 rooted at 0 (relabel 1-based input). modifyPath adds a delta; queryPath/querySubtree return maxima. With edge values each non-root vertex stores its parent edge. Initialize finite values before addition. Edge path (u,u) returns INT_MIN. Ordered/noncommutative queries need forward/reverse aggregates and path orientation.
Usage: HLD<false> hld(adj); hld.tree.set(1,sz(adj),0);
hld.modifyPath(u,v,5); int best=hld.queryPath(u,v);
int subtreeBest=hld.querySubtree(u);
Time:  $\mathcal{O}((\log N)^2)$ 
"../data-structures/LazySegmentTree.h"
template <bool VALS_EDGES> struct HLD {
    int n, timer = 0;
    vector<vector<int>> adj;
    vector<int> parent, subtreeSize, head, pos;
    LazySegTree tree;
    HLD(vector<vector<int>> graph)
        : n(sz(graph)), adj(graph), parent(n, -1), subtreeSize(n, 1), head(n), pos(n), tree(n) {
        dfsSz(0);
        dfsHld(0);
    }
    void dfsSz(int v) {
        for (int &u : adj[v]) {
            adj[u].erase(find(all(adj[u]), v));
            parent[u] = v;
            dfsSz(u);
            subtreeSize[v] += subtreeSize[u];
            if (subtreeSize[u] > subtreeSize[adj[v][0]]) swap(u, adj[v][0]);
        }
    }
    void dfsHld(int v) {
        pos[v] = timer++;
        for (int u : adj[v]) {
            head[u] = (u == adj[v][0] ? head[v] : u);
            dfsHld(u);
        }
    }
    template <class B> void process(int u, int v, B visitRange) {
        for (; v = parent[head[v]]) {
            if (pos[u] > pos[v]) swap(u, v);
            if (head[u] == head[v]) break;
            visitRange(pos[head[v]], pos[v] + 1);
        }
        visitRange(pos[u] + VALS_EDGES, pos[v] + 1);
    }
    void modifyPath(int u, int v, int val) {
        process(u, v, [&](int l, int r) { tree.add(l + 1, r, val); });
    }
    int queryPath(int u, int v) { // Modify depending on problem
        int answer = INT_MIN;
        process(u, v, [&](int l, int r) { answer = max(answer, tree.query(l + 1, r)); });
        return answer;
    }
    int querySubtree(int v) { // modifySubtree is similar

```

```
        return tree.query(pos[v] + VALS_EDGES + 1, pos[v] + subtreeSize[v]);
    }
};
Centroid Decomposition
Description: Path-counting skeleton; example counts unordered distinct pairs at exactly k edges. Customize the CHANGE blocks: state/storage, initial/extended state, query/insert/reset. Query a whole child BEFORE inserting it; only the centroid and earlier children are stored. Reset touched entries after each centroid. Recurse to count paths inside one child. For vertex sums, match k+value[centroid]-state; for edge sums adapt adjacency/state extension. Length ranges need a Fenwick range query; XOR needs mask states. Prune only if deeper states cannot become valid again. Root defaults to 1 (pass 0 if needed); use a fresh object per run. Single-vertex paths are excluded. Recursive DFS may have linear stack depth.
Usage: CentroidDecomposition tree(adj,k); // adj: vector<vector<int>>, both directions
ll answer=tree.solve(); // 1-based vertices; exact-k example
// For vertices 0..n-1: tree.solve(0).
// Change State, initialState/extendState, queryState/addState/clearStates for another problem.
// Example: a chain of 4 vertices with k=2 has answer 2.
Time:  $\mathcal{O}(N \log N)$  with constant-time state operations;  $\mathcal{O}(N)$  auxiliary memory.
struct CentroidDecomposition {
    // CHANGE 1: state, parameters, storage.
    using State = int; // Depth; change to ll, mask, or struct.
    int k;
    vector<vector<int>> adj; // Adapt for weighted edges.
    vector<int> subtreeSize, freq, touched;
    vector<bool> removed;
    CentroidDecomposition(const vector<vector<int>> &graph, int length)
        : k(length), adj(graph), subtreeSize(sz(graph)), freq(sz(graph)), removed(sz(graph)) {}
    // CHANGE 2: initial state and extension.
    State initialState(int centroid) { return 0; }
    State extendState(State state, int v) { return state + 1; }
    // CHANGE 3: match, insert, reset.
    ll queryState(State state, int centroid) {
        ll need = (ll)k - state; // Vertex sums: k + value[centroid] - state.
        return 0 <= need && need < sz(freq) ? freq[need] : 0;
    }
    void addState(State state) {
        if (!freq[state]) touched.push_back(state);
        freq[state]++;
    }
    void clearStates() {
        for (int state : touched) freq[state] = 0;
        touched.clear(); // Map: clear; Fenwick: undo updates.
    }
    int countChild(int u, int parent) {
        subtreeSize[u] = 1;
        for (int v : adj[u])
            if (v != parent && !removed[v]) subtreeSize[u] += countChild(v, u);
        return subtreeSize[u];
    }
    int findCentroid(int u, int parent, int total) {
        for (int v : adj[u])
            if (v != parent && !removed[v] && subtreeSize[v] > total / 2)
                return findCentroid(v, u, total);
        return u;
    }
    void collectStates(int u, int parent, State state, vector<State> &states) {

```

```
states.push_back(state);
for (int v : adj[u])
    if (v != parent && !removed[v]) collectStates(v, u,
        extendState(state, v), states);
}
ll countPaths(int centroid) {
    ll answer = 0;
    State start = initialState(centroid);
    addState(start); // Include the centroid.
    for (int v : adj[centroid]) {
        if (removed[v]) continue;
        vector<State> states;
        collectStates(v, centroid, extendState(start, v), states)
        ;
        for (State state : states) answer += queryState(state,
            centroid);
        for (State state : states) addState(state); // Keep these
            two loops separate!
    }
    clearStates(); // Reset before recursion.
    return answer;
}
ll solve(int u = 1) {
    int total = countChild(u, -1);
    int centroid = findCentroid(u, -1, total);
    ll answer = countPaths(centroid);
    removed[centroid] = true;
    for (int v : adj[centroid])
        if (!removed[v]) answer += solve(v);
    return answer;
}
};
```

7.8 Math

7.8.1 Number of Spanning Trees

Create an $N \times N$ matrix mat , and for each edge $a \rightarrow b \in G$, do $\text{mat}[a][b]--$, $\text{mat}[b][b]++$ (and $\text{mat}[b][a]--$, $\text{mat}[a][a]++$ if G is undirected). Remove the i th row and column and take the determinant; this yields the number of directed spanning trees rooted at i (if G is undirected, remove any row/column).

7.8.2 Erdős–Gallai theorem

A simple graph with node degrees $d_1 \geq \dots \geq d_n$ exists iff $d_1 + \dots + d_n$ is even and for every $k = 1 \dots n$,

$$\sum_{i=1}^k d_i \leq k(k-1) + \sum_{i=k+1}^n \min(d_i, k).$$

Geometry (8)

8.1 Geometric primitives

2D Point and Vector

Description: Use `Point<ll>` for exact integer cross/dot products and `Point<double>` for fractional coordinates. `p.cross(a,b)` means $(a-p)$ cross $(b-p)$. `perp` rotates 90 degrees CCW; `rotate` takes radians. `unit`/`normal` require a nonzero vector and floating `T`; integer division/rotation truncate. Large integer predicates need wider cross/dot intermediates, e.g. `_int128`.
Usage: `Point<ll> a(0,0), b(2,0), c(0,3);`
`ll cross=a.cross(b,c); // 6, c is left of a->b`

template <class T> int `sgn(T x)` { **return** `(x > 0) - (x < 0);` };

```
template <class T> struct Point {
    typedef Point P;
    T x, y;
    explicit Point(T x = 0, T y = 0) : x(x), y(y) {}
    bool operator<(P other) const { return tie(x, y) < tie(other.x, other.y); }
    bool operator==(P other) const { return tie(x, y) == tie(other.x, other.y); }
    P operator+(P other) const { return P(x + other.x, y + other.y); }
    P operator-(P other) const { return P(x - other.x, y - other.y); }
    P operator*(T scale) const { return P(x * scale, y * scale); }
    P operator/(T scale) const { return P(x / scale, y / scale); }
    T dot(P other) const { return x * other.x + y * other.y; }
    T cross(P other) const { return x * other.y - y * other.x; }
    T cross(P a, P b) const { return (a - *this).cross(b - *this); }
    T dist2() const { return x * x + y * y; }
    double dist() const { return sqrt((double)dist2()); }
    // angle to x-axis in interval [-pi, pi]
    double angle() const { return atan2(y, x); }
    P unit() const { return *this / dist(); } // makes dist()==1
    P perp() const { return P(-y, x); } // rotates +90 degrees
    P normal() const { return perp().unit(); }
    // returns point rotated angle radians ccw around the origin
    P rotate(double angle) const {
        return P(x * cos(angle) - y * sin(angle), x * sin(angle) + y * cos(angle));
    }
    friend ostream &operator<<(ostream &os, P other) {
        return os << "(" << other.x << ", " << other.y << ")";
    }
};
```

Point-to-Line Distance

Description:

Signed distance to the infinite line $a \rightarrow b$: positive on the left, negative on the right. Require $a \neq b$; take abs for unsigned distance. Cross products must fit the coordinate type.

Usage: `double distance=abs(lineDist(Point<double>(0,0), Point<double>(2,0),Point<double>(1,3))); // 3`
`"Point.h"`

```
template <class P>
double lineDist(P a, P b, P p) {
    return (double) (b - a).cross(p - a) / (b - a).dist();
}
```

Point-to-Segment Distance

Description:

Distance to the closed segment, including endpoints; equal endpoints are supported. Use `Point<double>` and a tolerance when comparing the distance with zero.
Usage: `double distance=segDist(P(0,0),P(2,0),P(3,1));`
`bool on=distance<1e-9;`
`"Point.h"`

typedef `Point<double>` `P;`
double `segDist(P start, P finish, P p)` {
 if (`start == finish`) **return** `(p - start).dist();`
 auto `length2 = (finish - start).dist2();`
 `projection = min(length2, max(.0, (p - start).dot(finish - start)));`
 return `((p - start) * length2 - (finish - start) * projection).dist() / length2;`

```
}
Segment Intersection
Description:
If a unique intersection point between the line segments going from  $s_1$  to  $e_1$  and from  $s_2$  to  $e_2$  exists then it is returned. If no intersection point exists an empty vector is returned. If infinitely many exist a vector with 2 elements is returned, containing the endpoints of the common line segment. The wrong position will be returned if P is Point<ll> and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or long long. Endpoints are included. The returned vector has size 0, 1 or 2 for disjoint segments, one intersection, or an overlapping segment. Use Point<double> for fractional coordinates; exact boundary checks may need a tolerance for noisy real input.
Usage: auto intersection=segInter(Point<double>(0,0), Point<double>(2,2),Point<double>(0,2),Point<double>(2,0));  
// intersection[0] is (1,1).  
"Point.h", "OnSegment.h"
template <class P> vector<P> segInter(P a, P b, P c, P d) {  
    auto sideA = c.cross(d, a), sideB = c.cross(d, b), sideC = a.cross(b, c), sideD = a.cross(b, d);  
    // Checks if intersection is single non-endpoint point.  
    if (sgn(sideA) * sgn(sideB) < 0 && sgn(sideC) * sgn(sideD) < 0)  
        return {(a * sideB - b * sideA) / (sideB - sideA)};  
    set<P> intersections;  
    if (onSegment(c, d, a)) intersections.insert(a);  
    if (onSegment(c, d, b)) intersections.insert(b);  
    if (onSegment(a, b, c)) intersections.insert(c);  
    if (onSegment(a, b, d)) intersections.insert(d);  
    return {all(intersections)};  
}
```

Line Intersection

Description:

If a unique intersection point of the lines going through s_1, e_1 and s_2, e_2 exists `{1, point}` is returned. If no intersection point exists `{0, (0,0)}` is returned and if infinitely many exists `{-1, (0,0)}` is returned. The wrong position will be returned if `P` is `Point<ll>` and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using `int` or `ll`. These are infinite lines, not bounded segments; each defining pair must contain distinct points. Inspect the status before reading the point. Use `Point<double>` for fractional coordinates; integer `Point` division truncates. For noisy real input, use tolerance for parallel/coincident tests, scaled to direction lengths.
Usage: `auto [status,p]=lineInter(Point<double>(0,0), Point<double>(2,2),Point<double>(0,2),Point<double>(2,0));`
`// status: 1 unique, 0 parallel, -1 coincident.`
`"Point.h"`

```
template <class P> pair<int, P> lineInter(P start1, P end1, P start2, P end2) {  
    auto crossDir = (end1 - start1).cross(end2 - start2);  
    if (crossDir == 0) // if parallel  
        return {-(start1.cross(end1, start2) == 0), P(0, 0)};  
    auto weightStart = start2.cross(end1, end2), weightEnd = start2.cross(end2, start1);  
    return {1, (start1 * weightStart + end1 * weightEnd) / crossDir};  
}
```

Orientation Test (Cross Product)

Description: Returns +1/0/-1 for p left/on/right of start->finish. The eps overload treats distance<=eps as on the line and requires distinct endpoints. Products must fit the coordinate type.

Usage: int side=sideOf(Point<ll>(0,0),Point<ll>(2,0),Point<ll>(1,3)); // +1: left
"Point.h"

```
template <class P> int sideOf(P start, P finish, P p) { return
    sgn(start.cross(finish, p)); }
template <class P> int sideOf(P start, P finish, P p, double
    eps) {
    auto crossValue = (finish - start).cross(p - start);
    double tolerance = (finish - start).dist() * eps;
    return (crossValue > tolerance) - (crossValue < -tolerance);
}
```

Point on Segment

Description: Exact membership on a closed segment, including endpoints and zero-length segments. Prefer integer coordinates; for Point<double> compare segDist with a tolerance.

Usage: bool on=onSegment(Point<ll>(0,0),Point<ll>(2,0),Point<ll>(1,0)); // true
"Point.h"

```
template <class P> bool onSegment(P start, P finish, P p) {
    return p.cross(start, finish) == 0 && (start - p).dot(finish - p) <= 0;
}
```

Linear Transformation

Description:

Apply the linear transformation (translation, rotation and scaling) which takes line p0-p1 to line q0-q1 to point r.

Use when a directed segment is mapped to another by translation, rotation and uniform scaling. The same mapping is applied to point. The source segment must have nonzero length. This does not implement reflection or arbitrary shear.

Usage: P mapped=linearTransformation(P(0,0),P(1,0),P(2,3),P(2,5),P(1,1)); // (0,5)
"Point.h"

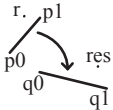
```
typedef Point<double> P;
P linearTransformation(P sourceStart, P sourceEnd, P
    targetStart, P targetEnd, P point) {
    P sourceDir = sourceEnd - sourceStart, targetDir = targetEnd
        - targetStart,
    transform(sourceDir.cross(targetDir), sourceDir.dot(
        targetDir));
    return targetStart +
        P((point - sourceStart).cross(transform), (point -
            sourceStart).dot(transform)) /
            sourceDir.dist2();
}
```

Angle Ordering

Description: A class for ordering angles (as represented by int points and a number of rotations around the origin). Useful for rotational sweeping. Sometimes also represents points or vectors. Use nonzero integer direction vectors to sort events without atan2. turns distinguishes directions after complete rotations. t90/t180/t360 rotate counterclockwise; segmentAngles returns the shorter angular interval covering two directions. Intermediate integer products must fit their types. angleDiff needs wider coordinate fields/products for large coordinates. Equal directions need a distance/event-type tie-break if sweep order matters.

Usage: vector<Angle> directions={{1,0},{0,1},{-1,0}}; sort(all(directions));
auto quarterTurn=Angle(1,0).t90();

```
struct Angle {
    int x, y;
    int turns;
```



```
Angle(int x, int y, int turns = 0) : x(x), y(y), turns(turns)
{ }
Angle operator-(Angle b) const { return {x - b.x, y - b.y,
    turns}; }
int half() const {
    assert(x || y);
    return y < 0 || (y == 0 && x < 0);
}
Angle t90() const { return {-y, x, turns + (half() && x >= 0)
    }; }
Angle t180() const { return {-x, -y, turns + half()}; }
Angle t360() const { return {x, y, turns + 1}; }
};
bool operator<(Angle a, Angle b) {
    // add a.dist2() and b.dist2() to also compare distances
    return make_tuple(a.turns, a.half(), a.y * (ll)b.x) <
        make_tuple(b.turns, b.half(), a.x * (ll)b.y);
}
// Given two points, this calculates the smallest angle between
// them, i.e., the angle that covers the defined line segment.
pair<Angle, Angle> segmentAngles(Angle a, Angle b) {
    if (b < a) swap(a, b);
    return (b < a.t180() ? make_pair(a, b) : make_pair(b, a.t360
        ());
}
Angle operator+(Angle a, Angle b) { // point a + vector b
    Angle result(a.x + b.x, a.y + b.y, a.turns);
    if (a.t180() < result) result.turns--;
    return result.t180() < a ? result.t360() : result;
}
Angle angleDiff(Angle a, Angle b) { // angle b - angle a
    int turnDiff = b.turns - a.turns;
    a.turns = b.turns;
    return {a.x * b.x + a.y * b.y, a.x * b.y - a.y * b.x,
        turnDiff - (b < a)};
}
```

8.2 Circles

Circle Intersection

Description: Computes the pair of points at which two circles intersect. Returns false in case of no intersection. On success the pair contains both intersections; at tangency they are equal. Coincident circles have infinitely many intersections and trigger an assertion, so handle that case before calling. Radii must be nonnegative; near tangency needs tolerant separation tests.

Usage: pair<P,P> points;
bool intersects=circleInter(P(0,0),P(2,0),2,2, &points);
"Point.h"

```
typedef Point<double> P;
bool circleInter(P center1, P center2, double r1, double r2,
    pair<P, P> *intersections) {
    if (center1 == center2) {
        assert(r1 != r2);
        return false;
    }
    P centerDir = center2 - center1;
    double distance2 = centerDir.dist2(), radiusSum = r1 + r2,
        radiusDiff = r1 - r2,
        projection = (distance2 + r1 * r1 - r2 * r2) / (
            distance2 * 2),
        height2 = r1 * r1 - projection * projection *
            distance2;
    if (radiusSum * radiusSum < distance2 || radiusDiff *
        radiusDiff > distance2) return false;
    P basePoint = center1 + centerDir * projection,
        offset = centerDir.perp() * sqrt(fmax(0, height2) /
            distance2);
    *intersections = {basePoint + offset, basePoint - offset};
}
```

```
    return true;
}
```

Circle Tangents

Description: Finds the external tangents of two circles, or internal if r2 is negated. Can return 0, 1, or 2 tangents - 0 if one circle contains the other (or overlaps it, in the internal case, or if the circles are the same); 1 if the circles are tangent to each other (in which case .first = .second and the tangent line is perpendicular to the line between the centers). .first and .second give the tangency points at circle 1 and 2 respectively. To find the tangents of a circle with a point set r2 to 0. Each returned pair is a tangent segment joining the two tangency points. Use Point<double>; negating the second radius requests internal tangents rather than a physically negative radius. Coincident centers return no tangents. If the tangent points coincide, use a line perpendicular to the center direction; the returned pair alone cannot define that line.

Usage: auto external=tangents(Point<double>(0,0),1,Point<double>(4,0),1); // two tangent pairs
"Point.h"

```
template <class P> vector<pair<P, P>> tangents(P c1, double r1,
    P c2, double r2) {
    P centerDir = c2 - c1;
    double radiusDiff = r1 - r2, distance2 = centerDir.dist2(),
        height2 = distance2 - radiusDiff * radiusDiff;
    if (distance2 == 0 || height2 < 0) return {};
    vector<pair<P, P>> tangentPairs;
    for (double sign : {-1, 1}) {
        P normal = (centerDir * radiusDiff + centerDir.perp() *
            sqrt(height2) * sign) / distance2;
        tangentPairs.push_back({c1 + normal * r1, c2 + normal * r2
            });
    }
    if (height2 == 0) tangentPairs.pop_back();
    return tangentPairs;
}
```

Circle-Polygon Intersection

Description: Returns the area of the intersection of a circle with a ccw polygon. The polygon may be concave, but must be simple with nonzero-length edges. It is copied and left unchanged. The area is signed: counterclockwise input gives positive area; take abs if needed. Radius must be positive; return 0 for radius 0 and remove consecutive duplicate vertices.

Usage: double area=abs(circlePoly(P(0,0),1,polygon));
Time: $\mathcal{O}(n)$

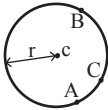
```
"../content/geometry/Point.h"
typedef Point<double> P;
#define arg(p, q) atan2(p.cross(q), p.dot(q))
double circlePoly(P center, double radius, vector<P> polygon) {
    auto triangleArea = [&](P p, P q) {
        auto halfRadius2 = radius * radius / 2;
        P direction = q - p;
        auto linearTerm = direction.dot(p) / direction.dist2(),
            constantTerm = (p.dist2() - radius * radius) /
                direction.dist2();
        auto discriminant = linearTerm * linearTerm - constantTerm;
        if (discriminant <= 0) return arg(p, q) * halfRadius2;
        auto enter = max(0., -linearTerm - sqrt(discriminant)),
            exit = min(1., -linearTerm + sqrt(discriminant));
        if (exit < 0 || 1 <= enter) return arg(p, q) * halfRadius2;
        P intersection1 = p + direction * enter, intersection2 = q
            + direction * (exit - 1);
        return arg(p, intersection1) * halfRadius2 + intersection1.
            cross(intersection2) / 2 +
            arg(intersection2, q) * halfRadius2;
    };
    auto area = 0.0;
    for (int i = 0; i < (sz(polygon)); ++i)
        area += triangleArea(polygon[i] - center, polygon[(i + 1) %
            sz(polygon)] - center);
}
```

```
    return area;
}
```

Circumcircle

Description:

The circumcirle of a triangle is the circle intersecting all three vertices. ccRadius returns the radius of the circle going through points A, B and C and ccCenter returns the center of the same circle.



The points must be distinct and noncollinear. This is the circle through the vertices, which is not always the triangle's minimum enclosing circle: an obtuse triangle uses its longest side as a diameter. Use Point<double> for the center.

```
Usage: P a(0,0), b(2,0), c(0,2);
P center=ccCenter(a,b,c); double radius=ccRadius(a,b,c);
"Point.h"
```

```
typedef Point<double> P;
double ccRadius(P A, P B, P C) {
    return (B - A).dist() * (C - B).dist() * (A - C).dist() / abs
        ((B - A).cross(C - A)) / 2;
}
P ccCenter(P A, P B, P C) {
    P dirAC = C - A, dirAB = B - A;
    return A + (dirAC * dirAB.dist2() - dirAB * dirAC.dist2()).
        perp() / dirAC.cross(dirAB) / 2;
}
```

Minimum Enclosing Circle

Description: Computes the minimum circle that encloses a set of points. Input must contain at least one point. Returns (center,radius), not radius squared. Random shuffling makes incremental construction fast on average, and the input is copied. Uses floating tolerance to decide whether a point lies outside.

```
Usage: auto [center,radius]=mec(vector<P>{P(0,0),P(2,0)});
// center=(1,0), radius=1.
Time: expected  $\mathcal{O}(n)$ 
"circumcircle.h"
```

```
pair<P, double> mec(vector<P> points) {
    shuffle(all(points), mt19937(time(0)));
    P center = points[0];
    double radius = 0, tolerance = 1 + 1e-8;
    for (int i = 0; i < (sz(points)); ++i)
        if ((center - points[i]).dist() > radius * tolerance) {
            center = points[i], radius = 0;
            for (int j = 0; j < (i); ++j)
                if ((center - points[j]).dist() > radius * tolerance) {
                    center = (points[i] + points[j]) / 2;
                    radius = (center - points[i]).dist();
                    for (int k = 0; k < (j); ++k)
                        if ((center - points[k]).dist() > radius *
                            tolerance) {
                            center = ccCenter(points[i], points[j], points[k]
                                );
                            radius = (center - points[i]).dist();
                        }
                }
            }
    return {center, radius};
}
```

8.3 Polygons

Point in Polygon

Description: Returns true if p lies within the polygon. If strict is true, it returns false for points on the boundary. The algorithm uses products in intermediate steps so watch out for overflow. Works for a simple, possibly concave polygon in either orientation; do not repeat the first vertex at the end. strict=true excludes the boundary. Use integer coordinates for exact boundary decisions, or adapt the on-segment check to a tolerance.

```
Usage: vector<Point<ll>> polygon={{0,0},{4,0},{0,4}};
bool inside=inPolygon(polygon,Point<ll>(1,1));
Time:  $\mathcal{O}(n)$ 
```

```
"Point.h", "OnSegment.h", "SegmentDistance.h"
template <class P> bool inPolygon(vector<P> &polygon, P point,
    bool strict = true) {
    int inside = 0, n = sz(polygon);
    for (int i = 0; i < (n); ++i) {
        P nextPoint = polygon[(i + 1) % n];
        if (onSegment(polygon[i], nextPoint, point)) return !strict
            ;
        //or: if (segDist(p[i], q, a) <= eps) return !strict;
        inside ^=
            ((point.y < polygon[i].y) - (point.y < nextPoint.y)) *
                point.cross(polygon[i], nextPoint) >
                0;
    }
    return inside;
}
```

Polygon Area

Description: Returns twice the signed area: positive CCW, negative clock-wise. Use ordered boundary vertices, at least one point, and no repeated final vertex. Geometric area is abs(area2)/2.0; cross-product sums must fit T.

```
Usage: vector<Point<ll>> p={{0,0},{4,0},{0,3}};
double area=abs(polygonArea2(p))/2.0; // 6
"Point.h"
```

```
template <class T> T polygonArea2(vector<Point<T>> &polygon) {
    T area2 = polygon.back().cross(polygon[0]);
    for (int i = 0; i < (sz(polygon) - 1); ++i) area2 += polygon[
        i].cross(polygon[i + 1]);
    return area2;
}
```

Polygon Center

Description: Area centroid of a simple polygon with ordered boundary vertices and nonzero area; either orientation works. Zero-area input divides by zero; choose a problem-specific point/segment centroid separately, not the area formula.

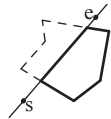
```
Usage: vector<P> p={{0,0},{3,0},{0,3}};
P center=polygonCenter(p); // (1,1)
Time:  $\mathcal{O}(n)$ 
"Point.h"
```

```
typedef Point<double> P;
P polygonCenter(const vector<P> &polygon) {
    P weightedCenter(0, 0);
    double area2 = 0;
    for (int i = 0, j = sz(polygon) - 1; i < sz(polygon); j = i
        ++i) {
        weightedCenter = weightedCenter + (polygon[i] + polygon[j])
            * polygon[j].cross(polygon[i]);
        area2 += polygon[j].cross(polygon[i]);
    }
    return weightedCenter / area2 / 3;
}
```

Polygon Cut

Description:

Returns a vector with the vertices of a polygon with every-thing to the left of the line going from s to e cut away.



Use a convex polygon when a single output polygon is required. The implementation keeps the strictly right side of start->finish, discarding points exactly on the line; it inserts crossing points. The cutting line must have distinct endpoints. The result may be empty. To retain boundary-only pieces, use <=0 in both side tests; reverse the line to keep the left.

```
Usage: auto clipped=polygonCut(polygon,P(0,0),P(1,0));
// Keep y<0 plus crossing points.
"Point.h"
```

```
typedef Point<double> P;
vector<P> polygonCut(const vector<P> &polygon, P start, P
    finish) {
    vector<P> clipped;
    for (int i = 0; i < (sz(polygon)); ++i) {
        P cur = polygon[i], prev = i ? polygon[i - 1] : polygon.
            back();
        auto sideCurrent = start.cross(finish, cur), sidePrevious =
            start.cross(finish, prev);
        if ((sideCurrent < 0) != (sidePrevious < 0))
            clipped.push_back(cur + (prev - cur) * (sideCurrent / (
                sideCurrent - sidePrevious)));
        if (sideCurrent < 0) clipped.push_back(cur);
    }
    return clipped;
}
```

Convex Hull

Description:

Returns extreme vertices in CCW order, omitting interior collinear boundary points. Input is copied and sorted; empty/single-point inputs work, and collinear input reduces to endpoints. Cross products must fit ll. To keep all boundary points, deduplicate, change <=0 to <0, and handle the all-collinear case separately (return sorted unique points).



```
Usage: vector<P> points={{0,0},{2,0},{0,2},{1,0}};
auto hull=convexHull(points); // (1,0) is omitted
Time:  $\mathcal{O}(n \log n)$ 
"Point.h"
```

```
typedef Point<ll> P;
vector<P> convexHull(vector<P> points) {
    if (sz(points) <= 1) return points;
    sort(all(points));
    vector<P> hull(sz(points) + 1);
    int chainStart = 0, hullSize = 0;
    for (int pass = 2; pass--; chainStart = --hullSize, reverse(
        all(points)))
        for (P p : points) {
            while (hullSize >= chainStart + 2 && hull[hullSize - 2].
                cross(hull[hullSize - 1], p) <= 0)
                hullSize--;
            hull[hullSize++] = p;
        }
    return {hull.begin(), hull.begin() + hullSize - (hullSize ==
        2 && hull[0] == hull[1])};
}
```

Convex Hull Diameter

Description: Returns the two points with max distance on a convex hull (ccw, no duplicate/collinear points). Use rotating calipers after convexHull. The hull must be nonempty; a single point returns that point twice. Returns endpoints, so compute dist2 of their difference for an exact squared diameter. Do not pass an unordered point cloud.

```
Usage: auto hull=convexHull(points); auto
ends=hullDiameter(hull);
ll distance2=(ends[0]-ends[1]).dist2();
```

Time: $\mathcal{O}(n)$
"Point.h"
<pre>typedef Point<ll> P; array<P, 2> hullDiameter(vector<P> hull) { int n = sz(hull), j = n < 2 ? 0 : 1; pair<ll, array<P, 2>> best({0, {hull[0], hull[0]}}); for (int i = 0; i < (j); ++i) for (; j = (j + 1) % n) { best = max(best, {(hull[i] - hull[j]).dist2(), {hull[i], hull[j]}}); if ((hull[(j + 1) % n] - hull[j]).cross(hull[i + 1] - hull[i]) >= 0) break; } return best.second; }</pre>

Point in Convex Polygon

Description: Determine whether a point t lies inside a convex hull (CCW order, with no collinear points). Returns true if point lies within the hull. If strict is true, points on the boundary aren't included. The hull must be nonempty, convex, and in boundary order, without interior collinear vertices; convexHull produces suitable input. Use for many containment queries rather than scanning every edge. strict=false includes edges and vertices; hulls of size 1/2 work. Return false before calling for an empty hull.
Usage: auto hull=convexHull(points);
bool inside=inHull(hull,Point<ll>(2,3),false);
Time: $\mathcal{O}(\log N)$
"Point.h", "sideOf.h", "OnSegment.h"

<pre>typedef Point<ll> P; bool inHull(const vector<P> &hull, P p, bool strict = true) { int lo = 1, hi = sz(hull) - 1, boundaryAllowed = !strict; if (sz(hull) < 3) return boundaryAllowed && onSegment(hull [0], hull.back(), p); if (sideOf(hull[0], hull[lo], hull[hi]) > 0) swap(lo, hi); if (sideOf(hull[0], hull[lo], p) >= boundaryAllowed sideOf(hull[0], hull[hi], p) <= -boundaryAllowed) return false; while (abs(lo - hi) > 1) { int mid = (lo + hi) / 2; (sideOf(hull[0], hull[mid], p) > 0 ? hi : lo) = mid; } return sgn(hull[lo].cross(hull[hi], p)) < boundaryAllowed; }</pre>

Line-Convex Hull Intersection

Description: Line-convex polygon intersection. The polygon must be ccw and have no collinear points. lineHull(line, poly) returns a pair describing the intersection of a line with the polygon: $\bullet(-1, -1)$ if no collision, $\bullet(i, -1)$ if touching the corner i , $\bullet(i, i)$ if along side $(i, i + 1)$, $\bullet(i, j)$ if crossing sides $(i, i + 1)$ and $(j, j + 1)$. In the last case, if a corner i is crossed, this is treated as happening on side $(i, i + 1)$. The points are returned in the same order as the line hits the polygon. extrVertex returns the index of the hull point with maximum projection onto a direction. Use a nondegenerate infinite line and a strictly convex counterclockwise hull with at least three vertices. Results are edge/vertex indices, not coordinates; wrap $i+1$ modulo $\text{hull.size}()$. Intersect reported sides with the line to get coordinates. extrVertex returns the index maximizing $\text{dot}(\text{direction}, \text{point})$; direction must be nonzero. Hulls of size <3 need point/segment intersection checks instead.
Usage: auto hit=lineHull(Point<ll>(0,0),Point<ll>(1,0),hull);
int rightmost=extrVertex(hull,Point<ll>(1,0));
Time: $\mathcal{O}(\log n)$
"Point.h"

<pre>#define cmp(i, j) sgn(direction.perp().cross(hull[(i)%n]-hull[(j)%n])) #define extr(i) cmp(i + 1, i) >= 0 && cmp(i, i - 1 + n) < 0 template <class P> int extrVertex(vector<P> &hull, P direction) { int n = sz(hull), lo = 0, hi = n; if (extr(0)) return 0;</pre>

<pre>while (lo + 1 < hi) { int mid = (lo + hi) / 2; if (extr(mid)) return mid; int loTrend = cmp(lo + 1, lo), midTrend = cmp(mid + 1, mid) ; (loTrend < midTrend (loTrend == midTrend && loTrend == cmp(lo, mid)) ? hi : lo) = mid; } return lo; } #define cmpL(i) sgn(a.cross(hull[i], b)) template <class P> array<int, 2> lineHull(P a, P b, vector<P> & hull) { int maxVertex = extrVertex(hull, (a - b).perp()); int minVertex = extrVertex(hull, (b - a).perp()); if (cmpL(maxVertex) < 0 cmpL(minVertex) > 0) return {-1, -1}; array<int, 2> intersections; for (int i = 0; i < (2); ++i) { int lo = minVertex, hi = maxVertex, n = sz(hull); while ((lo + 1) % n != hi) { int mid = ((lo + hi + (lo < hi ? 0 : n)) / 2) % n; (cmpL(mid) == cmpL(minVertex) ? lo : hi) = mid; } intersections[i] = (lo + !cmpL(hi)) % n; swap(maxVertex, minVertex); } if (intersections[0] == intersections[1]) return { intersections[0], -1}; if (!cmpL(intersections[0]) && !cmpL(intersections[1])) switch ((intersections[0] - intersections[1] + sz(hull) + 1) % sz(hull)) { case 0: return {intersections[0], intersections[0]}; case 2: return {intersections[1], intersections[1]}; } return intersections; }</pre>
--

8.4 Misc. Point Set Problems

Closest Pair

Description: Finds the closest pair of points. Input must contain at least two points. Returns the endpoints rather than their distance; it copies and sorts the input internally. Duplicate points give distance zero. Integer squared distances must fit ll.
Usage: auto [a,b]=closest(points);
ll distance2=(a-b).dist2();
Time: $\mathcal{O}(n \log n)$
"Point.h"

<pre>typedef Point<ll> P; pair<P, P> closest(vector<P> points) { assert(sz(points) > 1); set<P> active; sort(all(points), [](P a, P b) { return a.y < b.y; }); pair<ll, pair<P, P>> best{LLONG_MAX, {P(), P()}}; int j = 0; for (P p : points) { P window{1 + (ll)sqrt(best.first), 0}; while (points[j].y <= p.y - window.x) active.erase(points[j ++]); auto lo = active.lower_bound(p - window), hi = active. upper_bound(p + window); for (; lo != hi; ++lo) best = min(best, {(*lo - p).dist2(), { *lo, p }}); active.insert(p); } return best.second;</pre>

}

KD-Tree Nearest Neighbour

Description: KD-tree (2D; see below for higher dimensions). Static nearest-neighbor queries on a nonempty set of integer points. nearest returns (squared distance,point); the query point itself is allowed if present. Bounding boxes prune search, but worst-case queries can visit every point. Uncomment the leaf check to exclude equal points. To exclude only the same input item but allow duplicates, store IDs instead. No insertions/deletions; free allocated nodes between large repeated cases. Distances must fit T. For 3D, 4D, etc., set a fixed dimension D and replace Point.h/P with array<T,D>. Replace the x/y bounds with arrays lo/hi, initialized to INF/-INF, and update all D coordinates. Split on the coordinate k with largest $hi[k]-lo[k]$; sort with $a[k] < b[k]$. In distance(), clamp each $p[k]$ to $[lo[k],hi[k]]$ and sum the squared coordinate differences. At a leaf, replace (p-point).dist2() with the sum of $(p[k]-\text{point}[k])^2$ squared over all k. array supplies the lexicographic comparison needed for ties; construct queries as P3,7,2 for D=3 (four coordinates for D=4, etc.). Search/nearest otherwise stay unchanged. Pruning becomes less effective as D grows; logarithmic query time is not guaranteed.
Usage: KDTree tree(points);
auto [distance2,nearestPoint]=tree.nearest(P(3,7));
Time: Build $\mathcal{O}(n \log^2 n)$; query $\mathcal{O}(n)$ worst case, often near $\mathcal{O}(\log n)$ in low dimensions. For D dimensions: build $\mathcal{O}(n \log^2 n + Dn \log n)$, query $\mathcal{O}(Dn)$ worst case; space $\mathcal{O}(Dn)$.
"Point.h"

<pre>typedef long long T; typedef Point<T> P; const T INF = numeric_limits<T>::max(); bool on_x(P a, P b) { return a.x < b.x; } bool on_y(P a, P b) { return a.y < b.y; } struct KNode { P point; this is a leaf, the single point in it T minX = INF, maxX = -INF, minY = INF, maxY = -INF; // bounds KNode *first = 0, *second = 0; T distance(P p) { // min squared distance to a point T closestX = (p.x < minX ? minX : p.x > maxX ? maxX : p.x); T closestY = (p.y < minY ? minY : p.y > maxY ? maxY : p.y); return (P(closestX, closestY) - p).dist2(); } KNode(vector<P> &points) : point(points[0]) { for (P p : points) { minX = min(minX, p.x); maxX = max(maxX, p.x); minY = min(minY, p.y); maxY = max(maxY, p.y); } if (points.size() > 1) { // split on x if width >= height (not ideal...) sort(all(points), maxX - minX >= maxY - minY ? on_x : on_y); // divide by taking half the array for each child (not // best performance with many duplicates in the middle) int half = sz(points) / 2; first = new KNode({points.begin(), points.begin() + half }); second = new KNode({points.begin() + half, points.end()}) ; } } }; struct KDTree { KNode *root; KDTree(const vector<P> &points) : root(new KNode({all(points }))) {} pair<T, P> search(KNode *node, P p) { if (!node->first) { // uncomment if we should not find the point itself: // if (p == node->point) return {INF, P()};</pre>

```
        return make_pair((p - node->point).dist2(), node->point);
    }

    KNode *nearChild = node->first, *farChild = node->second;
    T nearDistance = nearChild->distance(p), farDistance =
        farChild->distance(p);
    if (nearDistance > farDistance) swap(farDistance,
        nearDistance), swap(nearChild, farChild);

    // search closest side first, other side if needed
    auto best = search(nearChild, p);
    if (farDistance < best.first) best = min(best, search(
        farChild, p));
    return best;
}
// find nearest point to a point, and its squared distance
// (requires an arbitrary operator< for Point)
pair<T, P> nearest(P p) { return search(root, p); }
```

Strings (9)

Trie (Pointers)

Description: Pointer trie for lowercase a..z, with duplicate strings. end-Count counts strings ending at a node; passCount counts strings with that prefix (including exact matches). addString inserts one copy; deleteString removes one copy, doing nothing if absent. Unused nodes are freed during deletion; the destructor frees the remaining trie. Empty strings are supported; countPrefix("") counts all inserted copies. Change the alphabet size and character mapping for other symbols. Recursive deletion/destruction may use stack depth equal to the longest string. Copying is disabled because the trie owns its pointers.

Usage: Trie trie; trie.addString("abc"); trie.addString("abc"); trie.addString("abd"); bool found=trie.findString("abc"); // true; a prefix alone is not an exact match

int count=trie.countPrefix("ab"); // 3, including duplicates trie.deleteString("abc"); // one copy remains

// Use a fresh object per test case; its destructor releases the nodes.

Time: $\mathcal{O}(L)$ per operation for string length L; $\mathcal{O}(26S)$ memory for S distinct prefixes.

```
struct Trie {
    struct Node {
        Node *child[26] = {};
        int endCount = 0, passCount = 0;
        ~Node() {
            for (Node *next : child) delete next;
        }
    };
    Node *root = new Node();
    Trie() = default;
    Trie(const Trie &) = delete;
    Trie &operator=(const Trie &) = delete;
    ~Trie() { delete root; }
    void addString(const string &s) {
        Node *node = root;
        node->passCount++;
        for (char ch : s) {
            int letter = ch - 'a';
            if (!node->child[letter]) node->child[letter] = new Node
                ();
            node = node->child[letter];
            node->passCount++;
        }
        node->endCount++;
    }
};
```

```
Node *findNode(const string &s) const {
    Node *node = root;
    for (char ch : s) {
        node = node->child[ch - 'a'];
        if (!node) return nullptr;
    }
    return node;
}

bool findString(const string &s) const {
    Node *node = findNode(s);
    return node && node->endCount;
}

int countPrefix(const string &s) const {
    Node *node = findNode(s);
    return node ? node->passCount : 0;
}

void deleteString(Node *&node, const string &s, int pos) {
    node->passCount--;
    if (pos == sz(s))
        node->endCount--;
    else
        deleteString(node->child[s[pos] - 'a'], s, pos + 1);
    if (node != root && node->passCount == 0) {
        delete node;
        node = nullptr; // Detach the deleted child from its
            parent.
    }
}

void deleteString(const string &s) {
    if (findString(s)) deleteString(root, s, 0);
}
};
```

Knuth-Morris-Pratt (KMP)

Description: kmpSearch(text,pattern) returns all 0-based match starts, including overlaps. Empty pattern matches positions 0.text.size(); no separator is needed. pi[i] from prefixFunction is the longest proper border of text[0..i].

Usage: auto positions=kmpSearch("ababa","aba"); // {0,2}

auto pi=prefixFunction("abab"); // {0,0,1,2}

Time: $\mathcal{O}(|text|+|pattern|)$ time; $\mathcal{O}(|pattern|)$ auxiliary memory.

```
vector<int> prefixFunction(const string &text) {
    int n = sz(text);
    vector<int> pi(n);
    for (int i = 1; i < n; i++) {
        int j = pi[i - 1];
        while (j > 0 && text[i] != text[j]) j = pi[j - 1];
        if (text[i] == text[j]) j++;
        pi[i] = j;
    }
    return pi;
}

vector<int> kmpSearch(const string &text, const string &pattern
) {
    vector<int> positions;
    int patternLength = sz(pattern);
    if (patternLength == 0) {
        for (int i = 0; i <= sz(text); i++) positions.push_back(i);
        return positions;
    }
    vector<int> prefix = prefixFunction(pattern);
    int matched = 0;
    for (int i = 0; i < sz(text); i++) {
        while (matched > 0 && text[i] != pattern[matched]) matched
            = prefix[matched - 1];
        if (text[i] == pattern[matched]) matched++;
        if (matched == patternLength) {
            positions.push_back(i - patternLength + 1);
        }
    }
}
```

```
        // Keep the longest border so overlapping matches are
        found.
        matched = prefix[matched - 1];
    }
}

return positions;
}
```

Z-Function

Description: z[i] is the common-prefix length of the string and its suffix starting at i; z[0]=0. For matching use pattern+separator+text, with a separator absent from both, and look for z[i]>=pattern.size(). If no separator symbol is available, use KMP; handle empty-pattern matching explicitly (all starts 0..text.size()).

Usage: auto z=zFunction("abacaba"); // {0,0,1,0,3,0,1}

Time: $\mathcal{O}(N)$ time and memory.

```
vector<int> zFunction(const string &text) {
    int n = sz(text);
    vector<int> z(n, 0);
    int left = 0;
    int right = -1;
    for (int i = 1; i < n; i++) {
        // [l,r] is the rightmost segment known to match a prefix
        of s.
        if (i <= right) z[i] = min(right - i + 1, z[i - left]);
        while (i + z[i] < n && text[z[i]] == text[i + z[i]]) z[i]
            ++;
        if (i + z[i] - 1 > right) {
            left = i;
            right = i + z[i] - 1;
        }
    }
    return z;
}
```

Manacher Palindromes

Description: For each position in a string, computes p[0][i] = half length of longest even palindrome around pos i, p[1][i] = longest odd (half rounded down). Indices are 0-based. Odd center i has length 2*p[1][i]+1 and covers [i-p[1][i],i+p[1][i]]. Even center i is the gap before s[i]: length 2*p[0][i], covering [i-p[0][i],i+p[0][i]-1]. The extra even entry at i=n is zero. Radii permit constant-time palindrome checks after construction.

Usage: auto p=manacher("abba");

int evenLength=2*p[0][2]; // 4

Time: $\mathcal{O}(N)$

```
array<vector<int>, 2> manacher(const string &s) {
    int n = sz(s);
    array<vector<int>, 2> radius = {vector<int>(n + 1), vector<
        int>(n)};
    for (int parity = 0; parity < (2); ++parity)
        for (int i = 0, windowLeft = 0, windowRight = 0; i < n; i
            ++){
            int remaining = windowRight - i + !parity;
            if (i < windowRight)
                radius[parity][i] = min(remaining, radius[parity][
                    windowLeft + remaining]);
            int left = i - radius[parity][i], right = i + radius[
                parity][i] - !parity;
            while (left >= 1 && right + 1 < n && s[left - 1] == s[
                right + 1])
                radius[parity][i]++, left--, right++;
            if (right > windowRight) windowLeft = left, windowRight =
                right;
        }
    return radius;
}
```

Minimum String Rotation

Description: Returns the 0-based start of a lexicographically smallest cyclic rotation, not the rotated string. Periodic input may have equivalent starts; empty input returns 0.

Usage: string s="baca"; int start=minRotation(s); rotate(s.begin(),s.begin()+start,s.end()); // "abac"
Time: $\mathcal{O}(N)$

```
int minRotation(string s) {
    int bestStart = 0, n = sz(s);
    s += s;
    for (int candidate = 0; candidate < (n); ++candidate)
        for (int offset = 0; offset < (n); ++offset) {
            if (bestStart + offset == candidate || s[bestStart +
                offset] < s[candidate + offset]) {
                candidate += max(0, offset - 1);
                break;
            }
            if (s[bestStart + offset] > s[candidate + offset]) {
                bestStart = candidate;
                break;
            }
        }
    return bestStart;
}
```

Rolling Hash

Description: Polynomial prefix hash with base 31 and modulus 1000000003. getHash uses 1-based inclusive [l,r], although the input string is normal 0-based. Compare equal-length substrings. A single fixed hash can collide: use KMP for exact matching, or two independent moduli when adversarial collisions matter. No updates; expects lowercase a..z. Other alphabets need positive symbol IDs and a suitable base; normalize signed chars. Require $l \leq r \leq n$.

Usage: StringHash hash("ababa");
bool same=hash.getHash(1,3)==hash.getHash(3,5); // both "aba"
StringHash pattern("aba");
bool match=hash.getHash(1,3)==pattern.getHash(1,3);
Time: $\mathcal{O}(N)$ construction and memory; $\mathcal{O}(1)$ per substring hash.

```
struct StringHash {
    static constexpr ll base = 31, mod = 1000000003;
    vector<ll> power, prefix;
    StringHash(const string &text) : power(sz(text) + 1), prefix(
        sz(text) + 1) {
        power[0] = 1;
        for (int i = 1; i <= sz(text); i++) {
            power[i] = power[i - 1] * base % mod;
            prefix[i] = (prefix[i - 1] * base + text[i - 1] - 'a' +
                1) % mod;
        }
    }
    ll getHash(int l, int r) const {
        return (prefix[r] - prefix[l - 1] * power[r - l + 1] % mod
            + mod) % mod;
    }
};
```

Various (10)

10.1 Intervals

Interval Union and Removal

Description: Add and remove intervals from a set of disjoint intervals. Will merge the added interval with any overlapping intervals in the set when adding. Intervals are [inclusive, exclusive). Maintain covered integer-coordinate ranges rather than individual points. Touching intervals merge too: [1,3) and [3,5) become [1,5). removeInterval can split an interval. Use $l \leq r$; empty intervals have no effect. A call touching k stored intervals costs $\mathcal{O}(\log N + k)$, because those intervals must be erased.

```
Usage: set<pii> intervals; addInterval(intervals,1,5);
removeInterval(intervals,2,4); // {[1,2],[4,5]}
Time:  $\mathcal{O}(\log N)$  amortized;  $\mathcal{O}(\log N + k)$  for a call erasing k intervals.
set<pii>::iterator addInterval(set<pii> &intervals, int l, int
    r) {
    if (l == r) return intervals.end();
    auto it = intervals.lower_bound({l, r}), hint = it;
    while (it != intervals.end() && it->first <= r) {
        r = max(r, it->second);
        hint = it = intervals.erase(it);
    }
    if (it != intervals.begin() && (--it)->second >= l) {
        l = min(l, it->first);
        r = max(r, it->second);
        intervals.erase(it);
    }
    return intervals.insert(hint, {l, r});
}
void removeInterval(set<pii> &intervals, int l, int r) {
    if (l == r) return;
    auto it = addInterval(intervals, l, r);
    auto oldRight = it->second;
    if (it->first == l)
        intervals.erase(it);
    else
        (int &)it->second = l;
    if (r != oldRight) intervals.emplace(r, oldRight);
}
```

Minimum Interval Cover

Description: Compute indices of smallest set of intervals covering another interval. Intervals should be [inclusive, exclusive). To support [inclusive, inclusive], change (A) to add || chosen.empty(). Returns empty set on failure (or if G is empty). A greedy sweep chooses the eligible interval reaching farthest right. Output is 0-based indices into the original input, not its sorted order. Minimizes interval count rather than total cost. Adjacent intervals may meet at an endpoint; a gap causes failure. Check whether the target is empty before interpreting an empty result as failure.

Usage: vector<pii> intervals={{0,3},{2,5},{0,1}};
auto chosen=cover(pii(0,5),intervals); // {0,1}
Time: $\mathcal{O}(N \log N)$

```
template <class T> vector<int> cover(pair<T, T> target, vector<
    pair<T, T>> intervals) {
    vector<int> order(sz(intervals)), chosen;
    iota(all(order), 0);
    sort(all(order), [&](int a, int b) { return intervals[a] <
        intervals[b]; });
    T coveredTo = target.first;
    int nextInterval = 0;
    while (coveredTo < target.second) { // (A)
        pair<T, int> farthest = make_pair(coveredTo, -1);
        while (nextInterval < sz(intervals) && intervals[order[
            nextInterval]].first <= coveredTo) {
            farthest =
                max(farthest, make_pair(intervals[order[nextInterval
                    ]].second, order[nextInterval]));
            nextInterval++;
        }
        if (farthest.second == -1) return {};
        coveredTo = farthest.first;
        chosen.push_back(farthest.second);
    }
    return chosen;
}
```

Constant-Value Interval Search

Description: Split a monotone function on [from, to) into a minimal set of half-open intervals on which it has the same value. Runs a callback g for each such interval. Use when evaluating a monotone integer-indexed function is expensive and it changes value only a few times. visitInterval receives (lo,hi,value) for each maximal [lo,hi). Increasing/decreasing functions both work. A value must not disappear then return, or equal endpoints can hide changes. valueAt must be deterministic.

Usage: constantIntervals(0,10,[](int i) { return i/3; },
 [](int lo,int hi,int value) { ... });
Time: $\mathcal{O}(k \log \frac{n}{k})$

```
template <class F, class G, class T>
void rec(int from, int to, F &valueAt, G &visitInterval, int &
    intervalStart, T &currentValue,
    T endValue) {
    if (currentValue == endValue) return;
    if (from == to) {
        visitInterval(intervalStart, to, currentValue);
        intervalStart = to;
        currentValue = endValue;
    } else {
        int mid = (from + to) >> 1;
        rec(from, mid, valueAt, visitInterval, intervalStart,
            currentValue, valueAt(mid));
        rec(mid + 1, to, valueAt, visitInterval, intervalStart,
            currentValue, endValue);
    }
}
template <class F, class G> void constantIntervals(int from,
    int to, F valueAt, G visitInterval) {
    if (to <= from) return;
    int intervalStart = from;
    auto currentValue = valueAt(intervalStart), endValue =
        valueAt(to - 1);
    rec(from, to - 1, valueAt, visitInterval, intervalStart,
        currentValue, endValue);
    visitInterval(intervalStart, to, endValue);
}
```

10.2 Misc. algorithms

Ternary Search

Description: Integer version returns the smallest maximizing index on inclusive [lo,hi]. Require a function that rises strictly before its maximum plateau, then never rises again. Double version returns an approximate maximizing position; require a continuous function strictly increasing before its maximum plateau and strictly decreasing after it. After 100 iterations, interval width is $(hi - lo)(2/3)^{100}$, subject to floating-point rounding. To minimize, reverse (A) and (B) for integers, or (C) for doubles. Use double bounds to select the double version; evaluate valueAt at the returned position to get the value.

Usage: int best=ternSearch(0,10,[](int i) { return
 -(i-4)*(i-4); });
// best=4
double x=ternSearch(0.0,10.0,[](double x) { return
 -(x-4.2)*(x-4.2); });
// x is approximately 4.2
Time: Integers: $\mathcal{O}(\log(hi - lo + 1))$; doubles: 100 iterations (200 evalua-
 tions).

```
template <class F> int ternSearch(int lo, int hi, F valueAt) {
    assert(lo <= hi);
    while ((ll)hi - lo >= 5) {
        int mid = (int)((ll)lo + ((ll)hi - lo) / 2);
        if (valueAt(mid) < valueAt(mid + 1))
            lo = mid; // (A)
        else
            hi = mid + 1;
    }
    for (int i = lo; i < hi;) {
```

```
++i;
    if (valueAt(lo) < valueAt(i)) lo = i; // (B)
}
return lo;
}

template <class F> double ternSearch(double lo, double hi, F
    valueAt) {
    assert(lo <= hi);
    for (int i = 0; i < 100; i++) {
        double x1 = lo + (hi - lo) / 3, x2 = hi - (hi - lo) / 3;
        if (valueAt(x1) > valueAt(x2))
            hi = x2; // (C)
        else
            lo = x1;
    }
    return lo + (hi - lo) / 2;
}
```

Fast Subset Sum (Knapsack)

Description: Given N non-negative integer weights w and a non-negative target t, computes the maximum S <= t such that S is the sum of some subset of the weights. This is 0/1 subset sum with bounded weights: each weight is used at most once and value equals weight. Returns only the best achievable sum, not chosen items. Weights/capacity must be nonnegative and sums must fit int. Useful when maxWeight is small even if capacity is large. This is not general profit/weight knapsack.

Usage: int best=knapsack({3,5,7},10); // 10, using 3+7

Time: $\mathcal{O}(N \max(w_i))$

```
int knapsack(vector<int> weights, int capacity) {
    int total = 0, prefixSize = 0, state;
    while (prefixSize < sz(weights) && total + weights[prefixSize]
        ] <= capacity)
        total += weights[prefixSize++];
    if (prefixSize == sz(weights)) return total;
    int maxWeight = *max_element(all(weights));
    vector<int> previous, dp(2 * maxWeight, -1);
    dp[total + maxWeight - capacity] = prefixSize;
    for (int i = prefixSize; i < (sz(weights)); ++i) {
        previous = dp;
        for (int state = 0; state < (maxWeight); ++state)
            dp[state + weights[i]] = max(dp[state + weights[i]],
                previous[state]);
        for (state = 2 * maxWeight; --state > maxWeight;)
            for (int j = max(0, previous[state]); j < (dp[state]); ++j)
                dp[state - weights[j]] = max(dp[state - weights[j]], j)
            ;
    }
    for (total = capacity; dp[total + maxWeight - capacity] < 0;
        total--);
    return total;
}
```

10.3 Dynamic programming

SOS DP: Subsets and Supersets

Description: In-place sums over subsets/supersets, including the mask itself and zero. dp has exactly 2^B entries (B>=0); copy the input separately for each transform. For frequencies, sub[full XOR mask] counts disjoint input masks. inverse subtracts to restore input. Keep the bit loop outermost. Intermediate sums must fit ll; normalize modular results. Min/max variants have no subtraction inverse. An empty vector is invalid; B=0 needs one entry.

Usage: vector<ll> freq={1,2,3,4}; // masks 0,1,2,3
auto sub=freq; subsetSOS(sub); // {1,3,4,10}
auto super=freq; supersetSOS(super); // {10,6,7,4}
int full=sz(freq)-1, mask=1;
ll disjoint=sub[full^mask]; // 4
subsetSOS(sub,true); // restores freq

```
Time:  $\mathcal{O}(B2^B)$  time;  $\mathcal{O}(1)$  auxiliary memory.

void subsetSOS(vector<ll> &dp, bool inverse = false) {
    int bits = __lg(sz(dp));
    for (int bit = 0; bit < bits; bit++)
        for (int mask = 0; mask < sz(dp); mask++)
            if (mask >> bit & 1) dp[mask] += (inverse ? -1 : 1) * dp[
                mask ^ (1 << bit)];
}

void supersetSOS(vector<ll> &dp, bool inverse = false) {
    int bits = __lg(sz(dp));
    for (int bit = 0; bit < bits; bit++)
        for (int mask = 0; mask < sz(dp); mask++)
            if (!(mask >> bit & 1)) dp[mask] += (inverse ? -1 : 1) *
                dp[mask | (1 << bit)];
}
```

Knuth DP Optimization

Description: When doing DP on intervals: $a[i][j] = \min_{i < k < j} (a[i][k] + a[k][j]) + f(i, j)$, where the (minimal) optimal k increases with both i and j , one can solve intervals in increasing order of length, and search $k = p[i][j]$ for $a[i][j]$ only between $p[i][j - 1]$ and $p[i + 1][j]$. This is known as Knuth DP. Sufficient criteria for this are if $f(b, c) \leq f(a, d)$ and $f(a, c) + f(b, d) \leq f(a, d) + f(b, c)$ for all $a \leq b \leq c \leq d$. Consider also: CHT (ch. Data structures), monotone queues, ternary search. This is a recurrence recipe, not a callable function. Typical use is optimal merging of adjacent segments with f(i,j) equal to interval sum. Initialize base intervals and optimal split positions, then process increasing lengths. The required monotonicity concerns optimal splits, not DP values; prove it or establish the sufficient inequalities before using the bound.

Usage: // For each increasing interval [i,j], try only splits
// opt[i][j-1]<=k<=opt[i+1][j], also restricted to i<k<j.
// Save dp[i][j] and its smallest minimizing split.

Time: $\mathcal{O}(N^2)$

Divide-and-Conquer DP

Description: Given $a[i] = \min_{l_0(i) \leq k < h_1(i)} (f(i, k))$ where the (minimal) optimal k increases with i , computes $a[i]$ for $i = L..R - 1$. Customization skeleton for one DP layer. Replace lo/hi with valid split bounds, f with previousLayer[split]+cost(split,index), and store with destination arrays (pair<int,ll> for answer). solve(l,r) covers [l,r). Each index needs at least one candidate; optimal split indices must be nondecreasing. Keep the previous layer fixed. Skip unreachable previous states to avoid INF+cost overflow; use a full split search if optimal indices are not monotone.

Usage: // Implement lo,hi,f,store and declare DP arrays first.
DivideConquerDP solver; solver.solve(l,n+1); // indices 1..n

Time: $\mathcal{O}((N + (hi - lo)) \log N)$

```
struct DivideConquerDP { // Modify at will:
    int lo(int index) { return 0; }
    int hi(int index) { return index; }
    ll f(int index, int split) { return dp[index][split]; }
    void store(int index, int split, ll value) { answer[index] =
        make_pair(split, value); }
    void rec(int l, int r, int optL, int optR) {
        if (l >= r) return;
        int mid = (l + r) >> 1;
        pair<ll, int> best(LLONG_MAX, optL);
        for (int split = max(optL, lo(mid)); split < (min(optR, hi(
            mid))); ++split)
            best = min(best, make_pair(f(mid, split), split));
        store(mid, best.second, best.first);
        rec(l, mid, optL, best.second + 1);
        rec(mid + 1, r, best.second, optR);
    }
    void solve(int l, int r) { rec(l, r, INT_MIN, INT_MAX); }
};
```

10.4 Optimization tricks

__builtin_ia32_ldmxcsr(40896); disables denormals (which make floats 20x slower near their minimum value).

10.4.1 Bit hacks

- x & -x is the least bit in x.
- for (int x = m; x;) { --x &= m; ... } loops over all subset masks of m (except m itself).
- c = x&-x, r = x+c; (((r^x) >> 2)/c) | r is the next number after x with the same number of bits set.

10.4.2 Pragmas

- #pragma GCC optimize ("Ofast") will make GCC auto-vectorize loops and optimizes floating points better.
- #pragma GCC target ("avx2") can double performance of vectorized code, but causes crashes on old machines.
- #pragma GCC optimize ("trapv") kills the program on integer overflows (but is really slow).